LPIC-3 · EXAM 303-300

Security

A Complete Study Guide

SECOND EDITION



Topics 331 – 335 · All exam objectives in one volume

WHAT THIS BOOK COVERS

331	332	333	334	335
Cryptography	Host Security	Access Control	Network Security	Threats & Vulnerability Assessment

Every objective explained from first principles, with worked command examples, 34 custom diagrams, hands-on labs, reference tables and exam tips.

Omid Zeynalpour

2026

Independent study guide. Not affiliated with or endorsed by the Linux Professional Institute (LPI). LPIC is a trademark of LPI.

Contents

Topic 331 — Cryptography **3**
331.1 X.509 Certificates and Public Key Infrastructure 4
331.2 X.509 Certificates for Encryption, Signing and Authentication 38
331.3 Encrypted File Systems 54
331.4 DNS and Cryptography 67

Topic 332 — Host Security **79**
332.1 Host Hardening 80
332.2 Host Intrusion Detection 91
332.3 Resource Control 97

Topic 333 — Access Control **101**
333.1 Discretionary Access Control (DAC) 102
333.2 Mandatory Access Control / SELinux 106

Topic 334 — Network Security **112**
334.1 Network Hardening 113
334.2 Network Intrusion Detection 116
334.3 Packet Filtering 118
334.4 Virtual Private Networks 122

Topic 335 — Threats & Vulnerability Assessment **125**
335.1 Common Security Vulnerabilities and Threats 126
335.2 Penetration Testing 128

Entries are clickable — in Word, LibreOffice or most PDF viewers, click a title to jump to it. Page numbers appear at the foot of every page.

Topic 331 — Cryptography

331.1 X.509 Certificates and Public Key Infrastructure

Objective weight: 5

Almost every secure connection you make — an HTTPS website, a signed email, a VPN, a client logging in with a smart card — rests on the same foundation: X.509 certificates and the Public Key Infrastructure (PKI) that issues and manages them. A certificate is a small, signed file that binds a public key to an identity (a hostname, an organisation, a person), so that anyone can verify "this key really belongs to that name" by checking the signature of a trusted authority. This chapter builds that picture from the ground up, then works through the practical tools — OpenSSL above all — used to create, sign, inspect, revoke and automate certificates, finishing with a three-tier hands-on lab.

1. What an X.509 Certificate Is

An X.509 certificate is a structured document (encoded in a format called ASN.1) that a Certificate Authority signs. At its heart it says: "I, the issuer, certify that this public key belongs to this subject, for this period of time, to be used for these purposes." The certificate itself is public — it contains no secrets — and can be handed to anyone. Its trustworthiness comes entirely from the issuer's signature over its contents.

The main fields

Field	What it holds
`Version`	The X.509 version — v3 is universal today (it added extensions).
`Serial Number`	A unique number the CA assigns; used to track and revoke the certificate.
`Signature Algorithm`	The algorithm the CA used to sign, e.g. sha256WithRSAEncryption.
`Issuer`	The Distinguished Name of the CA that signed it.
`Validity`	The Not Before / Not After dates between which it is valid.
`Subject`	The Distinguished Name of the entity the certificate identifies.
`Subject Public Key Info`	The subject's public key and its algorithm (RSA, EC…).
`Extensions (v3)`	Constraints and usage rules — the subject of Section 2.
`Signature`	The issuer's digital signature over everything above.

The one command you will use constantly is `openssl x509 -text`, which dumps a certificate in human-readable form so you can read every field and extension:

```
$ openssl x509 -in server.crt -text -noout
```

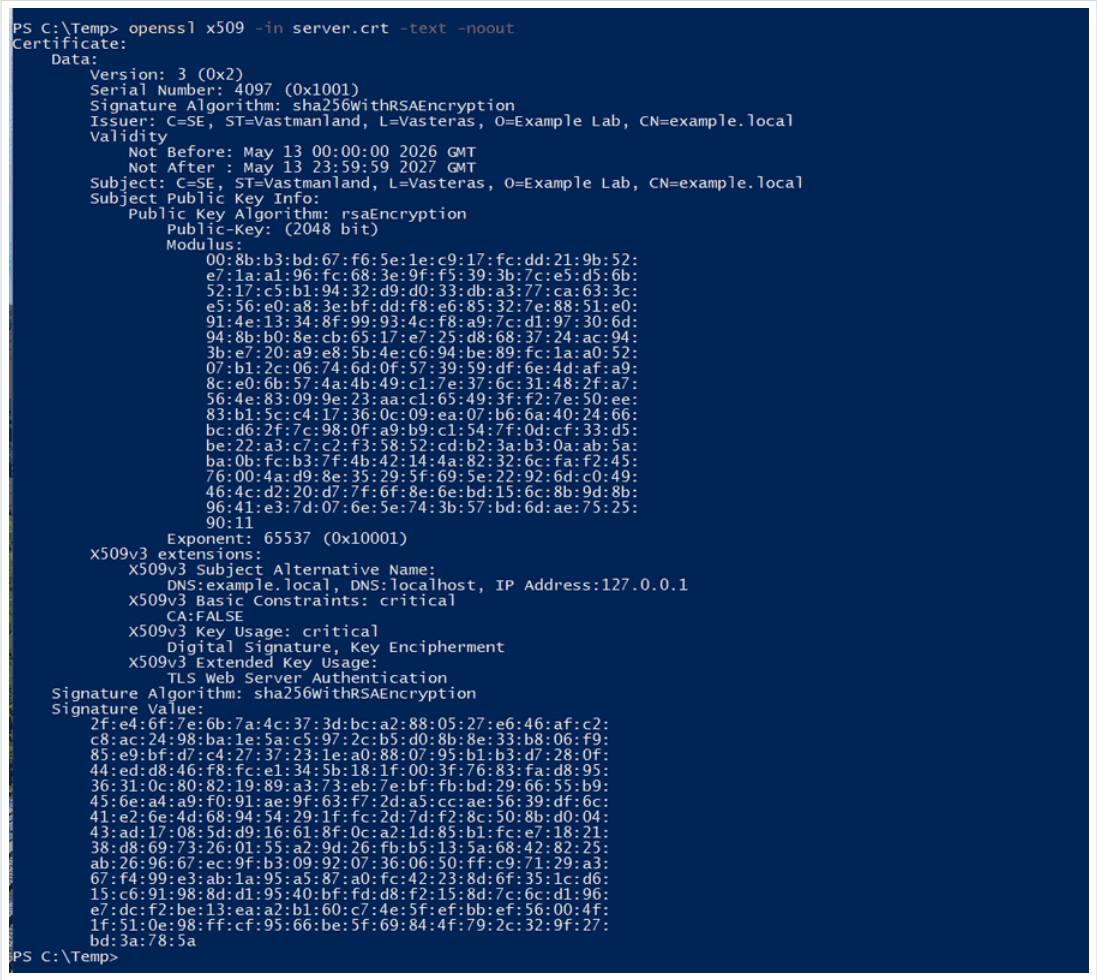


Figure — openssl x509 -text -noout showing every field of a server certificate: version, serial, issuer, validity, subject, the RSA public key, and the v3 extensions.



Figure — inspecting a certificate's fields.

```
[vagrant@lpic ~]$ openssl x509 -in server.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 4097 (0x1001)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = SE, ST = Vastmanland, L = Vasteras, O = Example Lab, CN = example.local
        Validity
            Not Before: May 13 00:00:00 2026 GMT
            Not After : May 13 23:59:59 2027 GMT
        Subject: C = SE, ST = Vastmanland, L = Vasteras, O = Example Lab, CN = example.local
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:8b:b3:bd:67:f6:5e:1e:c9:17:fc:dd:21:9b:52:
                    e7:1a:a1:96:fc:68:3e:9f:f5:39:3b:7c:e5:d5:6b:
                    52:17:c5:b1:94:32:d9:d0:33:db:a3:77:ca:63:3c:
                    e5:56:e0:a8:3e:bf:dd:f8:e6:85:32:7e:88:51:e0:
                    91:4e:13:34:8f:99:93:4c:f8:a9:7c:d1:97:30:6d:
                    94:8b:b0:8e:cb:65:17:e7:25:d8:68:37:24:ac:94:
                    3b:e7:20:a9:e8:5b:4e:c6:94:be:89:fc:1a:a0:52:
                    07:b1:2c:06:74:6d:0f:57:39:59:df:6e:4d:af:a9:
                    8c:e0:6b:57:4a:4b:49:c1:7e:37:6c:31:48:2f:a7:
                    56:4e:83:09:9e:23:aa:c1:65:49:3f:f2:7e:50:ee:
                    83:b1:5c:c4:17:36:0c:09:ea:07:b6:6a:40:24:66:
                    bc:d6:2f:7c:98:0f:a9:b9:c1:54:7f:0d:cf:33:d5:
                    be:22:a3:c7:c2:f3:58:52:cd:b2:3a:b3:0a:ab:5a:
                    ba:0b:fc:b3:7f:4b:42:14:4a:82:32:6c:fa:f2:45:
                    76:00:4a:d9:8e:35:29:5f:69:5e:22:92:6d:c0:49:
                    46:4c:d2:20:d7:7f:6f:8e:6e:bd:15:6c:8b:9d:8b:
                    96:41:e3:7d:07:6e:5e:74:3b:57:bd:6d:ae:75:25:
                    90:11
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Alternative Name:
                DNS:example.local, DNS:localhost, IP Address:127.0.0.1
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        2f:e4:6f:7e:6b:7a:4c:37:3d:bc:a2:88:05:27:e6:46:af:c2:
        c8:ac:24:98:ba:1e:5a:c5:97:2c:b5:d0:8b:8e:33:b8:06:f9:
        85:e9:bf:d7:c4:27:37:23:1e:a0:88:07:95:b1:b3:d7:28:0f:
        44:ed:d8:46:f8:fc:e1:34:5b:18:1f:00:3f:76:83:fa:d8:95:
        36:31:0c:80:82:19:89:a3:73:eb:7e:bf:fb:bd:29:66:55:b9:
        45:6e:a4:a9:f0:91:ae:9f:63:f7:2d:a5:cc:ae:56:39:df:6c:
        41:e2:6e:4d:68:94:54:29:1f:fc:2d:7d:f2:8c:50:8b:d0:04:
        43:ad:17:08:5d:d9:16:61:8f:0c:a2:1d:85:b1:fc:e7:18:21:
        38:d8:69:73:26:01:55:a2:9d:26:fb:b5:13:5a:68:42:82:25:
        ab:26:96:67:ec:9f:b3:09:92:07:36:06:50:ff:c9:71:29:a3:
        67:f4:99:e3:ab:1a:95:a5:87:a0:fc:42:23:8d:6f:35:1c:d6:
        15:c6:91:98:8d:d1:95:40:bf:fd:d8:f2:15:8d:7c:6c:d1:96:
        e7:dc:f2:be:13:ea:a2:b1:60:c7:4e:5f:ef:bb:ef:56:00:4f:
        1f:51:0e:98:ff:cf:95:66:be:5f:69:84:4f:79:2c:32:9f:27:
        bd:3a:78:5a
[vagrant@lpic ~]$
```

Figure — the full certificate structure, including the issuer's signature value.

The key fields, one by one

Serial number — a unique identifier the CA assigns to each certificate it issues. It is the handle used for revocation: when a certificate must be invalidated, the CA publishes its serial number on a revocation list.

Issuer and Subject — both are Distinguished Names (DNs), a set of attributes such as `C` (country), `ST` (state), `L` (locality), `O` (organisation), `OU` (organisational unit) and `CN` (common name). The issuer is the CA; the subject is the entity being identified. Historically the `CN` held the hostname, but modern TLS ignores CN for hostname checks and uses the Subject Alternative Name extension instead (Section 2).

Validity period — the `Not Before` and `Not After` dates. A certificate outside this window (especially an expired one) must not be trusted.

```
$ openssl x509 -in server.crt -noout -dates          # show the validity window
$ openssl x509 -in server.crt -checkend 2592000 -noout   # will it expire within 30
days?
#   (2,592,000 seconds = 30 days)
```

Public key — the subject's public key. The matching private key is never in the certificate. Extract just the public key with:

```
$ openssl x509 -in server.crt -pubkey -noout
```

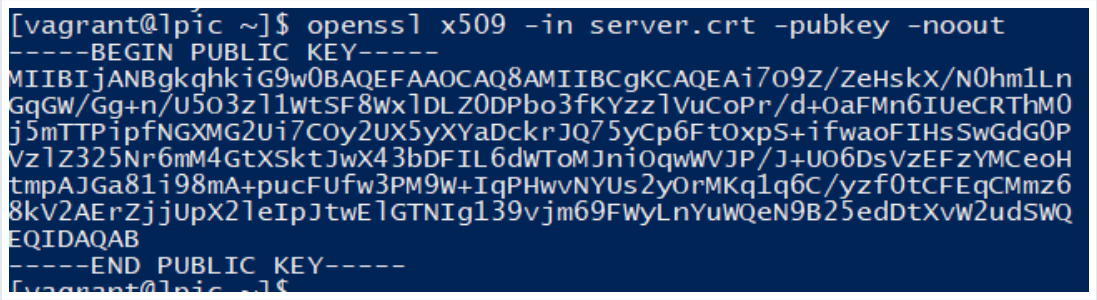


Figure — extracting the public key from a certificate.

Digital signature — created by the issuer over the whole certificate body with the issuer's private key. It guarantees **integrity** (the certificate has not been altered) and **authenticity** (it really was issued by that CA). Clients verify it with the issuer's public key; if it fails, the certificate is rejected.

2. X.509v3 Extensions

Version 3 added extensions, which is where most of a certificate's real behaviour lives: they say whether a certificate may act as a CA, what its key may be used for, which hostnames it covers, and where to check its revocation status. An extension can be marked critical, meaning any system that does not understand it must reject the certificate.

Basic Constraints — is this a CA?

Basic Constraints declares whether a certificate is allowed to sign other certificates. `CA:true` marks a Certificate Authority (a root or intermediate) that may issue and sign; `CA:false` marks an end-entity (leaf) certificate for a server or client, which must never sign others. It is normally critical, and can also cap the chain depth with `pathlen`.

```
basicConstraints = critical, CA:true          # a CA certificate
basicConstraints = critical, CA:false         # a server/client (leaf) certificate
```

Key Usage — what the key may do

Key Usage restricts the cryptographic operations the public key is allowed to perform. The important flags: `digitalSignature` (sign/authenticate), `keyEncipherment` (encrypt a key, as in classic RSA key exchange), `keyCertSign` (sign certificates — the CA power) and `cRLSign` (sign revocation lists).

Certificate type	Typical Key Usage
CA (root / intermediate)	`keyCertSign, cRLSign`
RSA TLS server	`digitalSignature, keyEncipherment`
ECDSA TLS server	`digitalSignature` (key exchange is separate, via ECDHE)

Extended Key Usage (EKU) — where the key may be used

Where Key Usage says *how* a key may be used, **Extended Key Usage** says *where* — the application purpose. Both must be satisfied for a certificate to be accepted.

EKU value	Purpose
`serverAuth`	TLS server (HTTPS).
`clientAuth`	Client authentication.
`OCSPSigning`	Signing OCSP revocation responses.
`codeSigning`	Signing software.
`emailProtection`	Secure email (S/MIME).

The other extensions you must know

Subject Alternative Name (SAN) lists the host names (and IPs) a certificate is valid for — `DNS:www.example.com, DNS:example.com, IP:10.0.0.1`. Modern browsers verify the hostname against the SAN, not the CN, so a server certificate **must** carry a SAN.

Subject Key Identifier (SKI) and Authority Key Identifier (AKI) are hashes that link a certificate to its issuer: the child's AKI matches the parent's SKI, which is how a validator finds the right issuer when building the chain.

CRL Distribution Points tells clients where to download the Certificate Revocation List, and **Authority Information Access (AIA)** gives the URL of the OCSP responder (for live revocation checks) and often the issuer's certificate. Both are essential for revocation to work (Section 8) and for OCSP stapling (covered in 331.2).

3. Generating and Managing Keys

A certificate is only as trustworthy as the private key behind it. This section covers choosing an algorithm, generating a key pair with OpenSSL, protecting the private key, and the file formats keys come in.

Choosing an algorithm

- **RSA** — universally supported. Use 3072-bit for new deployments (2048-bit is still common but less future-proof).
- **ECDSA (EC)** — strong security with much smaller keys (P-256 ≈ 3072-bit RSA), faster, lower CPU/memory. The modern default.
- **Ed25519** — modern, fast and secure for signatures (widely used for SSH).

Generating a key pair

The general-purpose command is `openssl genpkey`. Always encrypt a stored private key with a passphrase and specify the key size explicitly (defaults vary between OpenSSL versions):

```
# RSA, encrypted with AES-256, key size stated explicitly (recommended)
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
    -aes-256-cbc -out private_key.pem
# stronger:  -pkeyopt rsa_keygen_bits:3072

# Elliptic-curve (P-256) private key
$ openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:prime256v1 \
    -out ec_private_key.pem
```

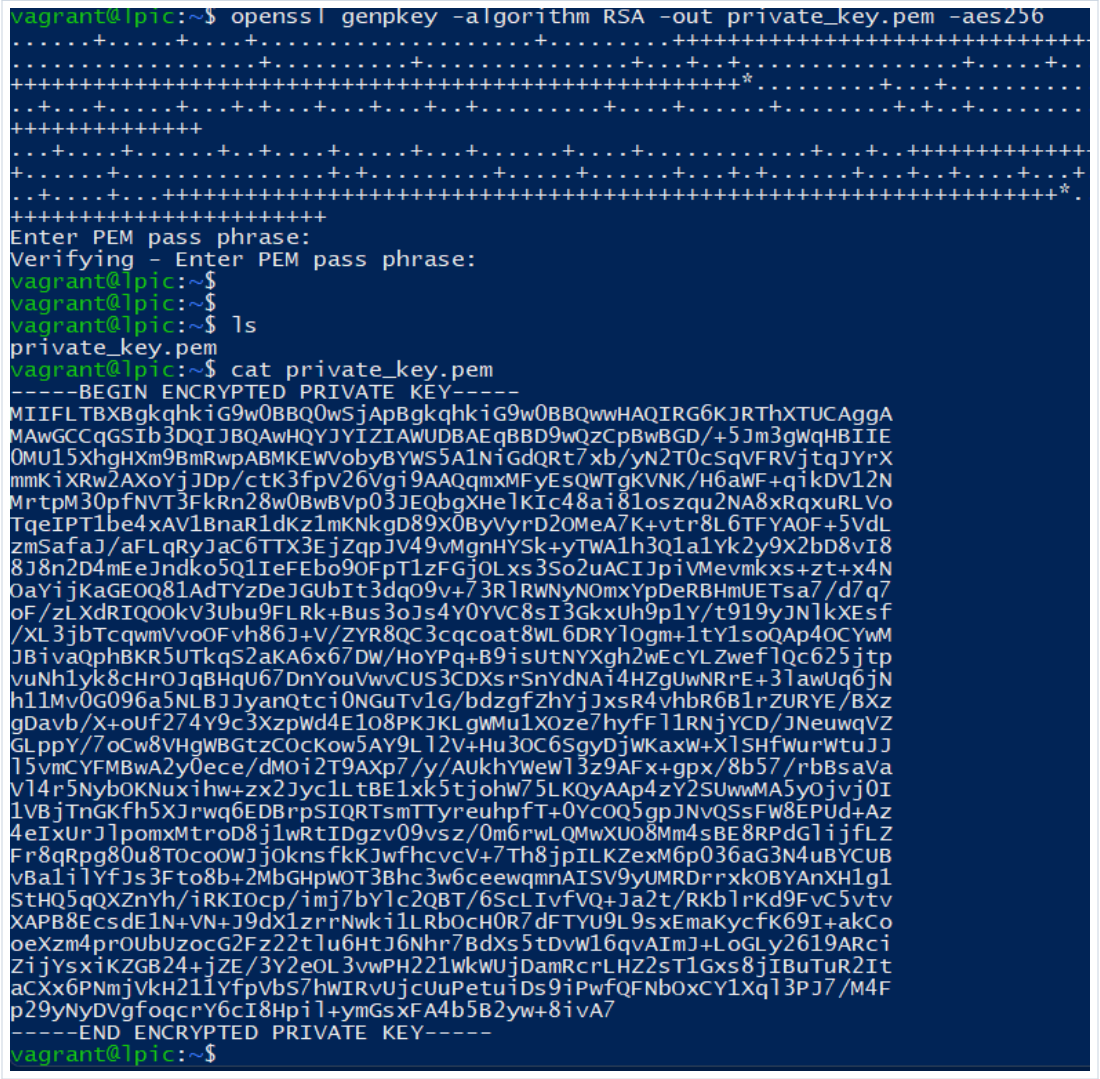


Figure — generating an RSA private key with openssl genpkey (basic form).

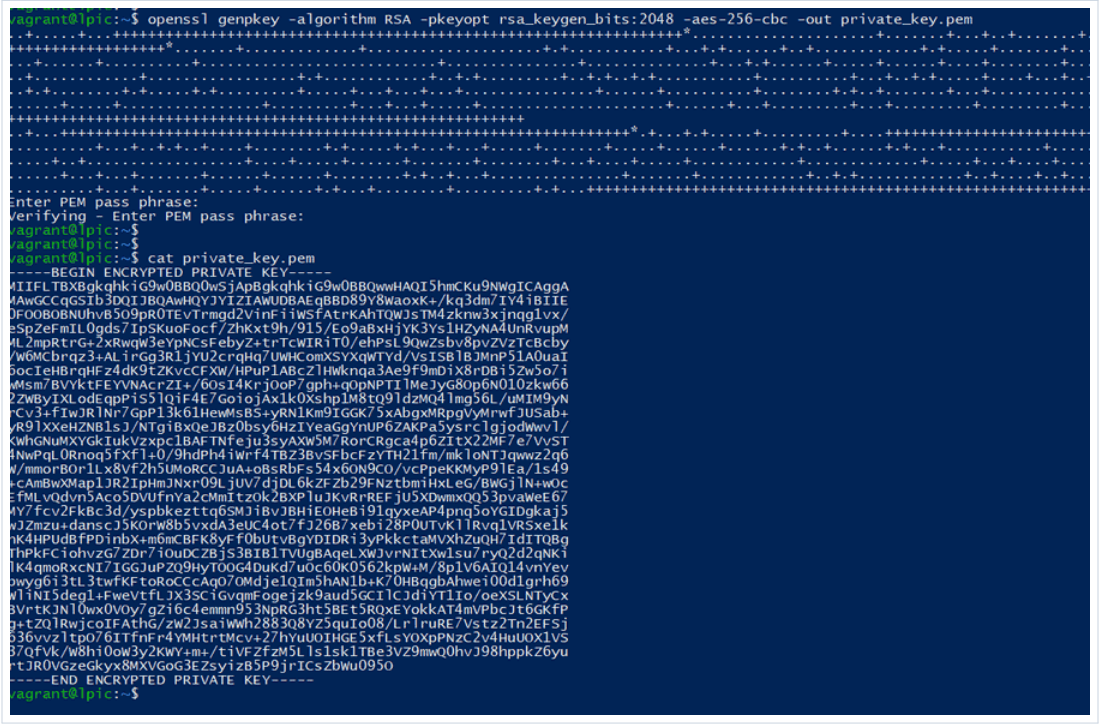


Figure — the recommended explicit form: 2048-bit RSA, AES-256-CBC encrypted, prompting for a passphrase.

Extract the public key from a private key with `openssl pkey -pubout` (the general form that works for RSA and EC alike):

```
$ openssl pkey -in private_key.pem -pubout -out public_key.pem
```

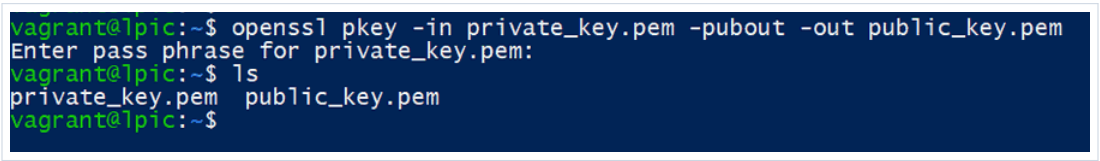


Figure — extracting the public key from a private key.

Protecting the private key

- Encrypt it at rest with a strong passphrase; store it on an encrypted disk, in a secrets vault, or in an HSM/smart card.
- Apply strict file permissions (e.g. `chmod 600`, owner-only) and restrict access to authorised staff.
- Never email or chat a private key; keep access-controlled backups; rotate keys on a schedule and immediately on suspected compromise.

Key formats

Keys and certificates are stored in a few formats; converting between them is a routine task.

Format	Encoding / use
PEM	Base64 text (-----BEGIN…-----). The general default for OpenSSL and web servers (.pem, .key, .crt).
DER	Binary. Used by Java and embedded systems, and APIs needing compact input (.der, .cer).

Format	Encoding / use
PKCS#1	Legacy RSA-only private key format (-----BEGIN RSA PRIVATE KEY-----).
PKCS#8	Modern standard private-key format for any algorithm (-----BEGIN PRIVATE KEY-----).
PKCS#12	A single encrypted bundle of key + certificate(s) (.p12/.pfx), for import into browsers/OSes.

```
# Convert a private key to modern PKCS#8 (encrypted)
$ openssl pkcs8 -topk8 -in private_key.pem -out private_key_pkcs8.pem

# PEM public key -> DER (binary)
$ openssl pkey -pubin -in public_key.pem -outform DER -out public_key.der

# Bundle key + cert into a PKCS#12 for browser/OS import
$ openssl pkcs12 -export -inkey private_key.pem -in server.crt -out bundle.p12
```

4. Certificate Signing Requests (CSRs)

To obtain a certificate you send a **Certificate Signing Request** to a CA. A CSR contains your public key and your identity (the subject DN), all signed with your private key to prove you hold it. The CA validates the details and issues a certificate. The flow is: **private key → CSR → CA → signed certificate**.

Generate a CSR interactively (OpenSSL prompts for each field), or non-interactively with -subj for automation and CI/CD:

```
# Interactive — prompts for country, org, common name, etc.
$ openssl req -new -key private_key.pem -out csr.pem

# Non-interactive — supply the whole subject on one line
$ openssl req -new -key private_key.pem -out csr.pem \
    -subj "/C=SE/ST=Vastmanland/L=Vasteras/O=Example/CN=example.com"
```

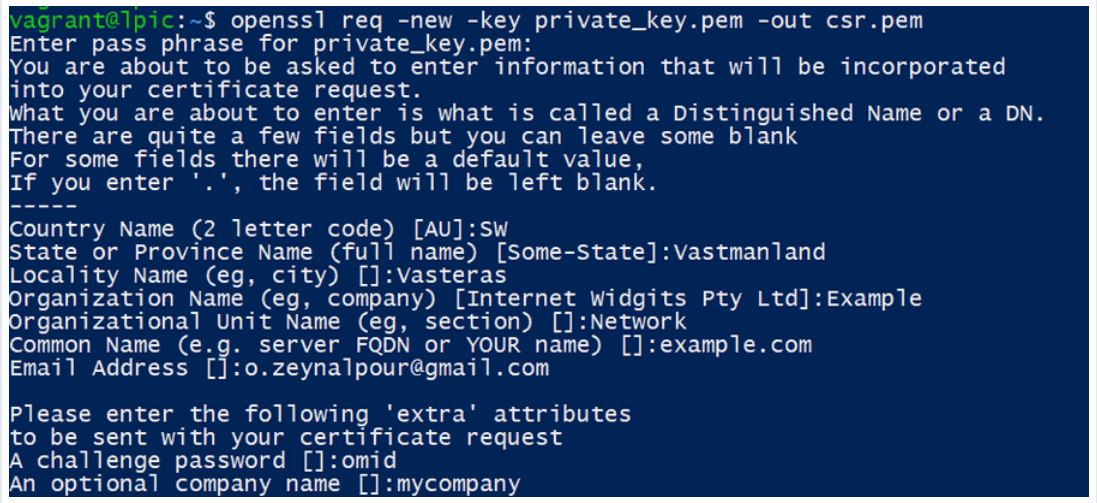


Figure — interactive CSR generation, entering the subject details.

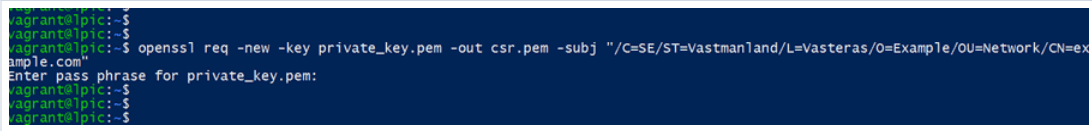


Figure — non-interactive CSR generation with -subj (automation-friendly).

Always verify what you produced before sending it — check the CSR, the private key, and the public key:

```
$ openssl req -in csr.pem -noout -text            # subject, public key, signature
$ openssl pkey -in private_key.pem -text -noout    # inspect the private key
$ openssl pkey -in public_key.pem -pubin -text -noout   # inspect the public key
```

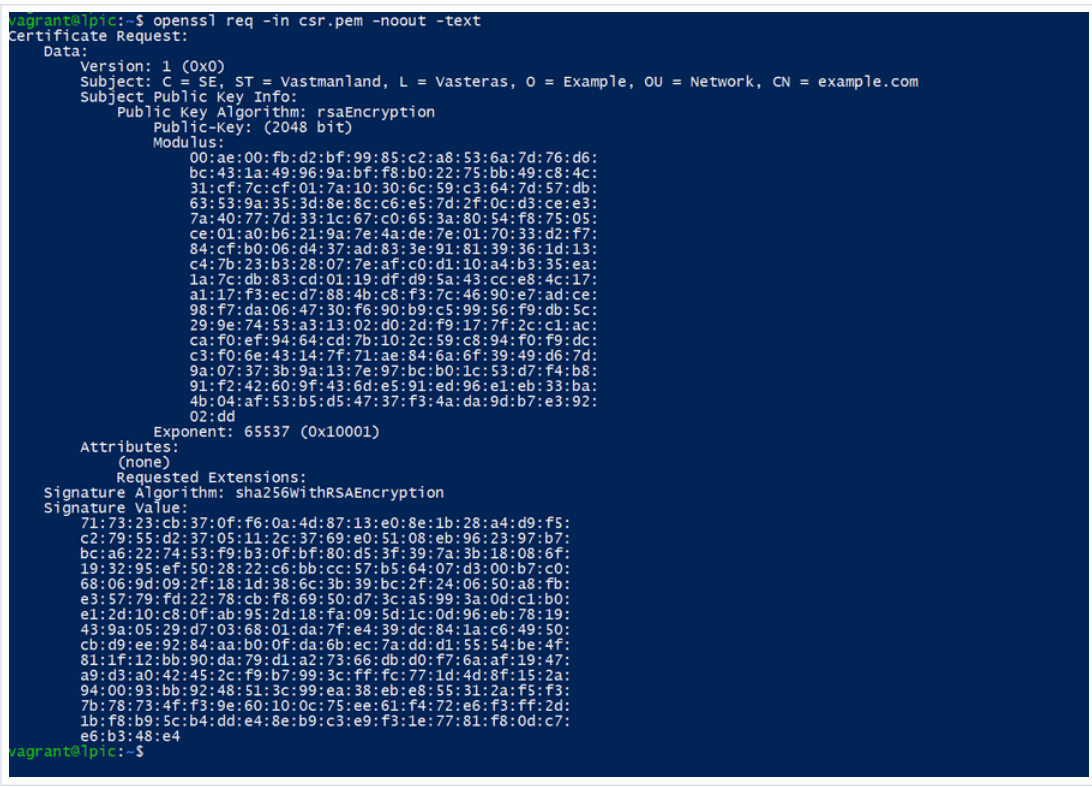


Figure — verifying the CSR with openssl req -text.

```
vagrant@lpic:~$ openssl pkey -in private_key.pem -text -noout
Enter pass phrase for private_key.pem:
Private-Key: (2048 bit, 2 primes)
modulus:
    00:ae:00:fb:d2:bf:99:85:c2:a8:53:6a:7d:76:d6:
    bc:43:1a:49:96:9a:bf:f8:b0:22:75:bb:49:c8:4c:
    31:cf:7c:cf:01:7a:10:30:6c:59:c3:64:7d:57:db:
    63:53:9a:35:3d:8e:8c:c6:e5:7d:2f:0c:d3:ce:e3:
    7a:40:77:7d:33:1c:67:c0:65:3a:80:54:f8:75:05:
    ce:01:a0:b6:21:9a:7e:4a:de:7e:01:70:33:d2:f7:
    84:cf:b0:06:d4:37:ad:83:3e:91:81:39:36:1d:13:
    c4:7b:23:b3:28:07:7e:af:c0:d1:10:a4:b3:35:ea:
    1a:7c:db:83:cd:01:19:df:d9:5a:43:cc:e8:4c:17:
    a1:17:f3:ec:d7:88:4b:c8:f3:7c:46:90:e7:ad:ce:
    98:f7:da:06:47:30:f6:90:b9:c5:99:56:f9:db:5c:
    29:9e:74:53:a3:13:02:d0:2d:f9:17:7f:2c:c1:ac:
    ca:f0:ef:94:64:cd:7b:10:2c:59:c8:94:f0:f9:dc:
    c3:f0:6e:43:14:7f:71:ae:84:6a:6f:39:49:d6:7d:
    9a:07:37:3b:9a:13:7e:97:bc:b0:1c:53:d7:f4:b8:
    91:f2:42:60:9f:43:6d:e5:91:ed:96:e1:eb:33:ba:
    4b:04:af:53:b5:d5:47:37:f3:4a:da:9d:b7:e3:92:
    02:dd
publicExponent: 65537 (0x10001)
privateExponent:
    25:3e:b8:a8:67:b2:a8:da:fd:c4:bd:2a:95:e1:68:
    f2:c4:11:b4:56:f3:78:7b:db:24:70:2e:5e:dd:50:
    5c:25:4e:f9:f5:f2:b9:2a:b8:1c:06:ca:7d:9f:cc:
    26:fc:ff:20:69:fe:e0:24:07:23:a7:0f:3f:5d:99:
    b1:6a:e2:35:aa:82:da:bf:d0:41:df:ee:3e:be:3b:
    56:45:dc:36:5d:27:63:47:34:99:82:36:d5:95:7d:
    ac:26:f5:8f:3f:4b:b8:a8:0f:e5:25:25:dc:1b:7b:
    17:10:91:2c:a4:4e:0d:b8:ac:07:6f:f9:9c:34:bb:
    ee:b5:de:04:67:93:ee:41:f1:99:87:cd:32:15:54:
    b8:48:7f:61:59:b9:d3:c2:98:8c:9c:90:8f:71:82:
    5b:70:f7:5f:2f:d5:37:80:43:0f:b2:6c:65:9d:6b:
    c1:07:a8:ed:bd:e4:f9:a9:51:c3:64:12:0d:5a:e9:
    03:48:8a:68:66:0c:bc:19:bd:2c:47:25:46:b3:58:
    2c:f2:41:76:96:3c:67:2d:62:e0:cb:a5:58:99:ad:
    56:b8:d9:68:76:ec:7f:7b:c0:a6:fe:43:b0:7b:de:
    34:48:0a:4c:32:6d:74:87:aa:8b:4e:73:f2:59:86:
    4a:f0:a9:6b:2c:49:9b:be:5f:78:17:7e:58:d2:a2:
```

Figure — inspecting the private key.

```
vagrant@lpic:~$ openssl pkey -in public_key.pem -pubin -text -noout
Public-Key: (2048 bit)
Modulus:
    00:ae:00:fb:d2:bf:99:85:c2:a8:53:6a:7d:76:d6:
    bc:43:1a:49:96:9a:bf:f8:b0:22:75:bb:49:c8:4c:
    31:cf:7c:cf:01:7a:10:30:6c:59:c3:64:7d:57:db:
    63:53:9a:35:3d:8e:8c:c6:e5:7d:2f:0c:d3:ce:e3:
    7a:40:77:7d:33:1c:67:c0:65:3a:80:54:f8:75:05:
    ce:01:a0:b6:21:9a:7e:4a:de:7e:01:70:33:d2:f7:
    84:cf:b0:06:d4:37:ad:83:3e:91:81:39:36:1d:13:
    c4:7b:23:b3:28:07:7e:af:c0:d1:10:a4:b3:35:ea:
    1a:7c:db:83:cd:01:19:df:d9:5a:43:cc:e8:4c:17:
    a1:17:f3:ec:d7:88:4b:c8:f3:7c:46:90:e7:ad:ce:
    98:f7:da:06:47:30:f6:90:b9:c5:99:56:f9:db:5c:
    29:9e:74:53:a3:13:02:d0:2d:f9:17:7f:2c:c1:ac:
    ca:f0:ef:94:64:cd:7b:10:2c:59:c8:94:f0:f9:dc:
    c3:f0:6e:43:14:7f:71:ae:84:6a:6f:39:49:d6:7d:
    9a:07:37:3b:9a:13:7e:97:bc:b0:1c:53:d7:f4:b8:
    91:f2:42:60:9f:43:6d:e5:91:ed:96:e1:eb:33:ba:
    4b:04:af:53:b5:d5:47:37:f3:4a:da:9d:b7:e3:92:
    02:dd
Exponent: 65537 (0x10001)
vagrant@lpic:~$
```

Figure — inspecting the public key.

5. Public Key Infrastructure and Trust Chains

A single certificate is only useful because of the system around it. **Public Key Infrastructure (PKI)** is that system: the policies, roles and components that issue, distribute, validate and revoke certificates. Its core parts are the **Certificate Authority (CA)** that signs certificates, the **Registration Authority (RA)** that verifies identities before issuance, a **repository** that publishes certificates and revocation information, and the **CRL/OCSP** services that report revocation status.

The chain of trust

Trust is delegated down a chain. A **root CA** is self-signed and highly protected; it signs one or more **intermediate CAs**; an intermediate signs the **end-entity** (leaf) certificates for servers and clients:

```
Root CA  ──signs──►  Intermediate CA  ──signs──►  Server / Client certificate
```

A client validates a certificate by building this chain up to a **root it already trusts** (roots ship in the operating system and browser trust stores). At each link it checks the issuer's signature, that Basic Constraints permits signing, that the name matches (SAN), that the dates are current, and that the certificate has not been revoked. If every link holds up to a trusted root, the certificate is accepted.

Certificate Transparency

Certificate Transparency (CT) adds public, append-only logs of every certificate a CA issues. Because mis-issued or fraudulent certificates become publicly visible, domain owners can watch the logs for certificates they did not request, and browsers can require that a certificate appear in CT logs before trusting it. It is a detection layer on top of PKI, not a replacement for it.

6. The Certificate Lifecycle

A certificate moves through a predictable lifecycle, and managing it well is largely about tracking where each certificate is in that cycle.

Stage	What happens
1. Generate key	Create the private/public key pair (keep the private key safe).
2. Create CSR	Build a signing request with the public key and subject.
3. Submit to CA	Send the CSR to the CA (public, internal or cloud).
4. CA validates	The CA (via an RA) verifies the requester controls the identity.
5. Issue	The CA signs and returns the certificate.
6. Install	Deploy the certificate and key on the server/device.
7. Monitor	Watch expiry and revocation status; alert before expiry.
8. Renew / replace	Re-issue before expiry (ideally automated).
9. Revoke	Invalidate early if the key is compromised or details change.
10. Expire	The certificate passes Not After and is no longer valid.

7. Creating, Operating and Securing a Certificate Authority

You can run your own CA — for a lab, an internal network, or a product. It comes in three parts: creating it, operating it, and securing it.

Creating a CA

A minimal CA is a private key plus a self-signed root certificate that asserts CA:true. In production you then create an intermediate CA and keep the root offline.

```
# 1. Root CA private key (protect this above all else)
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -aes-256-cbc -out
ca.key

# 2. Self-signed root certificate (long-lived; CA:true)
$ openssl req -x509 -new -key ca.key -days 3650 -out ca.crt \
    -subj "/C=SE/O=Example Lab/CN=Example Lab Root CA" \
    -addext "basicConstraints=critical,CA:true" \
    -addext "keyUsage=critical,keyCertSign,cRLSign"
```



Figure — creating the Certificate Authority key and self-signed root certificate.

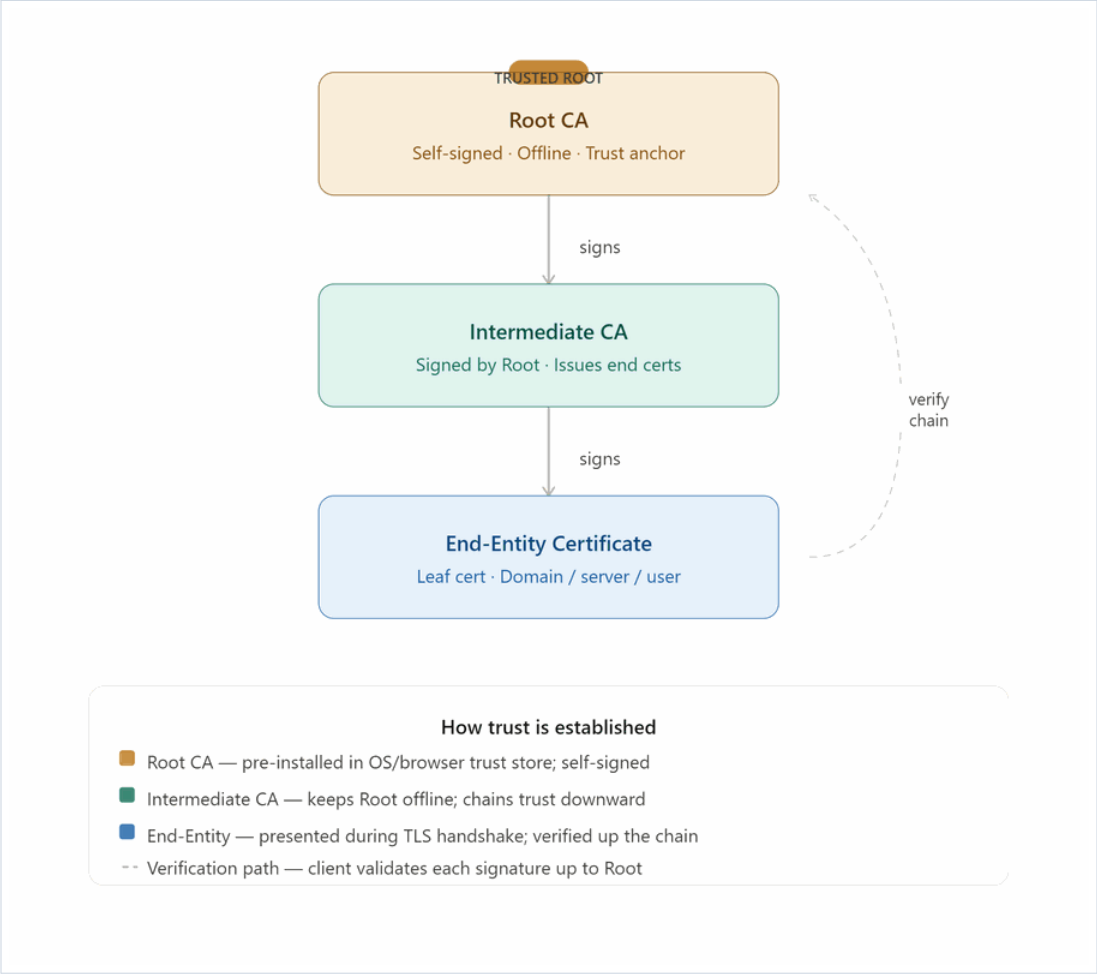


Figure — the CA certificate and its configuration.

Operating a CA

Operating a CA means issuing certificates on request and keeping records. The `openssl ca` command (driven by an `openssl.cnf` with a `[ CA_default ]` section) maintains a database (the `index.txt` file) and a serial-number counter, tracking every certificate issued and its status. This is what lets the CA later revoke by serial number and generate CRLs.

Securing a CA

- **Keep the root CA offline** (air-gapped) and issue only through intermediates — if the root key leaks, the entire trust hierarchy is lost.
- **Store CA keys in an HSM** where possible; apply strict access control, logging and multi-person procedures.
- **Use short-lived intermediates and monitor issuance** (Certificate Transparency, internal audit) for anything unexpected.

The three real-world flavours: a **public CA** (Let's Encrypt, DigiCert) trusted by browsers worldwide; an **internal/enterprise CA** (e.g. Microsoft AD CS, or CFSSL) trusted only within an organisation; and a **cloud CA** (AWS/GCP Private CA) managed as a service. Public CAs suit internet-facing services; internal CAs suit corporate and device identities.

8. Issuing Server and Client Certificates

With a CA in place, issuing a certificate is: the requester makes a key and CSR, and the CA signs it. Sign with openssl x509 -req (simple) or openssl ca (tracked in the CA database). A server certificate needs a SAN and serverAuth; a client certificate needs clientAuth.

```
# Sign a server CSR with the CA, adding a SAN and serverAuth EKU
$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -days 365 -out server.crt \
    -extfile <(printf "subjectAltName=DNS:example.local,DNS:localhost,IP:127.0.0.1\
nextendedKeyUsage=serverAuth\nbasicConstraints=critical,CA:false")

# A client certificate (clientAuth), then bundle it as PKCS#12 for import
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out alice.key
$ openssl req -new -key alice.key -subj "/O=Example Lab/CN=Alice" -out alice.csr
$ openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -days 365 -out alice.crt -extfile <(printf "extendedKeyUsage=clientAuth")
$ openssl pkcs12 -export -inkey alice.key -in alice.crt -certfile ca.crt -out
alice.p12
```

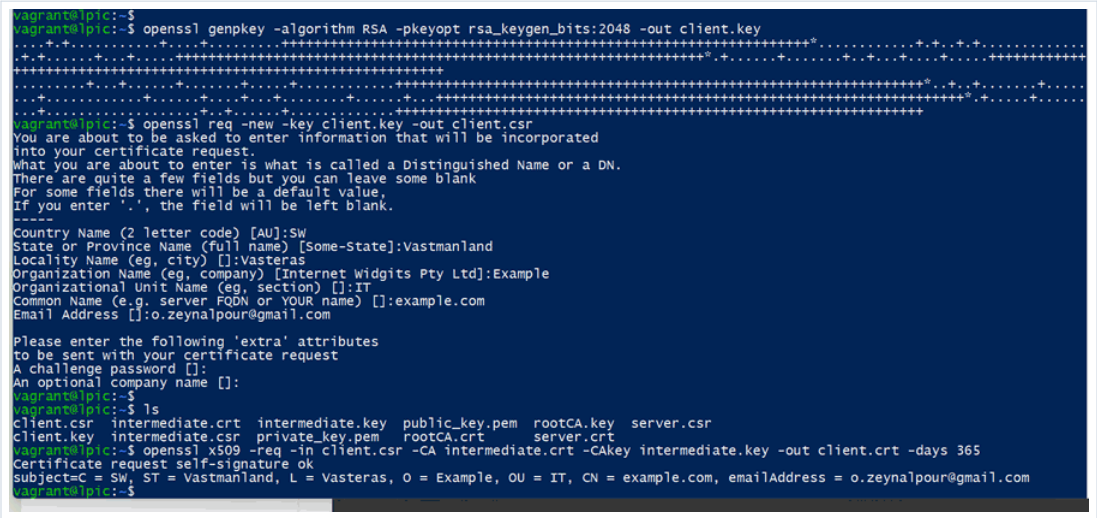


Figure — generating and signing a client certificate with the lab CA.

9. Revoking Certificates

Sometimes a certificate must be invalidated before it expires — a private key is compromised, an employee leaves, or details change. Because clients cache certificates, revocation needs its own publishing mechanism.

Certificate Revocation Lists (CRLs)

A **CRL** is a signed list of revoked serial numbers that the CA publishes periodically. Clients download it (from the CRL Distribution Point in the certificate) and reject anything on the list.

```
$ openssl ca -config openssl.cnf -revoke server.crt      # revoke a certificate
$ openssl ca -config openssl.cnf -gencrl -out ca.crl       # (re)generate the CRL
$ openssl crl -in ca.crl -noout -text                      # inspect the CRL
```

OCSP — real-time checking

CRLs can be large and stale. The **Online Certificate Status Protocol (OCSP)** lets a client ask the CA about *one* certificate's status in real time, using the OCSP URL in the certificate's AIA extension. To avoid the latency and privacy cost of every client asking the CA, servers use **OCSP stapling** — the server fetches a signed status itself and includes it in the TLS handshake (covered in detail in objective 331.2).

> **Revoking a CA is drastic**
>
> Revoking an intermediate or root CA invalidates **every** certificate beneath it at once. It is a last resort for a compromised CA key — which is exactly why root keys are kept offline and protected so heavily.

10. Automated Certificates: Let's Encrypt, ACME and CFSSL

Manually issuing and renewing certificates does not scale. Two tools dominate automation.

Let's Encrypt and ACME

Let's Encrypt is a free, automated public CA. It uses the **ACME** protocol, in which your client proves control of a domain by answering a challenge (serving a token over HTTP, or publishing a DNS record) and the CA then issues a short-lived (90-day) certificate automatically. The standard client is **Certbot**, which can obtain and install a certificate and set up automatic renewal in one command.

```
$ certbot --nginx -d example.com -d www.example.com   # obtain + install + auto-
renew
$ certbot certonly --standalone -d example.com        # just obtain the certificate
$ certbot renew --dry-run                             # test automatic renewal
```

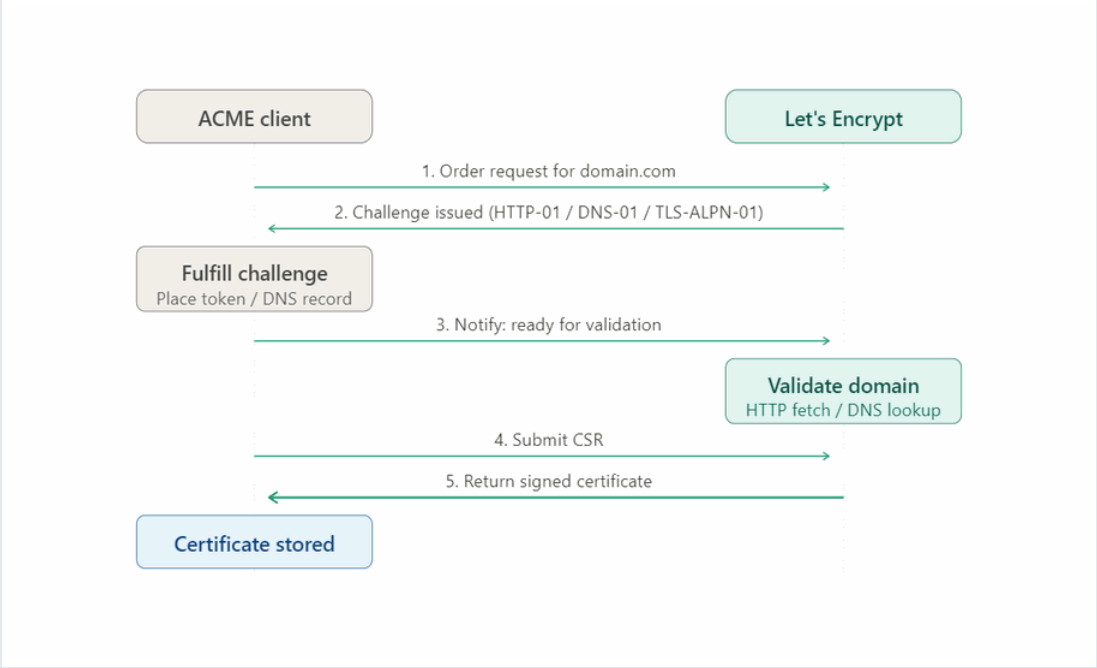


Figure — the ACME flow / obtaining a certificate with Certbot.

CFSSL

CFSSL (CloudFlare's SSL toolkit) is a popular way to run an *internal* PKI programmatically. It issues and signs certificates from JSON configuration and can run as an API server, which makes it well suited to microservices and automated infrastructure — you will use it in Tier 2 of the lab.

```
$ cfssl gencert -initca ca-csr.json | cfssljson -bare ca      # create a CA
$ cfssl gencert -ca ca.pem -ca-key ca-key.pem \
    -config config.json -profile server server-csr.json | cfssljson -bare server
```

11. Practical PKI Lab

The rest of the chapter is a hands-on lab that builds PKI three ways, from a single-use self-signed setup to a real internet certificate. The screenshots throughout are from the lab being run end to end.

> **Prerequisites**
>
> Install OpenSSL and (for serving) Nginx. The examples were run on both Windows (PowerShell, with the bundled openssl.exe) and Linux, so the commands are portable.

Tier 1 — Local self-signed PKI (development & testing)

The simplest PKI: create your own root CA, issue a server certificate from it, trust the root locally, and serve HTTPS with Nginx. Perfect for development, where you control both ends.

```
# 1. Create the local root CA
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -aes-256-cbc -out
ca.key
$ openssl req -x509 -new -key ca.key -days 3650 -out ca.crt \
    -subj "/C=SE/O=Example Lab/CN=example.local" -addext
"basicConstraints=critical,CA:true"

# 2. Server key + CSR
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out server.key
$ openssl req -new -key server.key -subj "/CN=example.local" -out server.csr

# 3. Sign the server cert with a SAN
$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days
365 \
    -out server.crt -extfile <(printf
"subjectAltName=DNS:example.local,DNS:localhost,IP:127.0.0.1")

# 4. Point Nginx at server.crt / server.key, and trust ca.crt in the OS/browser
store
```

```
PS C:\>
PS C:\> New-Item -ItemType Directory -Force -Path C:\local-pki\ca


    Directory: C:\local-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026  10:35 AM                ca

PS C:\> New-Item -ItemType Directory -Force -Path C:\local-pki\intermediate


    Directory: C:\local-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026  10:35 AM                intermediate

PS C:\> New-Item -ItemType Directory -Force -Path C:\local-pki\server


    Directory: C:\local-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026  10:35 AM                server

PS C:\> New-Item -ItemType Directory -Force -Path C:\local-pki\client


    Directory: C:\local-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026  10:35 AM                client


PS C:\> cd C:\local-pki
PS C:\local-pki>
```

Figure — Tier 1 lab: local self-signed PKI (1/16)

```
PS C:\> cd C:\local-pki
PS C:\local-pki>
PS C:\local-pki> ls


    Directory: C:\local-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026  10:35 AM                ca
d-----         5/26/2026  10:35 AM                client
d-----         5/26/2026  10:35 AM                intermediate
d-----         5/26/2026  10:35 AM                server


PS C:\local-pki>
```

Figure — Tier 1 lab: local self-signed PKI (2/16)

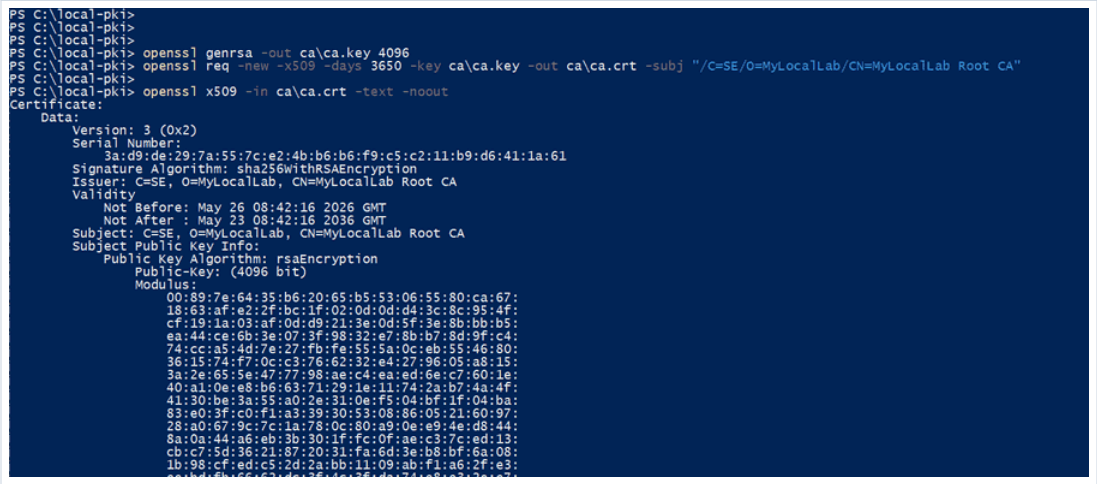


Figure — Tier 1 lab: local self-signed PKI (3/16)

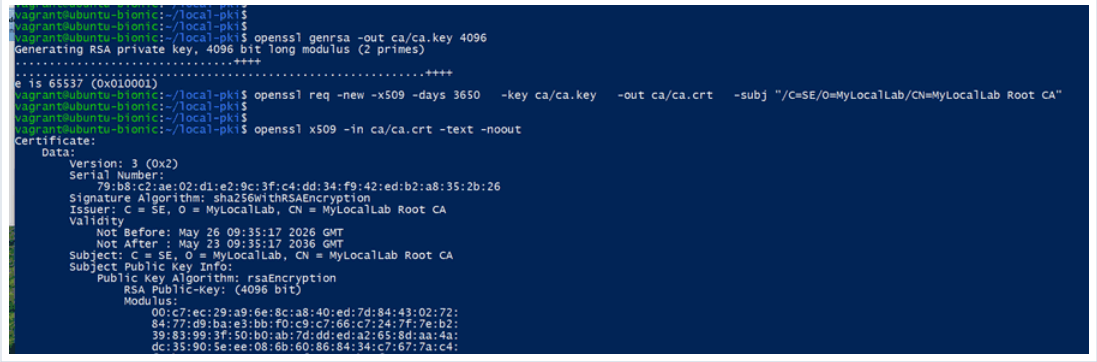


Figure — Tier 1 lab: local self-signed PKI (4/16)

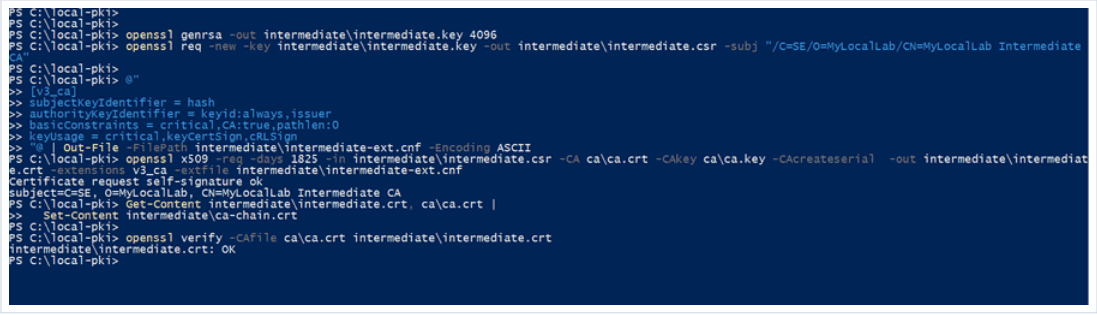


Figure — Tier 1 lab: local self-signed PKI (5/16)

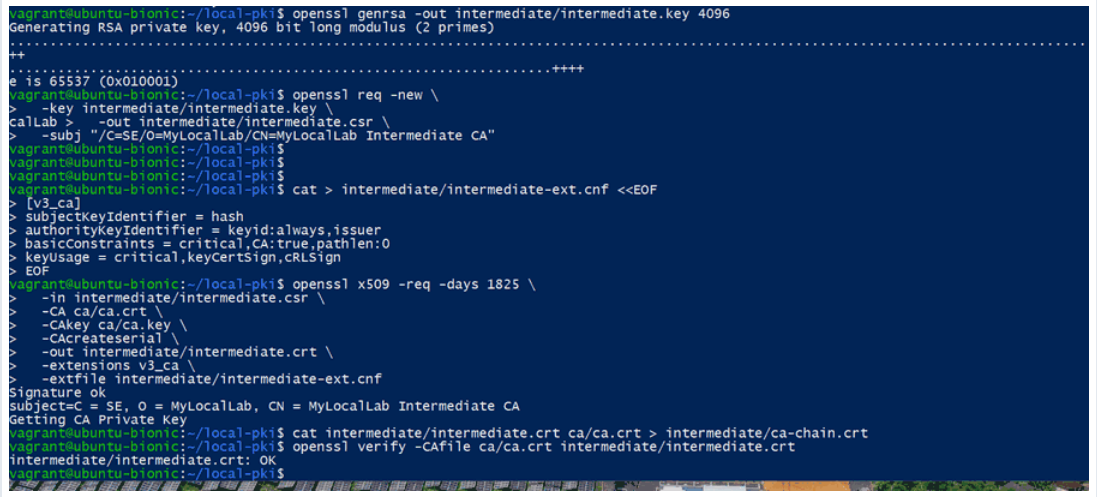


Figure — Tier 1 lab: local self-signed PKI (6/16)

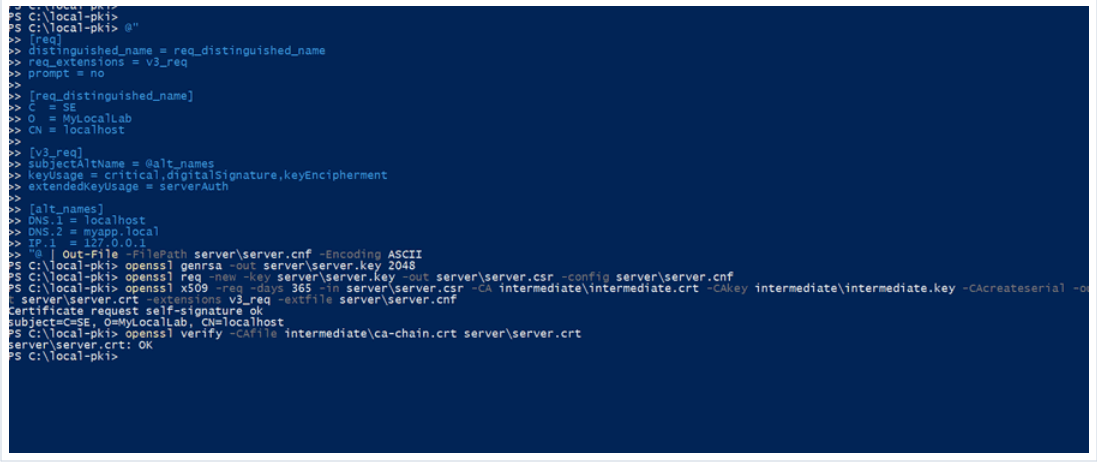


Figure — Tier 1 lab: local self-signed PKI (7/16)

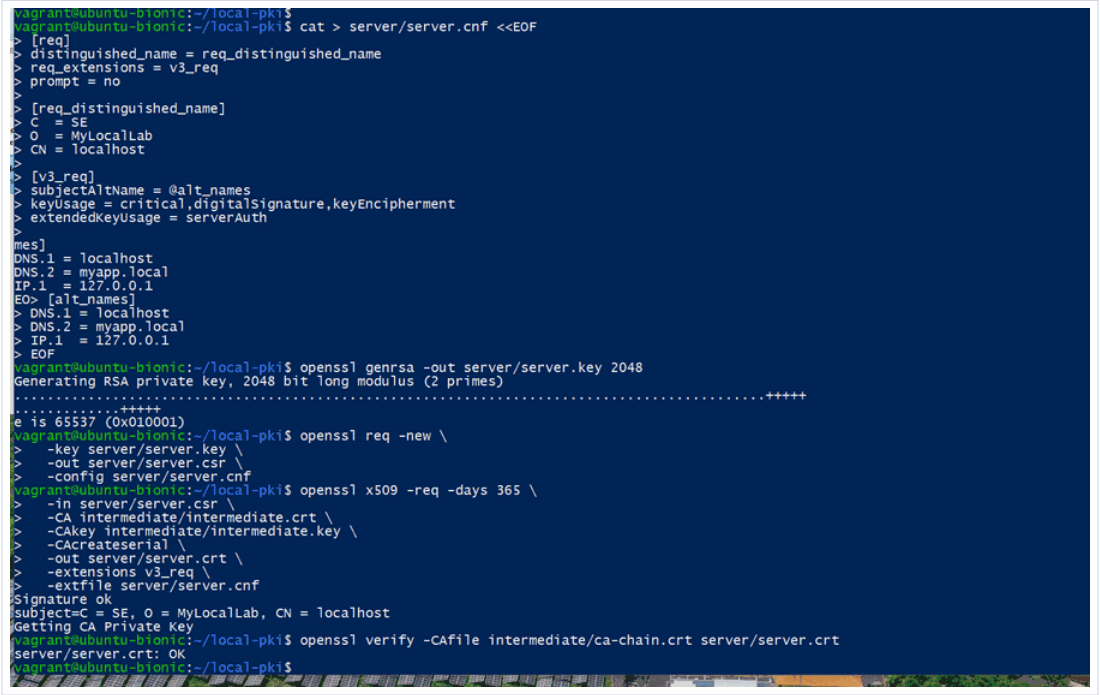


Figure — Tier 1 lab: local self-signed PKI (8/16)



Figure — Tier 1 lab: local self-signed PKI (9/16)

```
vagrant@ubuntu-bionic:~$
vagrant@ubuntu-bionic:~$ sudo tee /etc/nginx/sites-available/pki-lab <<EOF
> server {
>     listen 80;
>     server_name localhost myapp.local;
>     return 301 https://\$host\$request_uri;
> }
> server {
>     listen 443 ssl;
>     server_name localhost myapp.local;
>
>     ssl_certificate     $HOME/local-pki/server/server.crt;
>     ssl_certificate_key $HOME/local-pki/server/server.key;
>     ssl_protocols       TLSv1.2 TLSv1.3;
>     ssl_ciphers         HIGH:!aNULL:!MD5;
>
>     location / {
>         return 200 '<h1>PKI Lab Working!</h1><p>Root CA → Intermediate CA → Server Cert</p>';
>         add_header Content-Type text/html;
>     }
> }
> EOF
server {
    listen 80;
    server_name localhost myapp.local;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name localhost myapp.local;

    ssl_certificate     /home/vagrant/local-pki/server/server.crt;
    ssl_certificate_key /home/vagrant/local-pki/server/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    location / {
        return 200 '<h1>PKI Lab Working!</h1><p>Root CA → Intermediate CA → Server Cert</p>';
        add_header Content-Type text/html;
    }
}
vagrant@ubuntu-bionic:~$ sudo ln -sf /etc/nginx/sites-available/pki-lab /etc/nginx/sites-enabled/
vagrant@ubuntu-bionic:~$ sudo nginx -t && sudo systemctl reload nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
vagrant@ubuntu-bionic:~$
```

Figure — Tier 1 lab: local self-signed PKI (10/16)

```
vagrant@ubuntu-bionic:~$
vagrant@ubuntu-bionic:~$ echo "127.0.0.1  myapp.local" | sudo tee -a /etc/hosts
127.0.0.1  myapp.local
vagrant@ubuntu-bionic:~$
vagrant@ubuntu-bionic:~$
```

Figure — Tier 1 lab: local self-signed PKI (11/16)

```
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2> Import-Certificate -FilePath "C:\local-pki\ca\ca.crt" -CertStoreLocation Cert:\LocalMachine\Root


   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\Root

Thumbprint                                Subject
----------                                -------
93696AE544F9DA774C4AEB55162859DDFBBEFCF1  CN=MyLocalLab Root CA, O=MyLocalLab, C=SE


PS C:\nginx-1.30.2> Get-ChildItem Cert:\LocalMachine\Root | Where-Object { $_.Subject -like "*MyLocalLab*" }


   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\Root

Thumbprint                                Subject
----------                                -------
93696AE544F9DA774C4AEB55162859DDFBBEFCF1  CN=MyLocalLab Root CA, O=MyLocalLab, C=SE
27A727D50EBBC653B6CD630BE06F4B19BDFBF3E3  CN=MyLocalLab Root CA, O=MyLocalLab, C=SE


PS C:\nginx-1.30.2>
```

Figure — Tier 1 lab: local self-signed PKI (12/16)

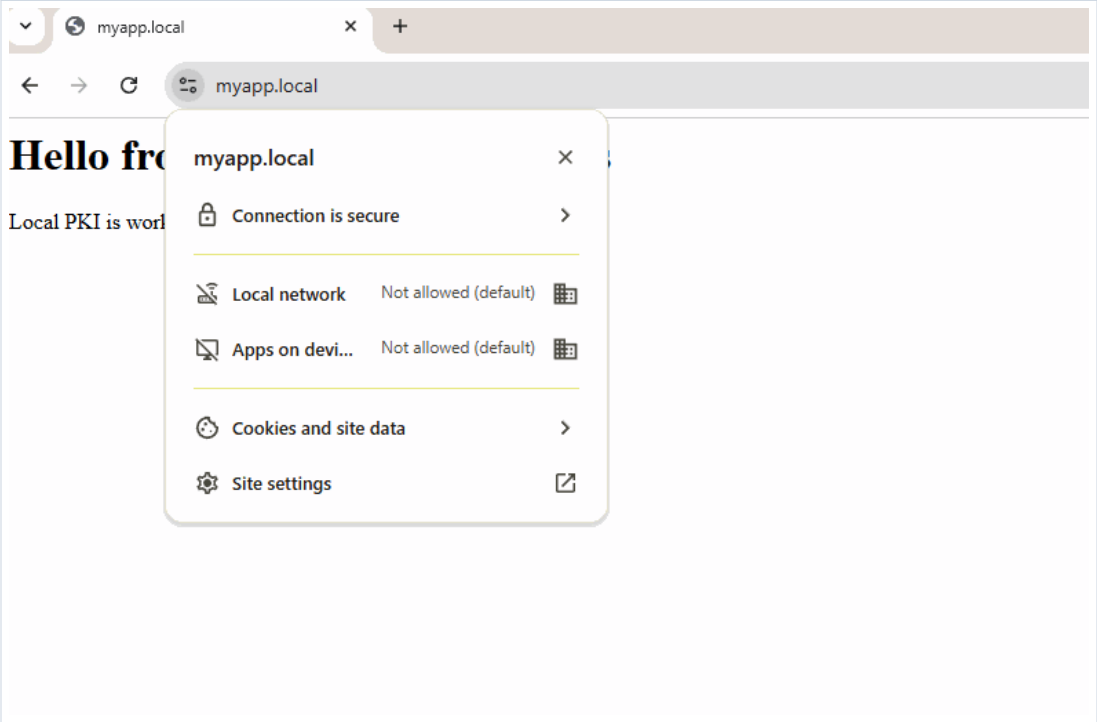


Figure — Tier 1 lab: local self-signed PKI (13/16)

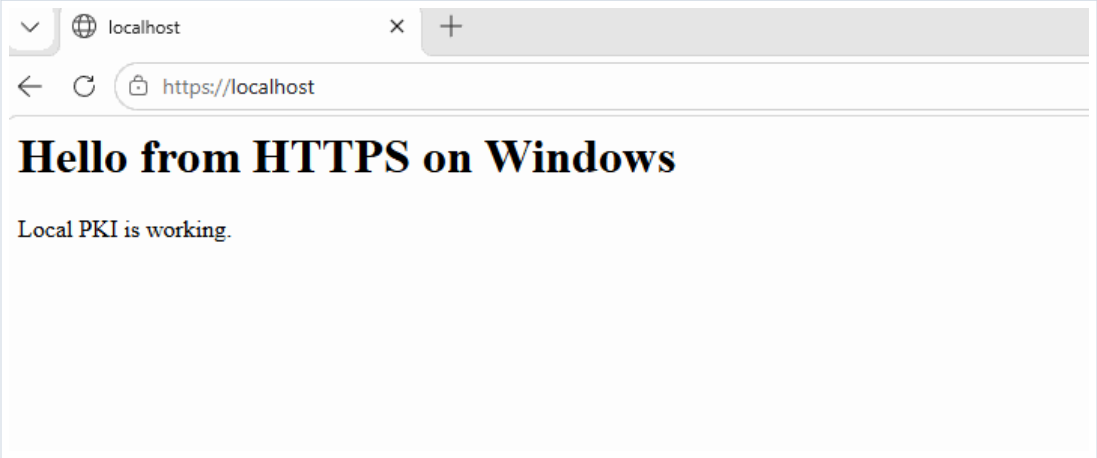


Figure — Tier 1 lab: local self-signed PKI (14/16)

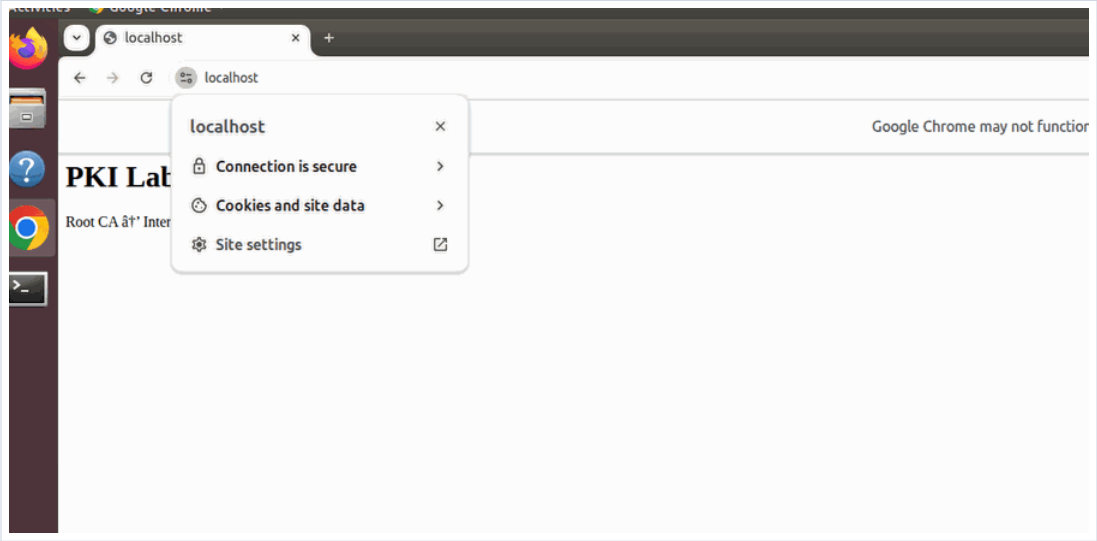


Figure — Tier 1 lab: local self-signed PKI (15/16)

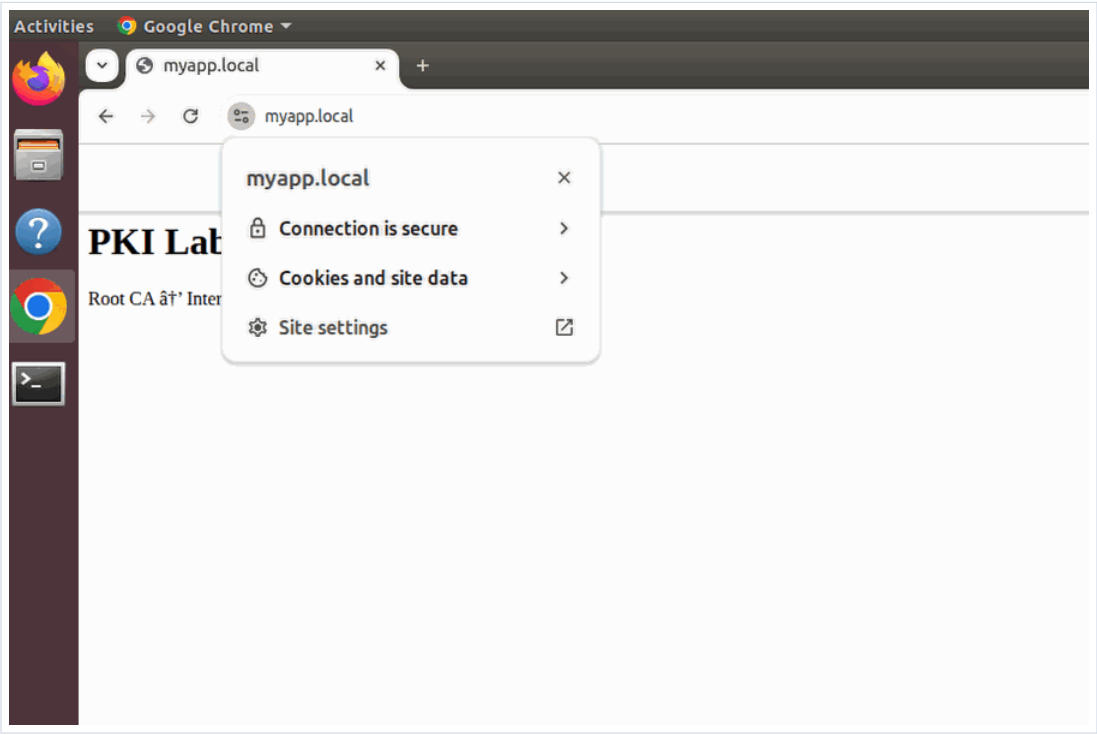


Figure — Tier 1 lab: local self-signed PKI (16/16)

Tier 2 — Enterprise PKI with CFSSL

A realistic internal PKI: a root and intermediate CA driven by JSON configuration with CFSSL, issuing server and client certificates, and finishing with a mutual-TLS (mTLS) service where a client certificate proves the caller's identity — the model used inside organisations and microservice platforms. The screenshots below are from this enterprise lab (the internal domain acme-corp.internal).

```
# Create the CA and issue certs from JSON profiles
$ cfssl gencert -initca ca-csr.json | cfssljson -bare ca
```

```
$ cfssl gencert -ca ca.pem -ca-key ca-key.pem -config cfssl.json \
    -profile server server-csr.json | cfssljson -bare server
$ cfssl gencert -ca ca.pem -ca-key ca-key.pem -config cfssl.json \
    -profile client client-csr.json | cfssljson -bare client

# Build the chain and verify it
$ cat intermediate/intermediate-ca.pem root/root-ca.pem > ca-chain.pem
$ openssl verify -CAfile root/root-ca.pem intermediate/intermediate-ca.pem
# then require the client cert in Nginx (ssl_verify_client on) for mTLS
```

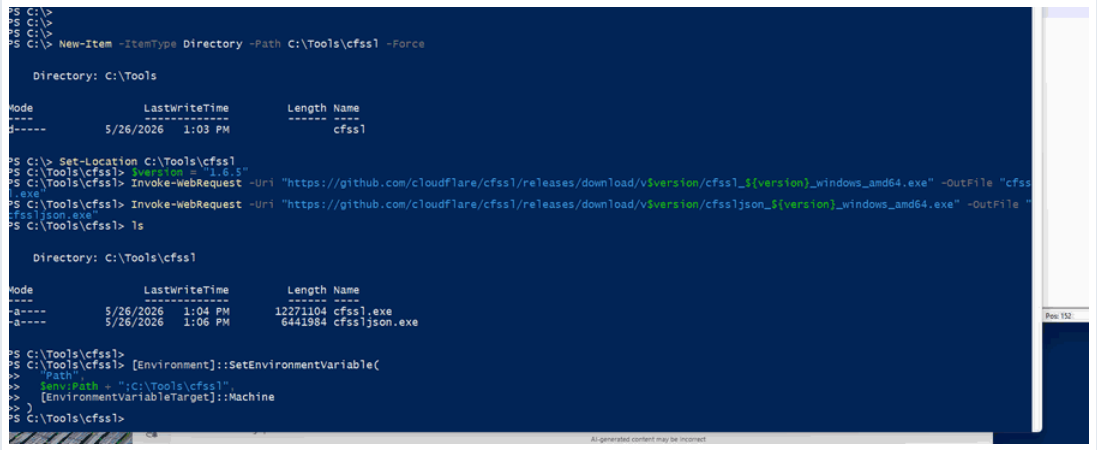


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (1/25)

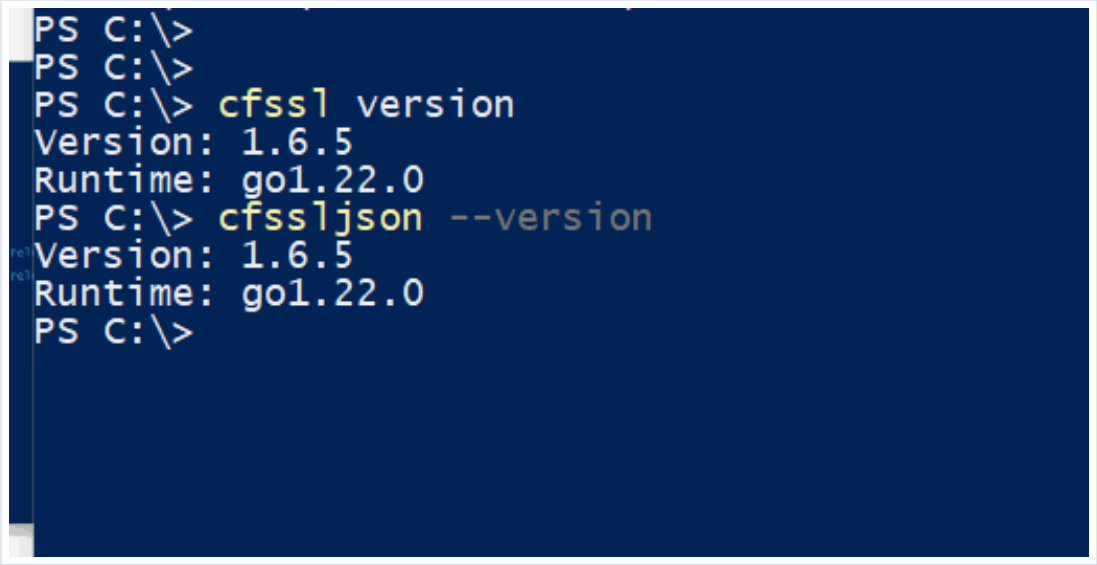


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (2/25)

```
PS C:\>
PS C:\>
PS C:\> New-Item -ItemType Directory -Path `
>> "C:\enterprise-pki\root",
>> "C:\enterprise-pki\intermediate",
>> "C:\enterprise-pki\profiles",
>> "C:\enterprise-pki\certs" -Force


    Directory: C:\enterprise-pki


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         5/26/2026   4:07 PM                root
d-----         5/27/2026   3:57 PM                intermediate
d-----         5/26/2026   4:11 PM                profiles
d-----         5/27/2026   1:19 PM                certs


PS C:\> cd C:\enterprise-pki\
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (3/25)

```
PS C:\enterprise-pki> cd C:\enterprise-pki
PS C:\enterprise-pki>
PS C:\enterprise-pki> @"
>> {
>>   "CN": "Enterprise Root CA",
>>   "key": { "algo": "rsa", "size": 4096 },
>>   "names": [{
>>     "C":  "SE",
>>     "ST": "Stockholm",
>>     "L":  "Stockholm",
>>     "O":  "Acme Corp",
>>     "OU": "Security"
>>   }],
>>   "ca": { "expiry": "87600h" }
>> }
>> "@ | Out-File -FilePath root/root-csr.json -Encoding ASCII
PS C:\enterprise-pki>
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (4/25)

```
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> cfssl gencert -initca root\root-csr.json | cfssljson -bare root\root-ca
2026/05/28 14:59:10 [INFO] generating a new CA key and certificate from CSR
2026/05/28 14:59:10 [INFO] generate received request
2026/05/28 14:59:10 [INFO] received CSR
2026/05/28 14:59:10 [INFO] generating key: rsa-4096
2026/05/28 14:59:12 [INFO] encoded CSR
2026/05/28 14:59:12 [INFO] signed certificate with serial number 111751601860321513692366983712434761377599865194
PS C:\enterprise-pki> Get-ChildItem root\


    Directory: C:\enterprise-pki\root


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         5/28/2026   2:59 PM           3243 root-ca-key.pem
-a----         5/28/2026   2:59 PM           1769 root-ca.csr
-a----         5/28/2026   2:59 PM           2057 root-ca.pem
-a----         5/28/2026   2:39 PM            234 root-csr.json


PS C:\enterprise-pki> openssl x509 -in root\root-ca.pem -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            13:93:1d:79:b4:93:5a:82:ac:93:7f:86:0b:a0:2e:32:56:66:41:6a
        Signature Algorithm: sha512WithRSAEncryption
        Issuer: C=SE, ST=Stockholm, L=Stockholm, O=Acme Corp, OU=Security, CN=Enterprise Root CA
        Validity
            Not Before: May 28 12:54:00 2026 GMT
            Not After : May 25 12:54:00 2036 GMT
        Subject: C=SE, ST=Stockholm, L=Stockholm, O=Acme Corp, OU=Security, CN=Enterprise Root CA
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
                    00:c2:4c:5f:27:02:a1:27:83:93:4a:30:06:4e:87:
                    e8:02:10:8c:c1:47:62:b9:de:da:58:23:2b:2f:4c:
                    84:1d:ee:40:6a:78:f6:27:27:9b:8b:64:18:f4:59:
                    49:51:57:d4:d6:7c:fe:aa:6b:e9:fb:92:ca:36:b2:
                    24:47:ad:4c:7e:43:cd:40:bf:33:a0:59:ff:1d:bb:
                    62:92:5b:9b:49:44:09:34:c6:b2:1f:d2:00:f9:4e:
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (5/25)

```
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> @"
>> {
>>   "signing": {
>>     "default": {
>>       "expiry": "8760h"
>>     },
>>     "profiles": {
>>       "intermediate": {
>>         "usages": ["cert sign", "crl sign"],
>>         "expiry": "43800h",
>>         "is_ca": true,
>>         "ca_constraint": {
>>           "is_ca": true,
>>           "max_path_len": 0,
>>           "max_path_len_zero": true
>>         }
>>       },
>>       "server": {
>>         "usages": ["signing", "key encipherment", "server auth"],
>>         "expiry": "8760h"
>>       },
>>       "client": {
>>         "usages": ["signing", "key encipherment", "client auth"],
>>         "expiry": "8760h"
>>       },
>>       "peer": {
>>         "usages": ["signing", "key encipherment", "server auth", "client auth"],
>>         "expiry": "8760h"
>>       }
>>     }
>>   }
>> }
>> "@ | Out-File -FilePath profiles/signing-config.json -Encoding ASCII
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (6/25)

```
PS C:\enterprise-pki> Get-Content C:\enterprise-pki\profiles\signing-config.json
{
  "signing": {
    "default": {
      "expiry": "8760h"
    },
    "profiles": {
      "intermediate": {
        "usages": ["cert sign", "crl sign"],
        "expiry": "43800h",
        "is_ca": true,
        "ca_constraint": {
          "is_ca": true,
          "max_path_len": 0,
          "max_path_len_zero": true
        }
      },
      "server": {
        "usages": ["signing", "key encipherment", "server auth"],
        "expiry": "8760h"
      },
      "client": {
        "usages": ["signing", "key encipherment", "client auth"],
        "expiry": "8760h"
      },
      "peer": {
        "usages": ["signing", "key encipherment", "server auth", "client auth"],
        "expiry": "8760h"
      }
    }
  }
}
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (7/25)

```
}
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> @"
>> {
>>   "CN": "Enterprise Intermediate CA",
>>   "key": { "algo": "rsa", "size": 4096 },
>>   "names": [{
>>     "C":  "SE",
>>     "ST": "Stockholm",
>>     "L":  "Stockholm",
>>     "O":  "Acme Corp",
>>     "OU": "Security"
>>   }]
>> }
>> "@ | Out-File -FilePath intermediate/intermediate-csr.json -Encoding ASCII
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (8/25)

```
S C:\enterprise-pki>
S C:\enterprise-pki>
S C:\enterprise-pki> cfssl gencert `
>   -ca root\root-ca.pem `
>   -ca-key root\root-ca-key.pem `
>   -config profiles\signing-config.json `
>   -profile intermediate
>   intermediate\intermediate-csr.json | cfssljson -bare intermediate\intermediate-ca
026/05/28 15:06:20 [INFO] generate received request
026/05/28 15:06:20 [INFO] received CSR
026/05/28 15:06:20 [INFO] generating key: rsa-4096
026/05/28 15:06:21 [INFO] encoded CSR
026/05/28 15:06:21 [INFO] signed certificate with serial number 560695358861808077251522508503487780511929956908
026/05/28 15:06:21 [WARNING] This certificate lacks a "hosts" field. This makes it unsuitable for
ebsites. For more information see the Baseline Requirements for the Issuance and Management
f Publicly-Trusted Certificates, v.1.1.6, from the CA/Browser Forum (https://cabforum.org);
pecifically, section 10.2.3 ("Information Requirements").
S C:\enterprise-pki> Get-ChildItem intermediate\


    Directory: C:\enterprise-pki\intermediate


ode                 LastWriteTime         Length Name
---                 -------------         ------ ----
a----         5/27/2026   4:00 PM           4175 ca-chain.pem
a----         5/28/2026   3:06 PM           3243 intermediate-ca-key.pem
a----         5/28/2026   3:06 PM           1732 intermediate-ca.csr
a----         5/28/2026   3:06 PM           2118 intermediate-ca.pem
a----         5/28/2026   3:04 PM            210 intermediate-csr.json


S C:\enterprise-pki> openssl verify -CAfile root\root-ca.pem intermediate\intermediate-ca.pem
ntermediate\intermediate-ca.pem: OK
S C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (9/25)

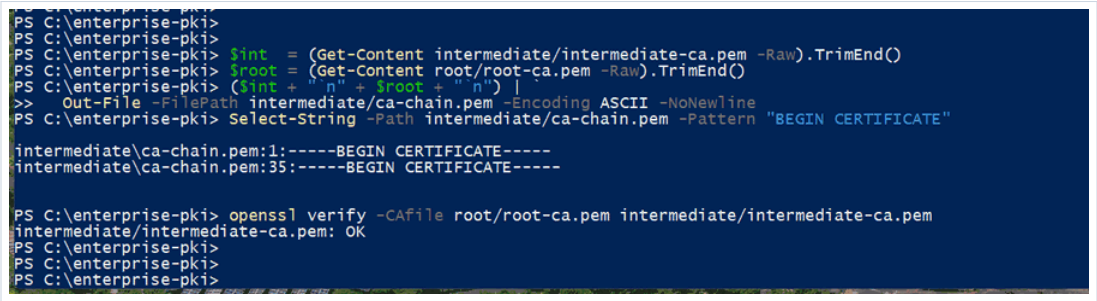


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (10/25)

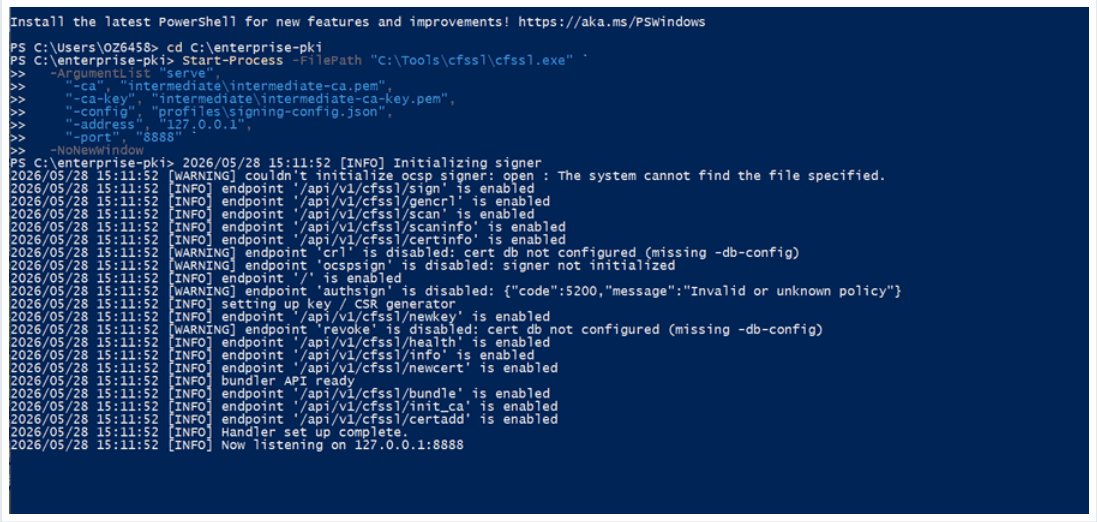


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (11/25)

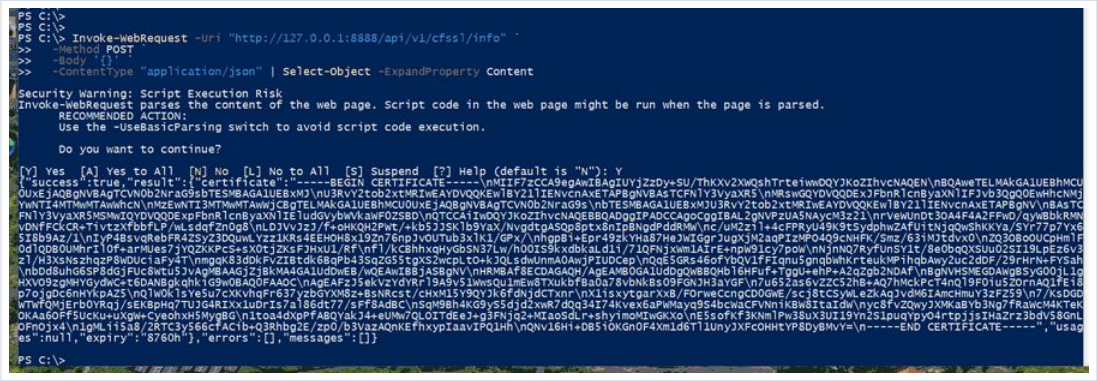


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (12/25)

```
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> @"
>> {
>>   "CN": "api.acme-corp.internal",
>>   "hosts": [
>>     "api.acme-corp.internal",
>>     "api",
>>     "127.0.0.1"
>>   ],
>>   "key": { "algo": "rsa", "size": 2048 },
>>   "names": [{ "C": "SE", "O": "Acme Corp" }]
>> }
>> "@ | Out-File -FilePath certs/server-csr.json -Encoding ASCII
PS C:\enterprise-pki> @"
>> {
>>   "CN": "alice",
>>   "hosts": [],
>>   "key": { "algo": "rsa", "size": 2048 },
>>   "names": [{
>>     "C":  "SE",
>>     "O":  "Acme Corp",
>>     "OU": "Engineering"
>>   }]
>> }
>> "@ | Out-File -FilePath certs/client-csr.json -Encoding ASCII
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (13/25)

```
PS C:\enterprise-pki>
PS C:\enterprise-pki> cfssl gencert `
>>   -remote "http://127.0.0.1:8888" `
>>   -profile server `
>>   certs/server-csr.json | cfssljson -bare certs/api-server
2026/05/28 15:16:46 [INFO] generate received request
2026/05/28 15:16:46 [INFO] received CSR
2026/05/28 15:16:46 [INFO] generating key: rsa-2048
2026/05/28 15:16:46 [INFO] encoded CSR
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> cfssl gencert `
>>   -remote "http://127.0.0.1:8888" `
>>   -profile client `
>>   certs/client-csr.json | cfssljson -bare certs/alice-client
2026/05/28 15:17:06 [INFO] generate received request
2026/05/28 15:17:06 [INFO] received CSR
2026/05/28 15:17:06 [INFO] generating key: rsa-2048
2026/05/28 15:17:06 [INFO] encoded CSR
2026/05/28 15:17:06 [WARNING] This certificate lacks a "hosts" field. This makes it unsuitable for
websites. For more information see the Baseline Requirements for the Issuance and Management
of Publicly-Trusted Certificates, v.1.1.6, from the CA/Browser Forum (https://cabforum.org);
specifically, section 10.2.3 ("Information Requirements").
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki> openssl verify -CAfile intermediate/ca-chain.pem certs/api-server.pem
certs/api-server.pem: OK
PS C:\enterprise-pki> openssl verify -CAfile intermediate/ca-chain.pem certs/alice-client.pem
certs/alice-client.pem: OK
PS C:\enterprise-pki>
PS C:\enterprise-pki>
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (14/25)

```
PS C:\>
PS C:\> Import-Certificate `
>>   -FilePath "C:\enterprise-pki\root\root-ca.pem" `
>>   -CertStoreLocation Cert:\LocalMachine\Root


   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\Root

Thumbprint                                Subject
----------                                -------
AE23509F58C2DF87653DFAAADC9708F1A924861D  CN=Enterprise Root CA, OU=Security, O=Acme Corp, L=Stockholm, S=Stockholm, C=SE


PS C:\>
PS C:\> Get-ChildItem Cert:\LocalMachine\Root |
>>   Where-Object { $_.Subject -like "*Acme Corp*" }


   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\Root

Thumbprint                                Subject
----------                                -------
AE23509F58C2DF87653DFAAADC9708F1A924861D  CN=Enterprise Root CA, OU=Security, O=Acme Corp, L=Stockholm, S=Stockholm, C=SE


PS C:\>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (15/25)

```
PS C:\enterprise-pki>
PS C:\enterprise-pki> @(
>>   "root/root-ca.pem",
>>   "intermediate/intermediate-ca.pem",
>>   "intermediate/ca-chain.pem",
>>   "certs/api-server.pem",
>>   "certs/api-server-key.pem",
>>   "certs/alice-client.pem",
>>   "certs/alice-client-key.pem"
>> ) | ForEach-Object {
>>   $f = Get-Item $_ -ErrorAction SilentlyContinue
>>   if ($f -and $f.Length -gt 0) {
>>     Write-Host "OK   $_ ($($f.Length) bytes)" -ForegroundColor Green
>>   } else {
>>     Write-Host "MISSING  $_" -ForegroundColor Red
>>   }
>> }
OK   root/root-ca.pem (2057 bytes)
OK   intermediate/intermediate-ca.pem (2118 bytes)
OK   intermediate/ca-chain.pem (4175 bytes)
OK   certs/api-server.pem (1781 bytes)
OK   certs/api-server-key.pem (1675 bytes)
OK   certs/alice-client.pem (1724 bytes)
OK   certs/alice-client-key.pem (1675 bytes)
PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (16/25)

```
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2> .\nginx.exe -t
nginx: the configuration file C:\nginx-1.30.2/conf/nginx.conf syntax is ok
nginx: configuration file C:\nginx-1.30.2/conf/nginx.conf test is successful
PS C:\nginx-1.30.2>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (17/25)

```
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2> $hostsPath = "C:\Windows\System32\drivers\etc\hosts"
PS C:\nginx-1.30.2> $entry = "127.0.0.1  api.acme-corp.internal"
PS C:\nginx-1.30.2> if (-not (Get-Content $hostsPath | Select-String "api.acme-corp.internal")) {
>>   Add-Content -Path $hostsPath -Value $entry
>>   Write-Host "Host entry added" -ForegroundColor Green
>> } else {
>>   Write-Host "Host entry already exists" -ForegroundColor Yellow
>> }
Host entry added
PS C:\nginx-1.30.2> start .\nginx.exe
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2>
PS C:\nginx-1.30.2> $proc = Get-Process nginx -ErrorAction SilentlyContinue
PS C:\nginx-1.30.2> if ($proc) {
>>   Write-Host "nginx is running (PID $($proc.Id))" -ForegroundColor Green
>> } else {
>>   Write-Host "nginx failed to start - check nginx -t output" -ForegroundColor Red
>> }
nginx is running (PID 9968 11672 15420 16968 24340 27652)
PS C:\nginx-1.30.2>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (18/25)

```
PS C:\enterprise-pki> openssl s_client `
>>   -connect api.acme-corp.internal:443 `
>>   -servername api.acme-corp.internal `
>>   -CAfile intermediate/ca-chain.pem `
>>   2>$null | Select-String -Pattern "Verify|subject|issuer|BEGIN"

-----BEGIN CERTIFICATE-----
subject=C=SE, O=Acme Corp, CN=api.acme-corp.internal
issuer=C=SE, ST=Stockholm, L=Stockholm, O=Acme Corp, OU=Security, CN=Enterprise Intermediate CA
Verify return code: 0 (ok)
    Verify return code: 0 (ok)
    Verify return code: 0 (ok)
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (19/25)

```
PS C:\enterprise-pki> Write-Host "TEST: No client cert — expecting 400" -ForegroundColor Yellow
TEST: No client cert — expecting 400
PS C:\enterprise-pki> curl.exe -v ` --cacert intermediate/ca-chain.pem ` --ssl-no-revoke ` https://api.acme-corp.internal ` 2>&1 | Select-String -Pattern "40
0|SSL|error|returned|HTTP"

* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* using HTTP/1.x
> GET / HTTP/1.1
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
< HTTP/1.1 400 Bad Request
<head><title>400 No required SSL certificate was sent</title></head>
<center><h1>400 Bad Request</h1></center>
<center>No required SSL certificate was sent</center>

PS C:\enterprise-pki>
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (20/25)

```
PS C:\enterprise-pki> # Test with client cert
PS C:\enterprise-pki> Write-Host "`nTEST: mTLS with client cert" -ForegroundColor Cyan

TEST: mTLS with client cert
PS C:\enterprise-pki> openssl s_client `
>>   -connect api.acme-corp.internal:443 `
>>   -servername api.acme-corp.internal `
>>   -CAfile  intermediate/ca-chain.pem `
>>   -cert    certs/alice-client.pem `
>>   -key     certs/alice-client-key.pem `
>>   2>$null | Select-String "Verify|subject|issuer|client"

subject=C=SE, O=Acme Corp, CN=api.acme-corp.internal
issuer=C=SE, ST=Stockholm, L=Stockholm, O=Acme Corp, OU=Security, CN=Enterprise Intermediate CA
Acceptable client certificate CA names
Verify return code: 0 (ok)
    Verify return code: 0 (ok)
    Verify return code: 0 (ok)
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (21/25)

```
C:\enterprise-pki>
C:\enterprise-pki> openssl pkcs12 -export `
  -in     certs/alice-client.pem `
  -inkey  certs/alice-client-key.pem `
  -certfile intermediate/ca-chain.pem `
  -out    certs/alice-client.p12 `
  -name   "Alice — Acme Corp" `
  -passout pass:test1234
C:\enterprise-pki>
C:\enterprise-pki> Write-Host "P12 file created: certs/alice-client.p12" -ForegroundColor Green
2 file created: certs/alice-client.p12
C:\enterprise-pki> Write-Host "Password: test1234" -ForegroundColor Yellow
ssword: test1234
C:\enterprise-pki>
C:\enterprise-pki>
C:\enterprise-pki> Import-PfxCertificate `
  -FilePath "C:\enterprise-pki\certs\alice-client.p12" `
  -CertStoreLocation Cert:\CurrentUser\My `
  -Password (ConvertTo-SecureString -String "test1234" -AsPlainText -Force)


PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

umbprint                                Subject
--------                                -------
2288830D9E3FE43A23B816497EF13882DA859B  CN=alice, OU=Engineering, O=Acme Corp, C=SE


C:\enterprise-pki> Get-ChildItem Cert:\CurrentUser\My |
  Where-Object { $_.Subject -like "*alice*" }


PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

umbprint                                Subject
--------                                -------
2288830D9E3FE43A23B816497EF13882DA859B  CN=alice, OU=Engineering, O=Acme Corp, C=SE
```

Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (22/25)

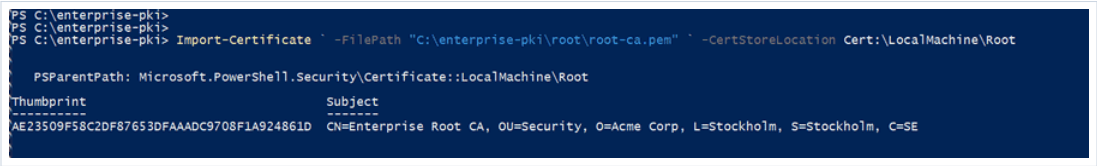


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (23/25)

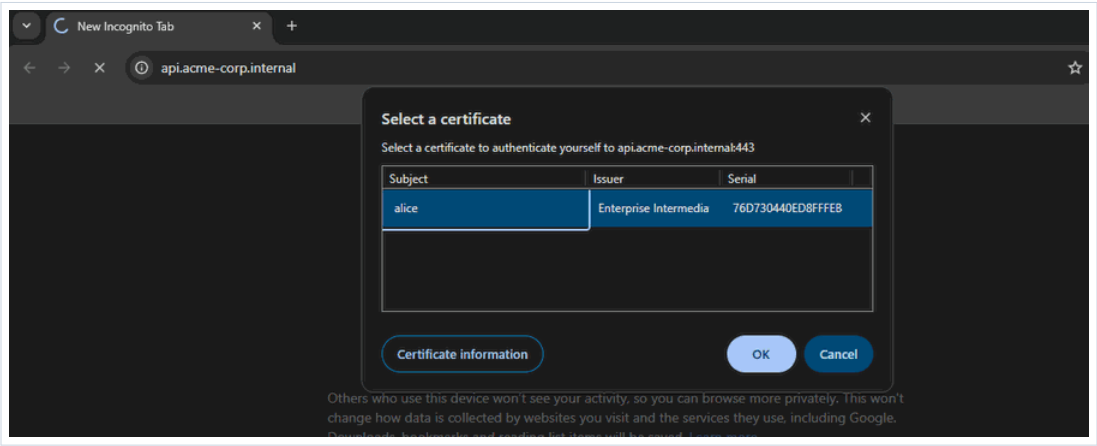


Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (24/25)



Figure — Tier 2 lab: enterprise PKI with CFSSL & mTLS (25/25)

Tier 3 — Internet PKI with Let's Encrypt

The third tier obtains a real, publicly-trusted certificate for a live domain, using Certbot over the ACME protocol. Unlike Tiers 1 and 2 there is no local CA to build and no trust store to edit — Certbot proves control of the domain, fetches a short-lived (90-day) certificate from Let's Encrypt, installs it, and sets up automatic renewal.

```
# Obtain and install a real certificate, with auto-renewal
$ sudo certbot --nginx -d example.com -d www.example.com
$ sudo certbot renew --dry-run          # confirm renewal works
# certificates land in /etc/letsencrypt/live/example.com/
# a systemd timer (certbot.timer) renews them automatically before expiry
```

Reference — key commands (331.1)

Command	Purpose
`openssl genpkey`	Generate a private key (RSA, EC, ...).

Command	Purpose
`openssl pkey -pubout`	Extract the public key from a private key.
`openssl req -new`	Create a Certificate Signing Request.
`openssl req -x509`	Create a self-signed certificate (e.g. a root CA).
`openssl x509 -req`	Sign a CSR into a certificate.
`openssl x509 -text`	Inspect a certificate in human-readable form.
`openssl x509 -dates / -checkend`	Show / test the validity period.
`openssl ca`	Operate a CA: issue, revoke, generate CRLs.
`openssl pkcs8 / pkcs12`	Convert key formats / bundle key + cert.
`certbot`	Automated Let's Encrypt (ACME) certificates.
`cfssl / cfssljson`	Programmatic internal PKI.

Exam tips — 331.1

- **An X.509 certificate binds a public key to an identity, signed by a CA**; inspect it with `openssl x509 -text -noout`.
- **Basic Constraints CA:true/false decides if a cert is a CA**; Key Usage (keyCertSign, cRLSign, digitalSignature, keyEncipherment) and EKU (serverAuth, clientAuth) restrict use.
- **Modern hostname checks use the SAN, not the CN**; AKI/SKI link a cert to its issuer; CRL DP and AIA point to revocation info.
- **genpkey makes keys, req makes CSRs/self-signed certs, x509 -req signs CSRs**; `pkey -pubout` extracts the public key.
- **Know the formats: PEM (text), DER (binary), PKCS#8 (modern private key), PKCS#12 (key+cert bundle)**.
- **Trust flows Root → Intermediate → leaf; keep the root offline**; revoke via CRL or OCSP; CT logs make issuance auditable.
- **Let's Encrypt uses ACME (Certbot) for automated 90-day certs; CFSSL runs an internal PKI**.

331.2 X.509 Certificates for Encryption, Signing and Authentication

Objective weight: 4

Overview

TLS (Transport Layer Security) is the technology that turns an ordinary, readable TCP connection into a private one. It does three things for you: it keeps the traffic secret (confidentiality), it makes sure nobody changed the data in transit (integrity), and it proves you are really talking to the right server (authentication).

The proof of identity comes from an **X.509 certificate** — a small file that binds a public key to a name (like `www.example.com`), signed by a trusted **Certificate Authority (CA)**. TLS is the modern name; the old name was SSL. People still mix the two words, which is why the Apache module is called `mod_ssl` even though it does TLS.

1. SSL and TLS Fundamentals

1.1 Which protocol versions remain acceptable?

New versions of the protocol have appeared over the years, and the older ones are now considered unsafe. For the exam you need to know which are still acceptable. The essential rule: TLS 1.2 is the oldest version you should still allow, and TLS 1.3 is the current best.

Version	Year	Status — in brief
SSLv2	1995	Completely broken. Banned. Removed from modern OpenSSL.
SSLv3	1996	Broken (the POODLE attack). Banned.
TLS 1.0	1999	Too old — must be turned off.
TLS 1.1	2006	Too old — must be turned off.
TLS 1.2	2008	Fine, if you pick modern ciphers. The minimum you should allow.
TLS 1.3	2018	The current version. Faster, simpler, only strong ciphers.

1.2 The TLS handshake

Before any real data is sent, the two sides run a short conversation called the handshake. It always accomplishes the same three jobs: agree on the protocol version and cipher, let the server (and sometimes the client) prove its identity with a certificate, and work out the shared secret keys that will encrypt the rest of the session.

In **TLS 1.2** this takes **two round trips**. The client starts with a `ClientHello` (its supported versions and ciphers, a random number, and the SNI host name). The server replies with `ServerHello`, its `Certificate`, and a signed key-exchange message — that signature is how the certificate actually authenticates the connection. The client checks the certificate chain against its trust store, sends its own half of the key exchange, and both sides switch to encrypted mode with `ChangeCipherSpec` and `Finished`.

TLS 1.3 does the same work in a single round trip. The client already includes its key material (`key_share`) in the very first message, and the server encrypts everything after `ServerHello` — including its own certificate. Only ephemeral (single-use) key exchange is allowed, so every TLS 1.3 connection automatically has **forward secrecy** (recording the traffic today and stealing the key later still will not decrypt it). When client certificates

are used, the server adds a `CertificateRequest` and the client answers with its own `Certificate` and `CertificateVerify`.

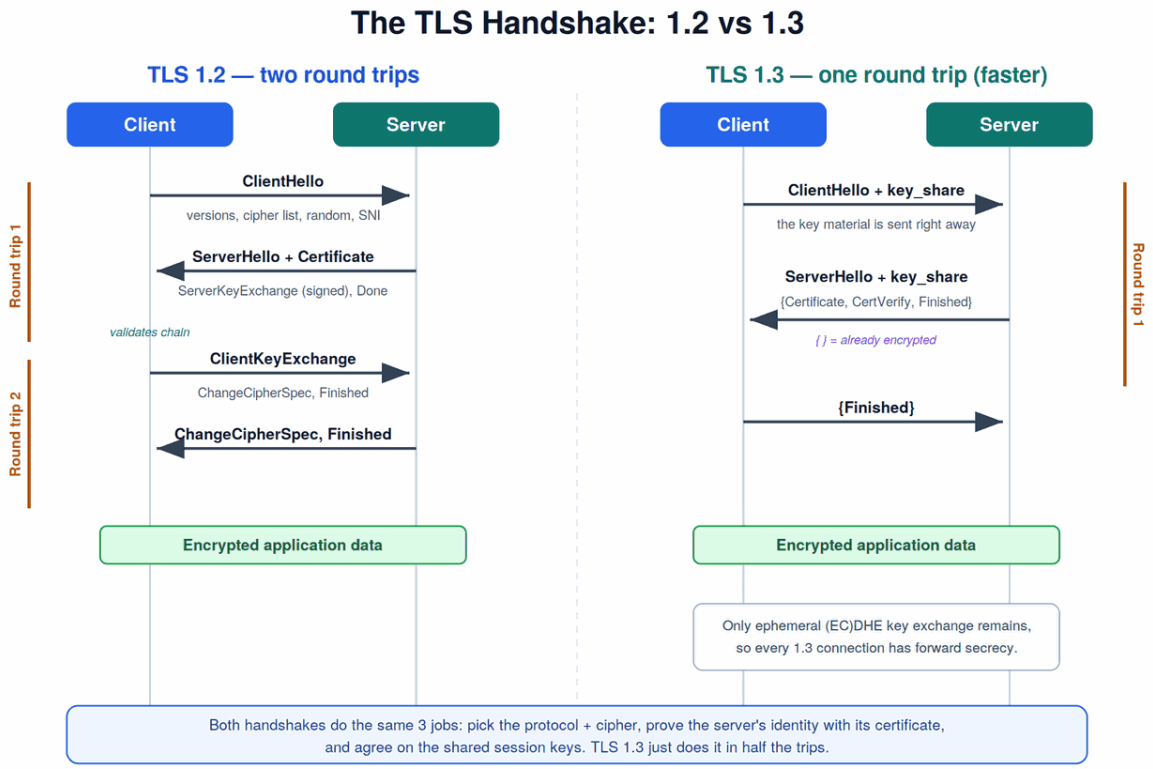


Figure 1 — The TLS 1.2 handshake (two round trips) next to the leaner TLS 1.3 handshake (one round trip).

1.3 Cipher suites

A cipher suite is simply the list of algorithms a TLS session will use. It has four parts, and in TLS 1.2 the OpenSSL name spells all four out. Take this example:

```
ECDHE-RSA-AES256-GCM-SHA384
```

Reading it left to right: the key exchange is ECDHE, the server's identity is proven with an RSA key, the data is encrypted with AES-256 in GCM mode, and SHA-384 is the handshake hash. TLS 1.3 made this shorter — a suite like TLS_AES_256_GCM_SHA384 only names the cipher and hash, because the key exchange and authentication are negotiated separately (and are always the forward-secret kind).

The four parts, in plain terms:

- **Key exchange (Kx)** — how the two sides agree on the shared secret. Old RSA key transport is risky: if the server key ever leaks, every recorded past session can be decrypted. DHE and ECDHE use single-use values instead, giving forward secrecy. TLS 1.3 keeps only these.
- **Authentication (Au)** — how the server proves who it is: it signs handshake values with the private key of its certificate. The algorithm (RSA or ECDSA) has to match the certificate you installed. Suites marked `aNULL` skip authentication entirely and must always be excluded (`!aNULL`).

- **Bulk encryption (Enc)** — the fast symmetric cipher that scrambles the actual data: AES-128/256 (GCM or older CBC) or ChaCha20-Poly1305. Old ciphers like RC4, DES and 3DES are weak and must be excluded.
- **Message integrity (Mac)** — protection against tampering. Modern AEAD ciphers (AES-GCM, ChaCha20-Poly1305) do encryption and integrity in one step; `openssl ciphers -v` shows them as `Mac=AEAD`.

Use `openssl ciphers` to expand a cipher string into the actual suites it selects — this is exactly how Apache reads `SSLCipherSuite`. The columns line up with the four parts above:

```
$ openssl ciphers -v 'HIGH:!aNULL:!MD5' | head -3
TLS_AES_256_GCM_SHA384         TLSv1.3 Kx=any   Au=any   Enc=AESGCM(256)  Mac=AEAD
TLS_CHACHA20_POLY1305_SHA256   TLSv1.3 Kx=any   Au=any   Enc=CHACHA20...  Mac=AEAD
ECDHE-ECDSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH  Au=ECDSA Enc=AESGCM(256) Mac=AEAD

# Only TLS 1.3 suites
$ openssl ciphers -s -tls1_3 -v
```

How cipher strings are written

Keywords are combined with separators: `HIGH` (strong ciphers), `!aNULL` (permanently remove), `!MD5`, `+` (move to the end), `-` (remove, can be re-added).

The exam does not ask you to memorise individual ciphers — but you must be able to read and write these strings.

2. Apache HTTPD and mod_ssl

Apache is built from modules. Its core listens on the addresses and ports set by `Listen` directives and passes each request to a matching virtual host. TLS is not in the core — it is added by `mod_ssl`, which wraps the OpenSSL library and steps in on every connection to a TLS port. On port 443 mod_ssl runs the handshake, picks the certificate for the host named in SNI, and only then hands the decrypted request on to the right `<VirtualHost>`.

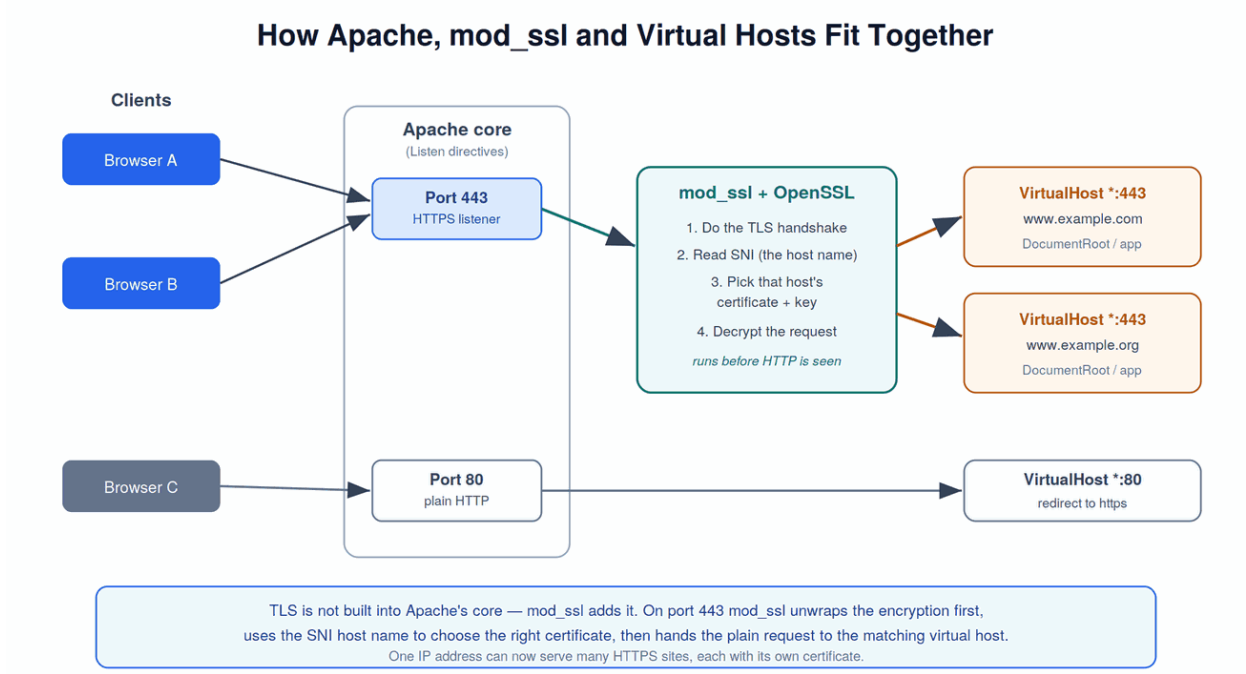


Figure 2 — Clients reach Apache's listeners; mod_ssl handles TLS on port 443 and routes to name-based virtual hosts.

2.1 Installing on Linux

Install Apache from your distribution's repositories. The package, service and file names differ between the two main families.

```
# Debian / Ubuntu
$ sudo apt install apache2
$ sudo a2enmod ssl headers          # enable modules
$ sudo a2ensite default-ssl         # enable the sample HTTPS site
$ sudo systemctl enable --now apache2

# RHEL / AlmaLinux / Rocky / Fedora
$ sudo dnf install httpd mod_ssl    # mod_ssl adds Listen 443 + ssl.conf
$ sudo systemctl enable --now httpd
$ sudo firewall-cmd --permanent --add-service={http,https} && sudo firewall-cmd --
reload
```

2.2 Installing on Windows

The Apache project publishes no official Windows binaries. The usual sources are the Apache Lounge builds (plus the matching Visual C++ runtime) or a bundle like XAMPP. The Apache Lounge ZIP is just extracted to `C:\Apache24`.

```
> cd C:\Apache24\bin
> httpd.exe -k install              # register as a Windows service
> httpd.exe -t                     # configtest
> httpd.exe -k start

# Enable TLS: edit C:\Apache24\conf\httpd.conf and uncomment:
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so
Include conf/extra/httpd-ssl.conf
```

The HTTPS virtual host lives in `conf\extra\httpd-ssl.conf` and expects `server.crt` and `server.key` in `C:\Apache24\conf`. OpenSSL ships with the Apache Lounge build, so every openssl example here works on Windows too.

2.3 Installing on macOS

macOS ships an outdated, deprecated Apache. For real work, install the current version with Homebrew.

```
$ brew install httpd
$ brew services start httpd         # listens on port 8080 by default

# Config (Apple Silicon; use /usr/local on Intel Macs)
$ vi /opt/homebrew/etc/httpd/httpd.conf       # change Listen 8080 -> 80
# Uncomment, as on Windows: ssl_module, socache_shmcb, httpd-ssl.conf
```

2.4 Where the config files live

Whenever this guide shows a config snippet, put it in (or compare it with) the right file for your platform. The examples use RHEL-style paths — translate with this table.

Platform	Main config	TLS config	Cert/key location
RHEL family	`/etc/httpd/conf/httpd.conf`	`conf.d/ssl.conf`	`/etc/pki/tls/certs, /private`
Debian/Ubuntu	`/etc/apache2/apache2.conf`	`sites-available/default-ssl.conf`	`/etc/ssl/certs, /private`
Windows	`conf\httpd.conf`	`conf\extra\httpd-ssl.conf`	`C:\Apache24\conf`
macOS (brew)	`etc/httpd/httpd.conf`	`extra/httpd-ssl.conf`	`etc/httpd`

Debian splits its config into `conf-/mods-/sites-available` folders, with `a2enmod`/`a2ensite` managing the symlinks. RHEL keeps one `httpd.conf` plus drop-ins in `conf.d/`. The test command `apachectl configtest` (or `httpd -t`) works the same everywhere.

2.5 The SSLEngine switch

The master on/off switch for TLS is `SSLEngine`, set per virtual host. It has three modes:

- `SSLEngine on` — treat the connection as TLS from the first byte. This is the normal mode for every HTTPS host. Without it, all other `SSL*` directives in that host are ignored.
- `SSLEngine off` — the default: plain HTTP, no TLS. Used for port-80 hosts, like the redirect host in Section 6.
- `SSLEngine optional` — allows an upgrade to TLS on the same port (RFC 2817). No browser implements this, so it is basically never used — but it explains why the mode exists.

Before you debug any SSL directive, confirm mod_ssl is actually loaded and the engine is on:

```
$ httpd -M | grep ssl              # apache2ctl -M on Debian
 ssl_module (shared)
$ httpd -S                         # dump vhosts: shows which listen on 443
```

3. Building a Test PKI with OpenSSL

All the examples below use a small lab setup: one private CA, one server certificate and one client certificate. First create the CA — its own key, plus a self-signed certificate that will vouch for everything else.

```
# CA private key and self-signed CA certificate (10 years)
$ openssl genrsa -aes256 -out ca.key 4096
$ openssl req -new -x509 -key ca.key -days 3650 \
    -subj "/C=DE/O=Example Lab/CN=Example Lab Root CA" -out ca.crt
```

Then make the web server's key and a Certificate Signing Request (CSR), and have the CA sign it. Modern browsers require the host name in the **subjectAltName** (SAN) extension, not only in the CN.

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out server.key
$ openssl req -new -key server.key -subj "/CN=www.example.com" -out server.csr
$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -days 365 -extfile <(printf
"subjectAltName=DNS:www.example.com,DNS:example.com") \
    -out server.crt
```

Then inspect and verify the result:

```
$ openssl x509 -in server.crt -noout -text          # full dump
$ openssl x509 -in server.crt -noout -subject -issuer -dates -ext subjectAltName
$ openssl verify -CAfile ca.crt server.crt
server.crt: OK

# Does the private key match the certificate? Compare public-key hashes:
$ openssl x509 -in server.crt -noout -pubkey | openssl sha256
$ openssl pkey -in server.key -pubout | openssl sha256
```

That last pair is a useful trick: if the two SHA-256 hashes match, the key and the certificate belong together.

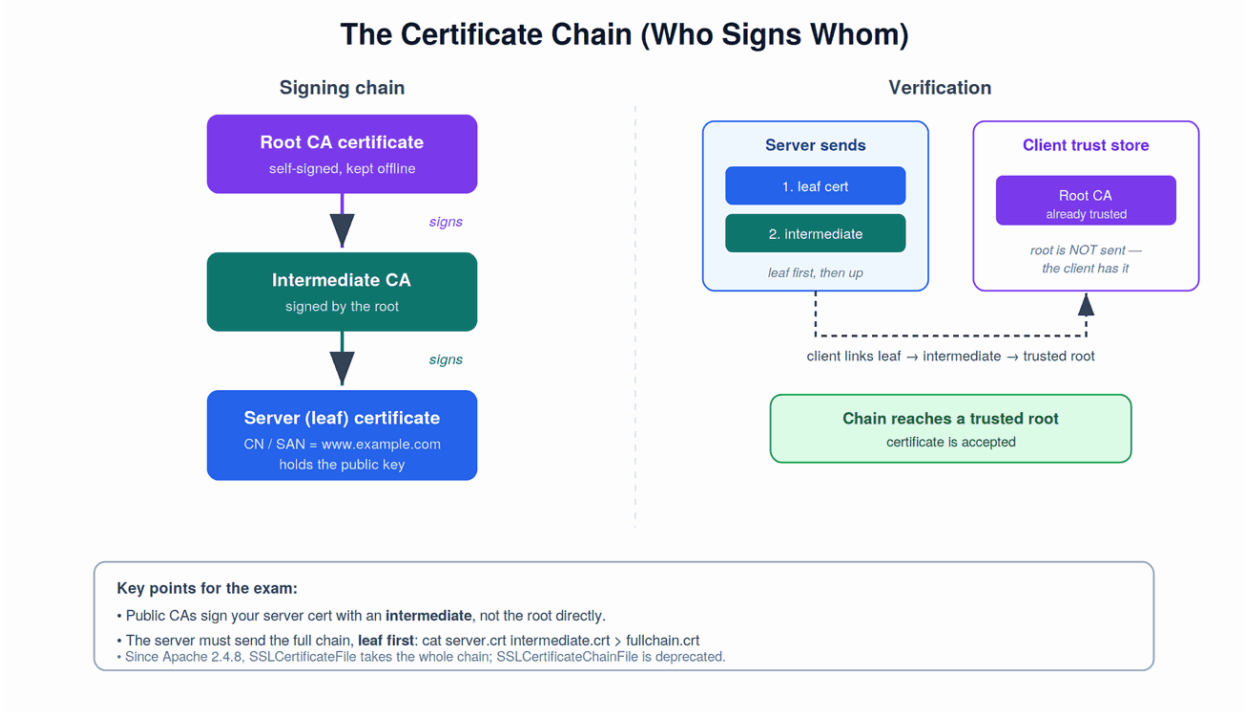


Figure 3 — A certificate chain: the root signs the intermediate, the intermediate signs your server. Clients trust the root and build the path up to it.

4. HTTPS with Apache and mod_ssl

On Debian/Ubuntu, enable the module with `a2enmod ssl`; on RHEL, install the `mod_ssl` package (it loads the module and adds `Listen 443 https`). A minimal HTTPS virtual host looks like this:

```
Listen 443 https

<VirtualHost *:443>
    ServerName www.example.com
    DocumentRoot /var/www/html

    SSLEngine               on
    SSLCertificateFile      /etc/pki/tls/certs/server.crt
    SSLCertificateKeyFile   /etc/pki/tls/private/server.key
</VirtualHost>
```

The key file must be readable only by root (`chmod 600`). If the key is encrypted, Apache asks for the passphrase at startup or uses `SSLPassPhraseDialog`. Check and apply the config:

```
$ apachectl configtest
Syntax OK
$ systemctl reload httpd      # or apache2
$ curl --cacert ca.crt https://www.example.com/
```

4.1 Serving certificate chains

Public CAs sign your server certificate with an **intermediate** CA, not the root. So the server has to send a **chain**: its own certificate first, then the intermediate(s). The client already trusts the root locally, so the root itself is not sent. Since Apache 2.4.8 you just put the whole chain in one file and point `SSLCertificateFile` at it — the old `SSLCertificateChainFile` directive is deprecated.

```
# Order matters: leaf first, then intermediates
$ cat server.crt intermediate.crt > server-fullchain.crt

SSLCertificateFile      /etc/pki/tls/certs/server-fullchain.crt
SSLCertificateKeyFile   /etc/pki/tls/private/server.key

# Pre-2.4.8 style, still seen in older configs:
# SSLCertificateChainFile /etc/pki/tls/certs/intermediate.crt
```

Verify the full chain is actually being served:

```
$ openssl s_client -connect www.example.com:443 -showcerts </dev/null \
    | grep -E 's:|i:'
 0 s:CN = www.example.com
   i:CN = Example Intermediate CA
 1 s:CN = Example Intermediate CA
   i:CN = Example Lab Root CA
```

4.2 Restricting protocols and ciphers

Protocol versions are controlled by `SSLProtocol`, ciphers by `SSLCipherSuite`. A solid current setup:

```
SSLProtocol             all -SSLv3 -TLSv1 -TLSv1.1     # i.e. TLS 1.2 + 1.3
SSLCipherSuite          HIGH:!aNULL:!MD5:!3DES
SSLCipherSuite          TLSv1.3 TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
SSLHonorCipherOrder     on        # prefer the server's order (TLS <= 1.2)
```

Good to know

`SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1` and `SSLProtocol -all +TLSv1.2 +TLSv1.3` mean exactly the same thing.

On Apache 2.4.36+ with OpenSSL 1.1.1+, the optional first word `TLSv1.3` on `SSLCipherSuite` sets the 1.3 suites separately, because they use a different OpenSSL API.

`SSLHonorCipherOrder on` lets the server, not the client, choose the suite (TLS 1.2 and earlier).

5. Name-Based HTTPS Virtual Hosts (SNI)

Long ago, every HTTPS site needed its own IP address, because the server must present a certificate before it can read the `Host:` header. **Server Name Indication (SNI)** fixed this: the client puts the host name it wants in the very first message (the `ClientHello`), and Apache uses it to pick the matching virtual host and certificate. In Apache 2.4 nothing special needs enabling — just define several `<VirtualHost *:443>` blocks with different names and certificates.

```
<VirtualHost *:443>
    ServerName www.example.com
    SSLEngine on
    SSLCertificateFile    /etc/pki/tls/certs/example.com-fullchain.crt
    SSLCertificateKeyFile /etc/pki/tls/private/example.com.key
</VirtualHost>

<VirtualHost *:443>
    ServerName www.example.org
    SSLEngine on
    SSLCertificateFile    /etc/pki/tls/certs/example.org-fullchain.crt
    SSLCertificateKeyFile /etc/pki/tls/private/example.org.key
</VirtualHost>
```

A client that sends no SNI (or an unknown name) gets the first matching virtual host — the default. To reject clients that send no SNI instead, set `SSLStrictSNIVHostCheck on` in the default host. You can test which certificate comes back for a given name:

```
$ openssl s_client -connect 192.0.2.10:443 -servername www.example.org </dev/null \
    | openssl x509 -noout -subject
subject=CN = www.example.org

# Without -servername no SNI is sent: the default vhost certificate appears
```

6. HTTP Strict Transport Security (HSTS)

HSTS tells a browser: “from now on, only ever reach this site over HTTPS.” That blocks downgrade attacks and stops cookies leaking over plain HTTP. It is just one response header, added with `mod_headers` (`a2enmod headers` on Debian).

```
<VirtualHost *:443>
    ServerName www.example.com
    SSLEngine on
    ...
    Header always set Strict-Transport-Security \
        "max-age=31536000; includeSubDomains"
</VirtualHost>

# Redirect all plain HTTP to HTTPS
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>
```

The `max-age` value is in seconds (`31536000` = one year) and says how long the browser enforces HTTPS. `includeSubDomains` extends it to every subdomain; the optional `preload` token allows the site onto browser preload lists. Use `always` so the header is set even on error pages. Browsers only honour the header when it arrives over HTTPS. Verify:

```
$ curl -sI https://www.example.com/ | grep -i strict
strict-transport-security: max-age=31536000; includeSubDomains
```

Deploy HSTS carefully

A long `max-age` effectively locks browsers into HTTPS for your site. Turn it on only once your certificate setup is stable, and start with a short `max-age` while testing.

7. Authenticating Users with Client Certificates

TLS can also work the other way around: the server can ask the **client** for a certificate and check it against a CA. This is mutual TLS (mTLS), and it can replace or add to passwords. First, make a client certificate with the lab CA and bundle it as a PKCS#12 file so it can be imported into a browser.

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out alice.key
$ openssl req -new -key alice.key \
    -subj "/O=Example Lab/CN=Alice Example" -out alice.csr
$ openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -days 365 -out alice.crt

# Browser/OS import format (prompts for an export password)
$ openssl pkcs12 -export -inkey alice.key -in alice.crt \
    -certfile ca.crt -name "Alice Example" -out alice.p12
```

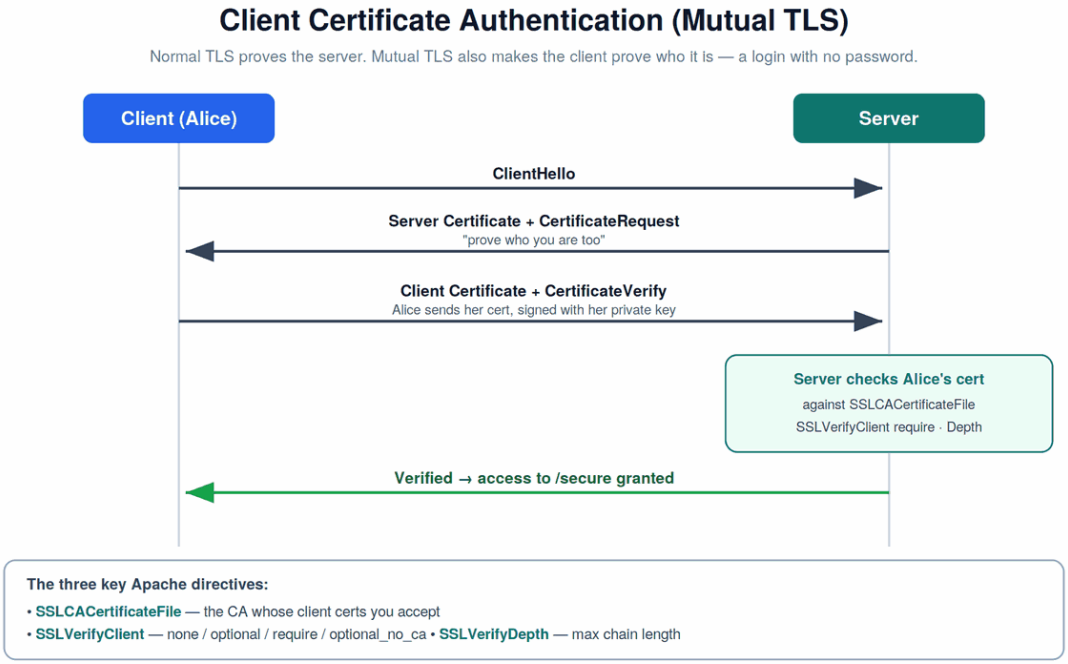


Figure 4 — Mutual TLS: the server asks for a certificate, the client presents one, and the server verifies it against the configured CA.

7.1 Apache configuration

Three directives do the work: `SSLCACertificateFile` (which CA's client certificates you accept), `SSLVerifyClient` (whether a certificate is required) and `SSLVerifyDepth` (how long the chain may be). You can set them for the whole host or, via renegotiation, per directory.

```
<VirtualHost *:443>
    ServerName www.example.com
    SSLEngine on
    SSLCertificateFile      /etc/pki/tls/certs/server-fullchain.crt
    SSLCertificateKeyFile   /etc/pki/tls/private/server.key

    SSLCACertificateFile    /etc/pki/tls/certs/ca.crt
```

```
    <Location /secure>
        SSLVerifyClient require
        SSLVerifyDepth  2
        SSLOptions      +StdEnvVars +ExportCertData
    </Location>
</VirtualHost>
```

`SSLVerifyClient` takes `none`, `optional`, `require` and `optional_no_ca` (accept any certificate without checking it — testing only). `+StdEnvVars` exports certificate details such as `SSL_CLIENT_S_DN_CN` to your application; `+ExportCertData` also exports the whole certificate (`SSL_CLIENT_CERT`).

7.2 Authorising on certificate fields

You can go further than "any valid certificate" and allow only specific subjects. In Apache 2.4 the preferred way is `Require expr`; the older `SSLRequire` is deprecated but still on the exam and in old configs.

```
<Location /secure>
    SSLVerifyClient require
    SSLVerifyDepth  2

    # Apache 2.4 style
    Require expr "%{SSL_CLIENT_S_DN_O} == 'Example Lab'"

    # Deprecated equivalent
    # SSLRequire %{SSL_CLIENT_S_DN_O} eq "Example Lab"
</Location>
```

Two more directives tie certificates into classic auth: `SSLOptions +FakeBasicAuth` turns the certificate's subject DN into a Basic-Auth username (with the fixed password `xxj31ZMTZzkVA`, the crypt hash of "password", in an `AuthUserFile`), and `SSLUserName SSL_CLIENT_S_DN_CN` sets the app-visible username (`REMOTE_USER`) from a certificate field.

7.3 Testing client authentication

```
# Without a certificate: rejected
$ curl --cacert ca.crt https://www.example.com/secure/
curl: (56) ... alert certificate required ...

# With certificate and key
$ curl --cacert ca.crt --cert alice.crt --key alice.key \
    https://www.example.com/secure/

# The same with openssl
$ openssl s_client -connect www.example.com:443 -CAfile ca.crt \
    -cert alice.crt -key alice.key
```

8. OCSP Stapling

OCSP (Online Certificate Status Protocol) lets a client ask the CA "has this certificate been revoked?" Doing that directly is slow and tells the CA which sites you visit. **OCSP stapling** fixes both: the web server fetches a signed,

time-stamped OCSP answer itself and “staples” it into the handshake, so the client gets fresh revocation status without contacting the CA.

```
# Global (outside any <VirtualHost>): cache for stapled responses
SSLStaplingCache "shmcb:/var/run/ocsp(128000)"

<VirtualHost *:443>
    ServerName www.example.com
    SSLEngine on
    ...
    SSLUseStapling                  on
    SSLStaplingResponderTimeout     5
    SSLStaplingReturnResponderErrors  off
</VirtualHost>
```

`SSLStaplingCache` is mandatory and must be defined **globally**. `SSLStaplingReturnResponderErrors off` stops temporary responder failures being passed to clients. Stapling needs a CA-issued certificate that contains an OCSP responder URL (the Authority Information Access extension) — self-signed lab certificates have none. Test with `openssl s_client -status`:

```
$ openssl s_client -connect www.example.com:443 -status </dev/null 2>/dev/null \
    | grep -A 5 'OCSP response'
OCSP response:
======================================
OCSP Response Data:
    OCSP Response Status: successful (0x0)
    Cert Status: good

# Without stapling: "OCSP response: no response sent"
```

9. Testing TLS with the OpenSSL Toolkit

9.1 openssl s_client

This is the universal TLS test client — you point it at a server and see exactly what happens. The options you will use most:

Option	What it does
`-connect host:port`	The target server.
`-servername name`	Send SNI (also picks the right virtual host).
`-showcerts`	Print the whole chain the server sends.
`-CAfile ca.crt`	CA used to verify the server certificate.
`-status`	Request and show a stapled OCSP response.
`-cert / -key`	Client certificate and key for mutual TLS.
`-tls1_2, -tls1_3, -no_tls1`	Force or exclude protocol versions.
`-cipher / -ciphersuites`	Offer specific TLS <=1.2 ciphers / TLS 1.3 suites.
`-brief`	Short summary: protocol, cipher, verification.

Some single commands worth remembering:

```
# Which protocol and cipher were negotiated?
$ openssl s_client -connect www.example.com:443 -brief </dev/null

# Does the server still accept TLS 1.0? (should fail)
$ openssl s_client -connect www.example.com:443 -tls1 </dev/null

# When does the certificate expire?
$ echo | openssl s_client -connect www.example.com:443 \
    -servername www.example.com 2>/dev/null | openssl x509 -noout -dates
notBefore=Jun 10 00:00:00 2026 GMT
notAfter=Jun 10 23:59:59 2027 GMT

# Test STARTTLS services as well
$ openssl s_client -connect mail.example.com:25 -starttls smtp
```

9.2 openssl s_server

A tiny TLS server — perfect for testing certificates and clients before you touch the Apache config.

```
# Simple HTTPS test server on port 4443 (-www returns a status page)
$ openssl s_server -accept 4443 -cert server.crt -key server.key -www

# Demand client certificates signed by our CA (depth 1)
$ openssl s_server -accept 4443 -cert server.crt -key server.key \
    -CAfile ca.crt -Verify 1 -www

# Connect from a second terminal
$ openssl s_client -connect localhost:4443 -CAfile ca.crt \
```

```
    -cert alice.crt -key alice.key
```

Capital `-Verify` requires a client certificate; lowercase `-verify` only requests one.

10. Key mod_ssl Directives at a Glance

Directive	What it is for
`SSLEngine on`	Enable TLS for the virtual host.
`SSLCertificateFile`	Server certificate; since 2.4.8 may hold the full chain.
`SSLCertificateKeyFile`	Server private key.
`SSLCertificateChainFile`	Intermediate CA file (deprecated since 2.4.8).
`SSLProtocol`	Allowed protocol versions.
`SSLCipherSuite`	Allowed cipher suites (OpenSSL cipher string).
`SSLHonorCipherOrder`	Server picks the cipher (TLS <= 1.2).
`SSLCACertificateFile`	CA(s) for verifying client certificates.
`SSLVerifyClient`	none | optional | require | optional_no_ca.
`SSLVerifyDepth`	Max client chain verification depth.
`SSLOptions`	+StdEnvVars +ExportCertData +FakeBasicAuth ...
`SSLUserName`	Set REMOTE_USER from an SSL variable.
`SSLRequire / Require expr`	Authorise based on certificate fields.
`SSLUseStapling / SSLStaplingCache`	OCSP stapling / the mandatory global cache.
`SSLStrictSNIVHostCheck`	Reject clients that send no SNI.
`Header ... Strict-Transport-Security`	HSTS (via mod_headers).

Exam tips — the things people get wrong

- **Chain order is leaf first**, then the intermediate(s). The root is never sent.
- `SSLCertificateChainFile` has been obsolete since Apache 2.4.8 — put the chain in `SSLCertificateFile` instead.
- `SSLStaplingCache` is **global**, while `SSLUseStapling` is per-virtual-host.
- Know the four `SSLVerifyClient` values: `none, optional, require, optional_no_ca`.
- Remember the key `s_client` options: `-servername, -showcerts, -status, -cert/-key`.

331.3 Encrypted File Systems

Objective weight: 3

Overview

Files sitting on a disk are called data "at rest". If someone steals the laptop, pulls the drive, or obtains a backup tape, ordinary files are fully exposed — the login password protects nothing once the disk is read on another machine. Encryption at rest fixes that: the bytes on the disk are scrambled, and without the right key they are useless.

Linux offers two families of tools, and the whole objective is really about knowing which to reach for and how to drive it. The first family encrypts a whole block device underneath the file system (dm-crypt with LUKS). The second encrypts individual files on top of a normal file system (eCryptfs). We will look at each, then at how to unlock devices automatically with Clevis, TPM2 and Tang.

1. Two Ways to Encrypt: Block Device vs File System

The key idea is **where** in the storage stack the scrambling happens. Everything below that point is ciphertext; everything above it sees plain data.

1.1 Block-device (full-disk) encryption

Here the encryption sits **below** the file system, at the block layer. The file system (ext4, xfs, …) is created inside the encrypted device and never knows encryption is happening — it just reads and writes blocks, and the **dm-crypt** layer scrambles them on the way to disk and unscrambles them on the way back. Because it works on raw blocks, **everything** is protected: file contents, file names, sizes, permissions, the directory structure, even the free space. This is what you want for a laptop or a whole partition.

1.2 File-system (per-file) encryption

Here the encryption sits **above** an existing file system. Tools like **eCryptfs** are “stacked” file systems: you point them at a normal directory (the lower directory, holding ciphertext) and mount a decrypted view on top. Each file is encrypted on its own, so you can even copy an encrypted file to a USB stick and it stays protected. It needs no dedicated partition and is perfect for protecting a single user’s home directory — but it does not hide how many files you have or how much free space is left.

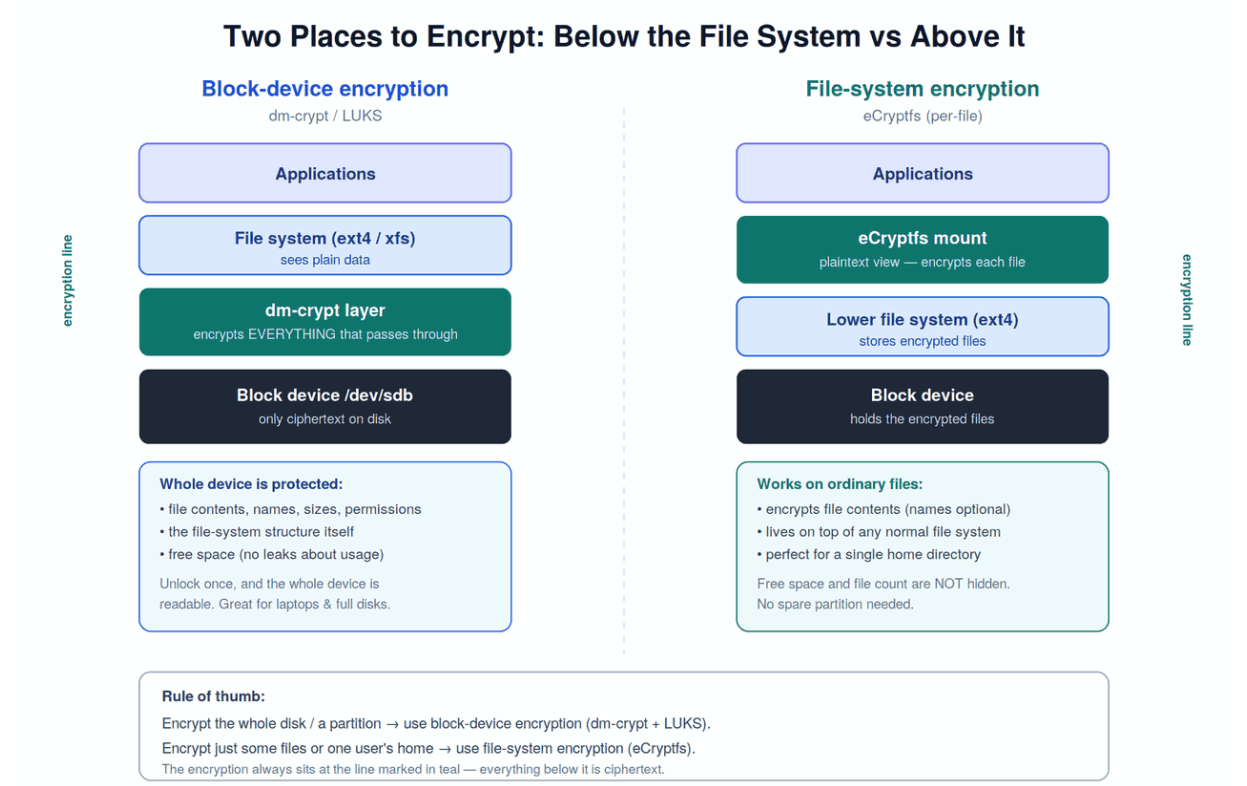


Figure 1 — Block-device encryption scrambles everything below the file system; file-system encryption protects individual files above it.

Question	Block device (dm-crypt/LUKS)	File system (eCryptfs)
Where it sits	Below the file system	Above the file system
What is hidden	Everything, incl. free space & metadata	File contents (names optional)
Needs its own partition?	Yes (or a file used as a loop device)	No — reuses a normal directory
Granularity	The whole device, all at once	Per file
Best for	Laptops, whole disks, partitions	One home directory, a ~/Private folder
Copies stay encrypted?	No (decrypted once the device is open)	Yes (each file is self-contained)

2. dm-crypt and the Device Mapper

On Linux, block-device encryption is done by **dm-crypt**, a target of the kernel's **device mapper** — the same framework behind LVM and software RAID. The device mapper builds a new virtual block device that maps onto a real one; the `crypt` target adds encryption to that mapping. In practice you rarely talk to dm-crypt directly; you use the friendly front-end **cryptsetup**.

The flow is always the same: take a raw partition, turn it into an encrypted device, "open" it to get a decrypted door at `/dev/mapper/<name>`, then put a normal file system on that door and mount it. Writes going in are encrypted; reads coming out are decrypted; the real partition only ever holds ciphertext.

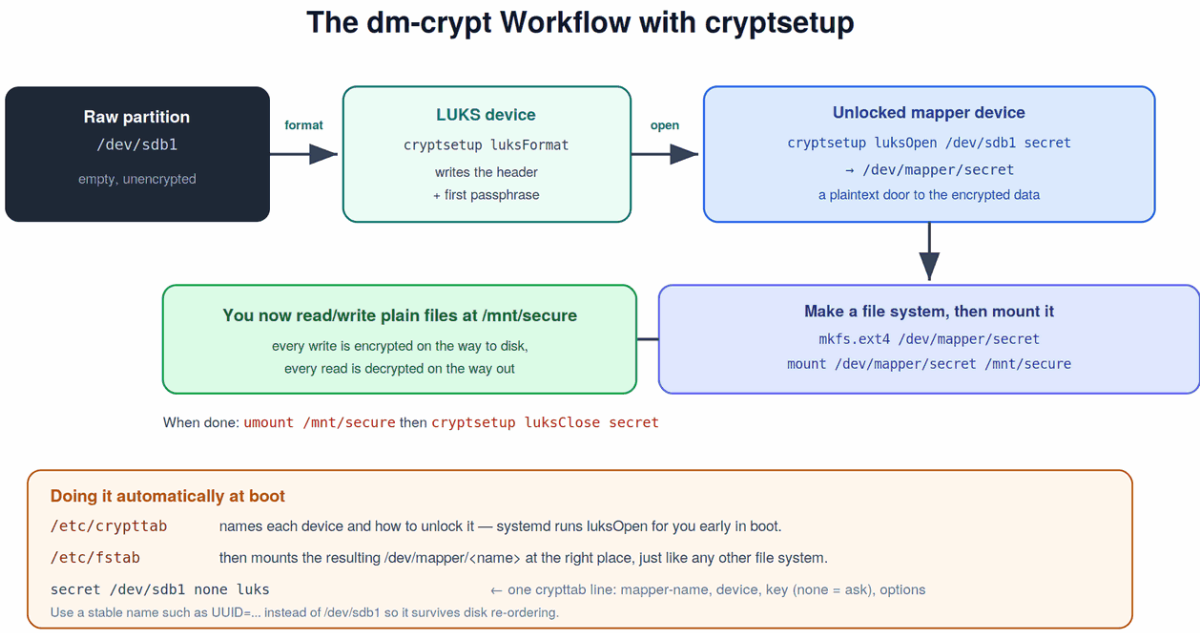


Figure 2 — Format → open → make a file system → mount. At boot, /etc/crypttab and /etc/fstab do the open and mount for you.

3. LUKS1: Encrypting Block Devices with cryptsetup

You could hand dm-crypt the cipher and key yourself ("plain" mode, Section 4), but almost everyone uses **LUKS** — the Linux Unified Key Setup. LUKS adds a standard **header** at the front of the device that records the cipher settings and stores the encryption key in a safe, structured way. That header is what makes LUKS convenient and robust.

3.1 The header, key slots and the master key

The data is really encrypted with one randomly generated **master key** (also called the volume key). You never see it. Instead, the header has up to eight **key slots**, and each slot holds a copy of the master key that has itself been locked with one of your passphrases (or a key file). To open the device, any single passphrase that fits any slot is enough — it unlocks that slot, which reveals the master key, which decrypts the data.

This design has three big consequences worth remembering: you can give up to eight different people (or methods) their own passphrase for the same device; adding, changing or removing a passphrase only rewrites one slot and never re-encrypts the actual data; and the header is precious — lose it and every passphrase becomes worthless, because the only copies of the master key lived inside it.

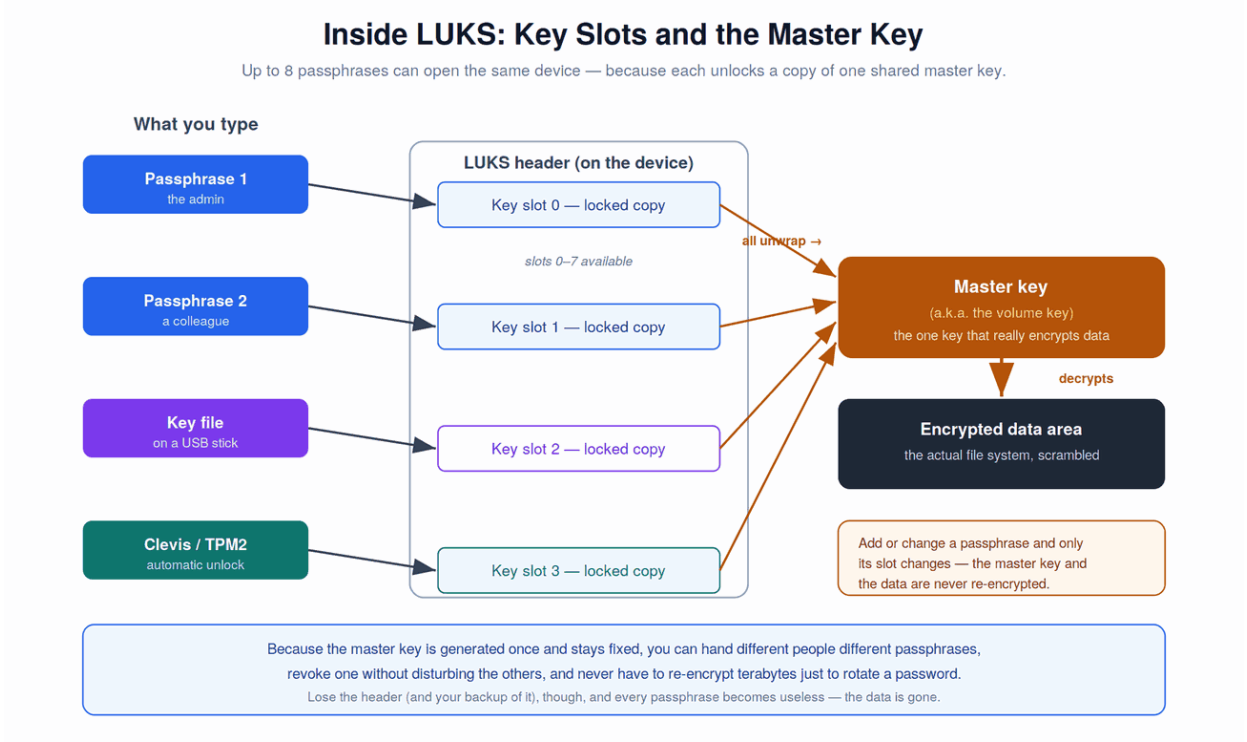


Figure 3 — Each passphrase or key file unlocks a key slot, and every slot reveals the same master key that encrypts the data.

3.2 The everyday workflow, command by command

Create the encrypted device, then open, format, mount, and later close it:

```
# 1. Turn a partition into a LUKS device (this ERASES it). Sets passphrase in slot
0.
$ sudo cryptsetup luksFormat /dev/sdb1

# 2. Open it -> creates the decrypted door /dev/mapper/secret
$ sudo cryptsetup luksOpen /dev/sdb1 secret        # "open" also works: cryptsetup
open

# 3. Put a file system on the OPENED device, not on /dev/sdb1
$ sudo mkfs.ext4 /dev/mapper/secret

# 4. Mount and use it like any disk
$ sudo mount /dev/mapper/secret /mnt/secure

# 5. When finished: unmount, then lock it again
$ sudo umount /mnt/secure
$ sudo cryptsetup luksClose secret                 # or: cryptsetup close secret
```

Watch the two device names

You **format and open** the real partition (`/dev/sdb1`), but you **mkfs and mount** the opened mapper device (`/dev/mapper/secret`). Running mkfs on the raw partition would destroy the LUKS header.

3.3 Managing passphrases and key files

Because slots are independent, key management never touches your data:

```
$ sudo cryptsetup luksAddKey /dev/sdb1               # add a passphrase to a free
slot
$ sudo cryptsetup luksAddKey /dev/sdb1 /root/keyfile # add a key FILE instead of a
passphrase
$ sudo cryptsetup luksChangeKey /dev/sdb1            # replace the passphrase in a
slot
$ sudo cryptsetup luksRemoveKey /dev/sdb1           # remove a passphrase by
supplying it
$ sudo cryptsetup luksKillSlot /dev/sdb1 2          # wipe slot 2 by number (needs
another key)

# Open with a key file instead of typing:
$ sudo cryptsetup luksOpen /dev/sdb1 secret --key-file /root/keyfile
```

3.4 Inspecting and protecting the header

Look at what a device is, dump its header (cipher, slots in use), and — most importantly — **back up the header**. If the first sectors of the disk are ever damaged, that backup is the only way to recover your data.

```
$ sudo cryptsetup isLuks /dev/sdb1 && echo "yes, LUKS"   # test quietly (exit
status)
$ sudo cryptsetup luksDump /dev/sdb1                     # show header: cipher, key
slots
$ sudo cryptsetup status secret                         # info about an open
mapping
$ cryptsetup benchmark                                  # measure cipher speed on
this CPU

# Header backup / restore -- keep the backup somewhere safe and OFF this disk
$ sudo cryptsetup luksHeaderBackup /dev/sdb1 --header-backup-file sdb1.luks.img
$ sudo cryptsetup luksHeaderRestore /dev/sdb1 --header-backup-file sdb1.luks.img
```

A header backup is a full key

The header holds the (wrapped) master key, so anyone with the header backup *and* a valid passphrase can open the device. Store the backup as carefully as the passphrase itself.

3.5 Unlocking automatically at boot: /etc/crypttab

To have a device opened during boot, describe it in `/etc/crypttab`. Each line has four fields: the mapper name to create, the source device, the key (or `none` to prompt for a passphrase), and options. Then `/etc/fstab` mounts the resulting `/dev/mapper/<name>` just like any other file system.

```
# /etc/crypttab :  <name>   <device>           <key>           <options>
secure    UUID=1234-abcd...   none            luks            # prompt at boot
backup    /dev/sdc1           /root/backup.key luks           # unlock from a key
file

# /etc/fstab : mount the opened mapper device
```

```
/dev/mapper/secure   /mnt/secure   ext4   defaults   0 2
```

Prefer a stable `UUID=` (from `blkid`) over `/dev/sdb1`, so the entry keeps working if the kernel numbers the disks differently on the next boot.

4. Plain dm-crypt (Awareness)

LUKS is the normal choice, but dm-crypt also has a **plain** mode with **no header at all**. There are no key slots and no metadata on the disk; instead you must supply the exact cipher, hash and key details every single time you open it, and the key is derived directly from your passphrase.

- **Upside:** the disk looks like pure random data with nothing identifying it as encrypted (plausible deniability), and there is no header to corrupt or back up.
- **Downside:** no key slots (only one passphrase, no easy rotation), and if you forget any parameter — cipher, hash, offset, key size — the data is unrecoverable. A single typo opens "successfully" but yields garbage.

```
# Plain mode: every parameter must match exactly, every time
$ sudo cryptsetup --type plain --cipher aes-xts-plain64 --key-size 256 \
    open /dev/sdb1 plainsecret
```

For the exam, just be **aware** that plain mode exists, that it stores no header, and that this makes it both deniable and unforgiving.

5. LUKS2 Features (Awareness)

LUKS2 is the newer on-disk format (the default in current distributions) and is backward-friendly: `cryptsetup` drives both, and LUKS1 is still fine to know for the exam. You should be **aware** of what LUKS2 adds:

- **A JSON metadata header** that is more flexible than LUKS1's fixed layout, kept in two copies on disk so a damaged header can be self-repaired.
- **Argon2 as the default key-derivation function** (instead of PBKDF2), which is memory-hard and far more resistant to brute-force cracking on GPUs.
- **Optional authenticated encryption (integrity)** by combining with `dm-integrity`, so tampering with the ciphertext is detected, not silently decrypted.
- **Tokens** that let helpers such as Clevis attach metadata to a key slot — this is how automatic unlocking (Section 8) plugs in.
- **Online re-encryption** and resizing, so a device can be re-encrypted or grown while it is still in use.

```
# Choose the format explicitly (LUKS2 is usually already the default)
$ sudo cryptsetup luksFormat --type luks2 /dev/sdb1
$ sudo cryptsetup luksFormat --type luks1 /dev/sdb1     # force the older format
$ sudo cryptsetup convert --type luks2 /dev/sdb1        # convert an existing LUKS1
device
```

6. eCryptfs: Encrypting File Systems and Home Directories

eCryptfs is a **stacked cryptographic file system**: it does not need its own partition, it layers on top of a normal directory. You have a **lower** directory that stores the encrypted files and an **upper** mount point that shows the decrypted view. Each file is encrypted individually with its own metadata, so files remain protected even if copied elsewhere.

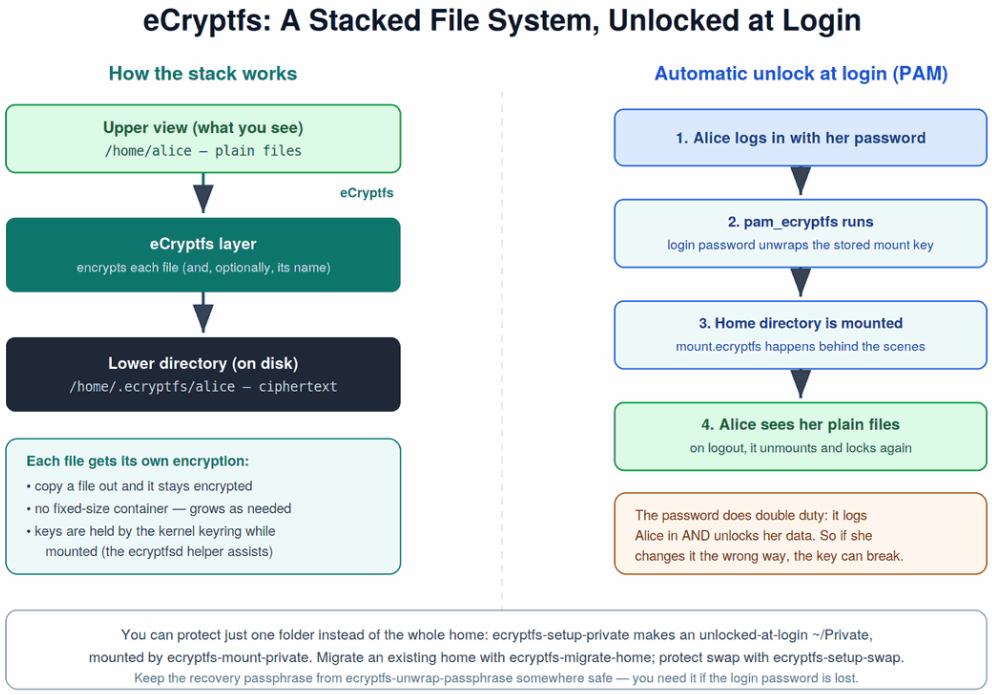


Figure 4 — eCryptfs stacks a decrypted view over an encrypted lower directory, and PAM can mount a user's home automatically at login.

6.1 Mounting it by hand

The lowest-level way is a normal `mount` of type `ecryptfs`, where the same directory is both source and target. `mount.ecryptfs` asks interactively for the cipher, key size and options; `umount.ecryptfs` unmounts it and evicts the keys.

```
# Mount an encrypted view of ~/secret on top of itself
$ sudo mount -t ecryptfs ~/secret ~/secret
  # prompts for passphrase, cipher (aes), key bytes, plaintext-passthrough,
  # and whether to encrypt file names

$ sudo umount.ecryptfs ~/secret                # unmount and remove keys

$ ecryptfs-stat ~/secret/somefile              # show a file's eCryptfs metadata
```

6.2 The helper commands

You almost never do the above by hand. A family of `ecryptfs-*` helpers set things up for you, and a small daemon, `ecryptfsd`, helps the kernel module talk to user space (for example to prompt for a passphrase). The most useful helpers:

Command	What it does
`ecryptfs-setup-private`	Create an encrypted ~/Private that unlocks at login.
`ecryptfs-mount-private`	Mount that ~/Private (ecryptfs-umount-private unmounts it).
`ecryptfs-migrate-home`	Convert an existing plain home directory to eCryptfs.
`ecryptfs-setup-swap`	Encrypt the swap space (so secrets paged out stay safe).
`ecryptfs-add-passphrase`	Insert a passphrase (mount key) into the kernel keyring.
`ecryptfs-unwrap-passphrase`	Reveal the recovery mount passphrase — save it somewhere safe.
`ecryptfs-stat`	Show the eCryptfs metadata of a single file.

6.3 Unlocking a home directory at login: pam_ecryptfs

The headline feature is encrypting a whole home directory that unlocks automatically when the user logs in. This is done through PAM (Pluggable Authentication Modules) with the `pam_ecryptfs` module. At login your password does double duty: it authenticates you and it unwraps the stored mount key, so `pam_ecryptfs` can mount your home before your session starts. At logout it unmounts and locks again.

```
# Debian/Ubuntu: install and enable, then set up a user
$ sudo apt install ecryptfs-utils
$ sudo ecryptfs-migrate-home -u alice        # encrypt alice's existing home

# pam_ecryptfs is added to the PAM stack (auth + session), e.g. in
# /etc/pam.d/common-auth   and   /etc/pam.d/common-session:
auth     required   pam_ecryptfs.so unwrap
session  optional   pam_ecryptfs.so unwrap
```

The login password IS part of the key

Because the login password unwraps the mount key, changing it the wrong way (for example, `root` resetting it rather than the user changing it) can leave the home directory unrecoverable. Keep the recovery passphrase from `ecryptfs-unwrap-passphrase` in a safe place.

7. cryptmount (Awareness)

`cryptmount` is an alternative front-end for encrypted file systems (dm-crypt/LUKS and, historically, loopback setups). Its selling point is that, once an administrator has configured a target, an **ordinary user** can mount and unmount it without needing full root each time. Targets are described in its own configuration file, `/etc/cryptmount/cmtab`, which names the device, the mapper name, the mount point, the file system and where the key lives.

```
# /etc/cryptmount/cmtab : one named target
secure {
    dev=/dev/sdb1
    dir=/mnt/secure
    fstype=ext4
    keyformat=luks
}
```

```
$ cryptmount secure          # unlock + mount the target "secure"
$ cryptmount -u secure       # unmount + lock it again
$ cryptmount -l              # list configured targets
```

For the exam, be **aware** that `cryptmount` exists, that it uses `/etc/cryptmount/cmtab`, and that it lets non-root users manage pre-defined encrypted mounts — a different convenience model from `cryptsetup` plus `/etc/crypttab`.

8. Automatic Unlocking: Clevis, TPM2 and NBDE / Tang

Typing a passphrase at every boot is fine for a laptop but impossible for a rack of servers that must come back up on their own. **Clevis** is a framework that unlocks a LUKS device automatically by satisfying a **pin** — a policy describing where the unlock secret comes from. Clevis simply fills a spare LUKS key slot, so your normal passphrase in slot 0 always remains as a fallback.

8.1 The pins

- `tpm2` — bind the unlock secret to this machine's **TPM2** chip. The key is sealed in hardware and released only if the machine boots in a known-good state. Ideal for a laptop: it unlocks itself, but a disk pulled out of the machine stays locked.
- `tang` — bind to a **Tang** server on the network. This is **NBDE (Network-Bound Disk Encryption)**: the device unlocks by itself only while it can reach the Tang server. A stolen disk taken off the network cannot unlock. Crucially, Tang stores no client keys and needs no authentication — it takes part in a small key-agreement exchange (McCallum–Relyea) so the secret is reconstructed without ever being stored on the server.
- `sss` — **Shamir Secret Sharing**: combine several pins and require a threshold, e.g. "the TPM plus at least one of two Tang servers." This adds redundancy so a single offline server does not lock you out.

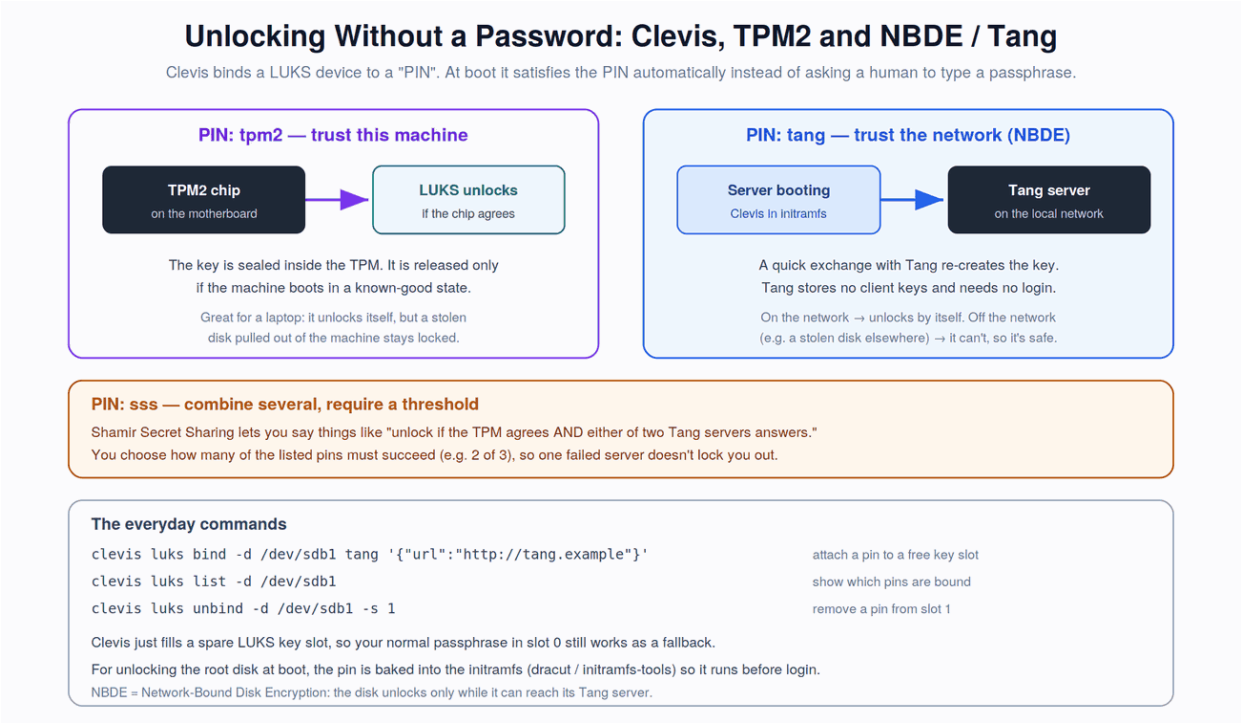


Figure 5 — Clevis pins: seal the key to the TPM2 (trust the machine), bind it to a Tang server (trust the network / NBDE), or require a threshold with SSS.

8.2 The commands

```
# Bind a device to a Tang server (fills a spare key slot)
$ sudo clevis luks bind -d /dev/sdb1 tang '{"url":"http://tang.example.com"}'

# Bind to the local TPM2 chip instead
```

```
$ sudo clevis luks bind -d /dev/sdb1 tpm2 '{}'

$ sudo clevis luks list   -d /dev/sdb1        # show which pins are bound to which
slots
$ sudo clevis luks unbind -d /dev/sdb1 -s 1   # remove the pin in slot 1

# Manually unlock using whatever pin is bound (normally automatic at boot)
$ sudo clevis luks unlock -d /dev/sdb1 -n secret
```

For the root disk to unlock this way during boot, the Clevis pin is built into the **initramfs** (via `dracut` on RHEL-family systems or `initramfs-tools` on Debian), so it runs before the login prompt. The exam expects a **conceptual** understanding: Clevis + pins, the `tpm2` and `tang` pins, and that NBDE means the disk is bound to a network service.

9. Quick Reference

9.1 cryptsetup subcommands

Subcommand	Purpose
`luksFormat`	Initialise a device as LUKS (erases it, sets the first passphrase).
`luksOpen / open`	Unlock a device to /dev/mapper/<name>.
`luksClose / close`	Lock and remove the mapper device.
`luksAddKey`	Add a passphrase or key file to a free slot.
`luksChangeKey`	Replace the passphrase in a slot.
`luksRemoveKey / luksKillSlot`	Remove a passphrase (by value / by slot number).
`luksDump`	Print the header: cipher, key-derivation, slots in use.
`isLuks`	Test (quietly) whether a device is LUKS.
`status`	Show information about an open mapping.
`benchmark`	Measure cipher/hash speed on this CPU.
`luksHeaderBackup / luksHeaderRestore`	Save or restore the all-important LUKS header.
`convert`	Switch a device between LUKS1 and LUKS2.

9.2 Files, tools and terms

Name	What it is
`cryptsetup`	The main front-end for dm-crypt / LUKS.
`/etc/crypttab`	Devices to unlock at boot: name, device, key, options.
`cryptmount + /etc/cryptmount/cmtab`	Alternative tool letting users mount pre-defined encrypted targets.
`ecryptfsd`	User-space daemon assisting the eCryptfs kernel module.
`mount.ecryptfs / umount.ecryptfs`	Mount / unmount an eCryptfs directory.
`ecryptfs-* commands`	Helpers: setup-private, migrate-home, setup-swap, add/unwrap-passphrase, stat.
`pam_ecryptfs`	PAM module that mounts an encrypted home at login.
`Clevis / pins (tpm2, tang, sss)`	Automatic LUKS unlocking policies.
`Tang / NBDE`	Network service that unlocks disks bound to it, storing no keys.

Exam tips — the things people get wrong

- Format and open the raw partition, but mkfs and mount the `/dev/mapper/<name>` device — never the other way round.
- LUKS stores the real key as a **master key** wrapped in up to **eight key slots**; changing a passphrase never re-encrypts data.
- **Back up the LUKS header** — losing it loses everything, and a header backup is as sensitive as a passphrase.
- **Plain dm-crypt** has no header and no key slots; every open must specify the exact cipher parameters.

- **eCryptfs is per-file and stacked**; `pam_ecryptfs` unlocks a home directory at login using the login password.
- **Clevis fills a spare LUKS slot**; the `tang` pin is NBDE (network-bound), the `tpm2` pin binds to the machine, and Tang stores no keys.

331.4 DNS and Cryptography

Objective weight: 5

Overview

Ordinary DNS was designed in an earlier, more trusting era: answers travel in plain UDP packets with nothing to prove they are genuine. That means a well-placed attacker can forge a reply and send your browser to the wrong server — the classic cache-poisoning attack. DNSSEC fixes this by adding digital signatures to DNS data, so a resolver can check that an answer really came from the zone's owner and was not altered on the way.

A crucial point up front: DNSSEC adds **authenticity and integrity**, not **secrecy**. The answers are still readable by anyone; they are just signed so they cannot be forged. This chapter walks from plain DNS up through DNSSEC (keys, records, the chain of trust, denial of existence), then how to run BIND as both a signing authoritative server and a validating resolver, how to roll keys safely, how to protect server-to-server traffic with TSIG, and how DNS can now carry certificate information through CAA and DANE. It finishes with the newer privacy transports (DoT, DoH) and Multicast DNS, and a toolbox of `dig` and `delv` commands.

1. DNS Refresher: Zones and Resource Records

DNS is a large, distributed database shaped like a tree, read from the right: `www.example.org` means the `www` record, in the `example.org` zone, under the `.org` top-level domain, under the root. A **zone** is the slice of that tree one set of servers is responsible (authoritative) for, and its contents are **resource records (RRs)** — the individual entries like `A` (IPv4), `AAAA` (IPv6), `NS` (name servers), `SOA` (the zone's master record), `MX`, `CNAME`, `TXT` and so on.

Two server roles matter. An **authoritative** server holds the real data for its zones and answers for them. A **recursive** server (a resolver) holds no zones of its own; it performs the work of chasing a name from the root down to the authoritative server, caches the result, and hands it back to the client. DNSSEC touches both: authoritative servers *sign* their zones, and recursive servers *validate* those signatures.

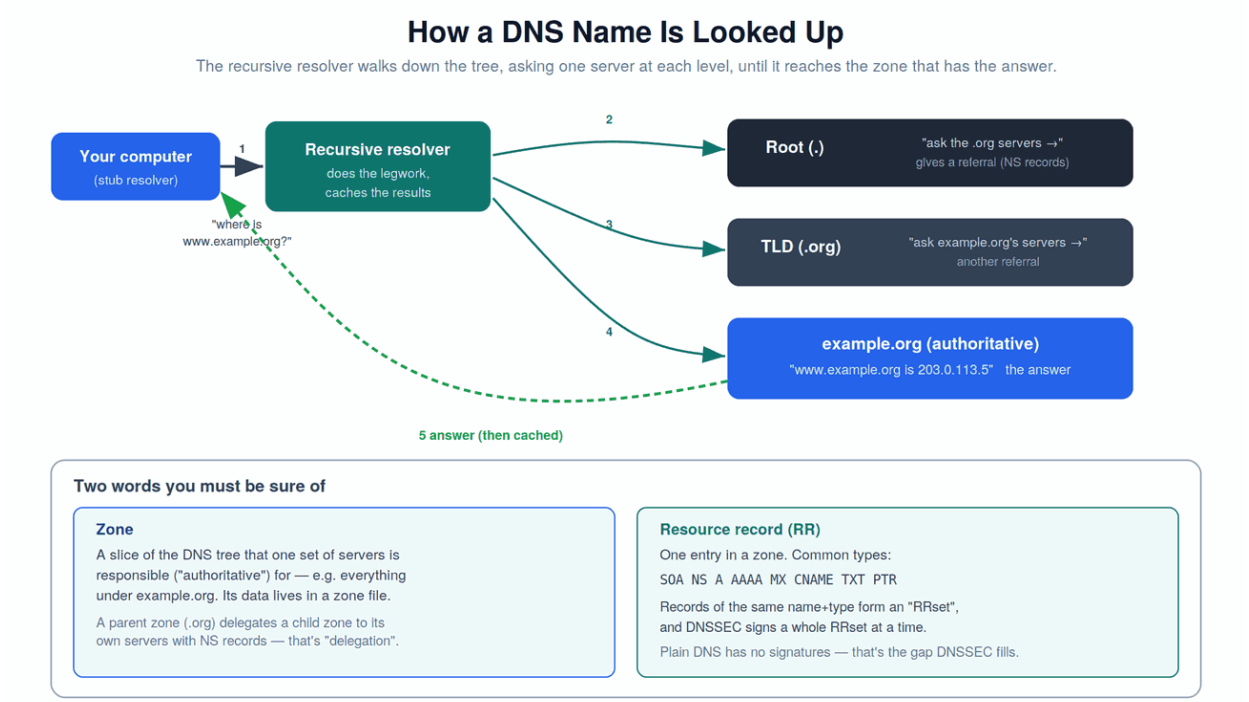


Figure 1 — A recursive resolver walks the tree from the root to the authoritative server; a zone is a slice of that tree and resource records are its entries.

2. The DNSSEC Records

DNSSEC introduces a handful of new record types. Everything else in the chapter is really just these records working together, so it pays to learn them first:

Record	What it holds / does
`DNSKEY`	The zone's public keys (its KSK and ZSK). Resolvers fetch this to check signatures.
`RRSIG`	A signature over one RRset, with the algorithm, key tag and validity dates.
`DS`	Delegation Signer: a hash of the child's KSK, published in the PARENT zone — the link in the chain.
`NSEC`	Points to the next existing name, to prove (signed) that a queried name does NOT exist.
`NSEC3`	Like NSEC but over hashed names, so the zone cannot be easily enumerated.
`NSEC3PARAM`	Tells resolvers the hash algorithm, salt and iterations used for NSEC3.

The signatures use public-key cryptography: the zone's **private** key signs, and the matching **public** key (published in `DNSKEY`) lets anyone verify. A signature covers a whole **RRset** (all records of the same name and type) and carries an expiry date, which is why signed zones must be periodically *re-signed*.

3. Two Keys per Zone: KSK and ZSK

A signed zone normally has **two** keys, and knowing the division of labour is core exam material. The **Zone Signing Key (ZSK)** signs all the ordinary records in the zone, producing the `RRSIG` records. The **Key Signing Key**

(KSK) has just one job: it signs the DNSKEY RRset (the set of public keys). Only the KSK is referenced by the parent — the parent's DS record is a hash of the KSK.

Why split them? Because the KSK is the one the parent vouches for, changing it means coordinating with the parent (your registrar) — slow and manual — so you keep it large and change it rarely. The ZSK signs everything else and can be rolled frequently and entirely on your own, since the parent never sees it. The KSK is flagged as a **Secure Entry Point (SEP)** so tools can tell the two apart.

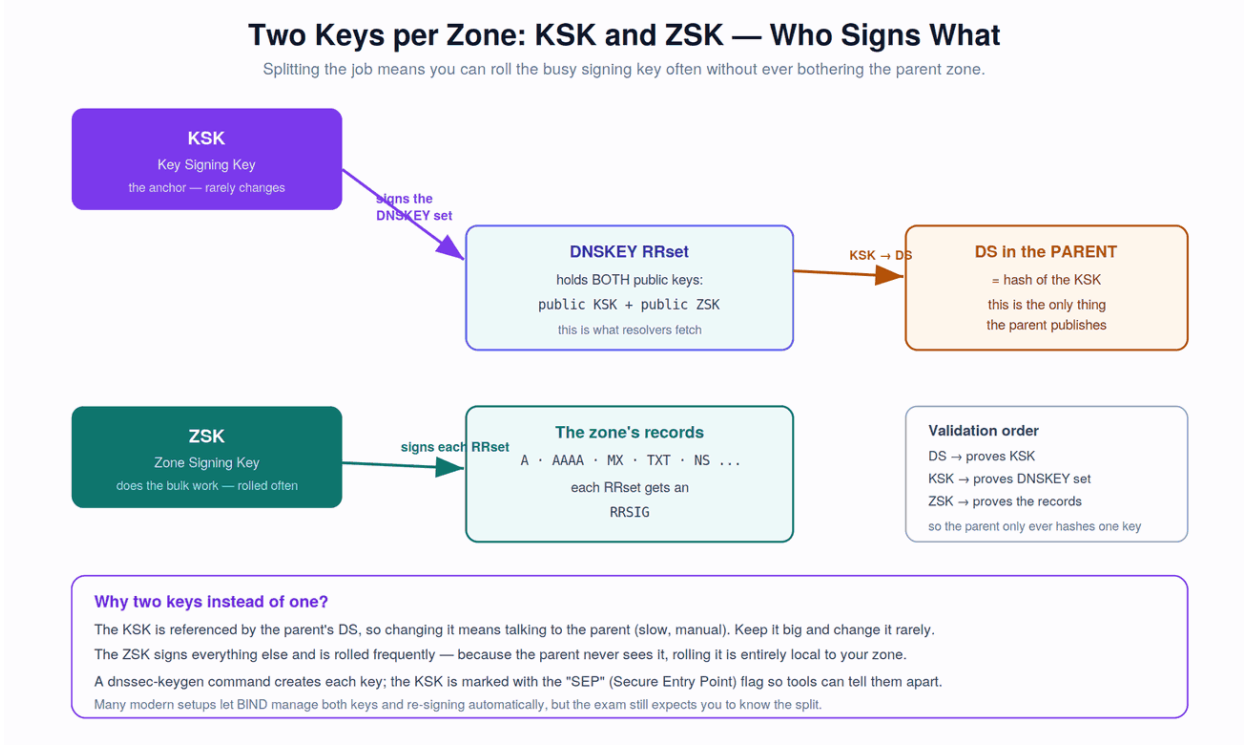


Figure 2 — The ZSK signs the zone's records; the KSK signs the DNSKEY set; and the parent's DS is a hash of the KSK.

4. The Chain of Trust

Validation works top-down. Every resolver ships with one key it trusts unconditionally — the **root** zone's KSK, the **trust anchor**. From there each parent vouches for its child: the parent holds a DS record that is a hash of the child's KSK. So the resolver hashes the child's KSK, compares it to the parent's DS, and if they match the child's keys are trusted; the child's KSK then validates its DNSKEY set, and the ZSK validates the actual records. Repeat down the tree and you have an unbroken chain from the root to any single record.

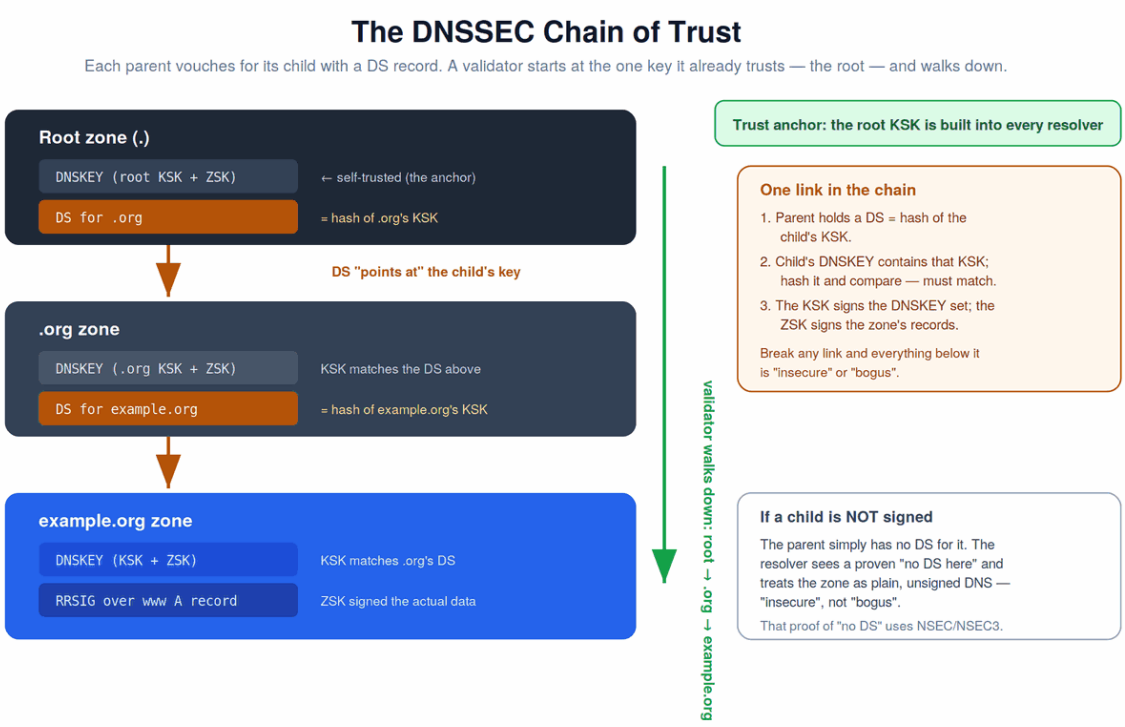


Figure 3 — Each parent's DS record authenticates the child's KSK, forming an unbroken chain from the root trust anchor down to the record you asked for.

Three validation results you should recognise

Secure — a full chain of valid signatures was found. **Insecure** — the zone is legitimately unsigned (the parent proves there is no DS), so DNSSEC steps aside. **Bogus** — signatures should be there but are missing, expired, or wrong; a validating resolver returns `SERVFAIL` rather than trust it.

5. Authenticated Denial: NSEC and NSEC3

A signature can only cover records that exist — so how do you prove, without lying, that a name does *not* exist? DNSSEC signs the gaps. With `NSEC`, the names in a zone are sorted into a ring and each record points to the next one; a signed "nothing exists between name A and name B" proves any name in that gap is absent. The weakness is that anyone can follow the pointers and list the whole zone — "zone walking". `NSEC3` solves that by pointing between **hashes** of the names instead of the names themselves, and `NSEC3PARAM` records the hash algorithm, salt and iteration count.

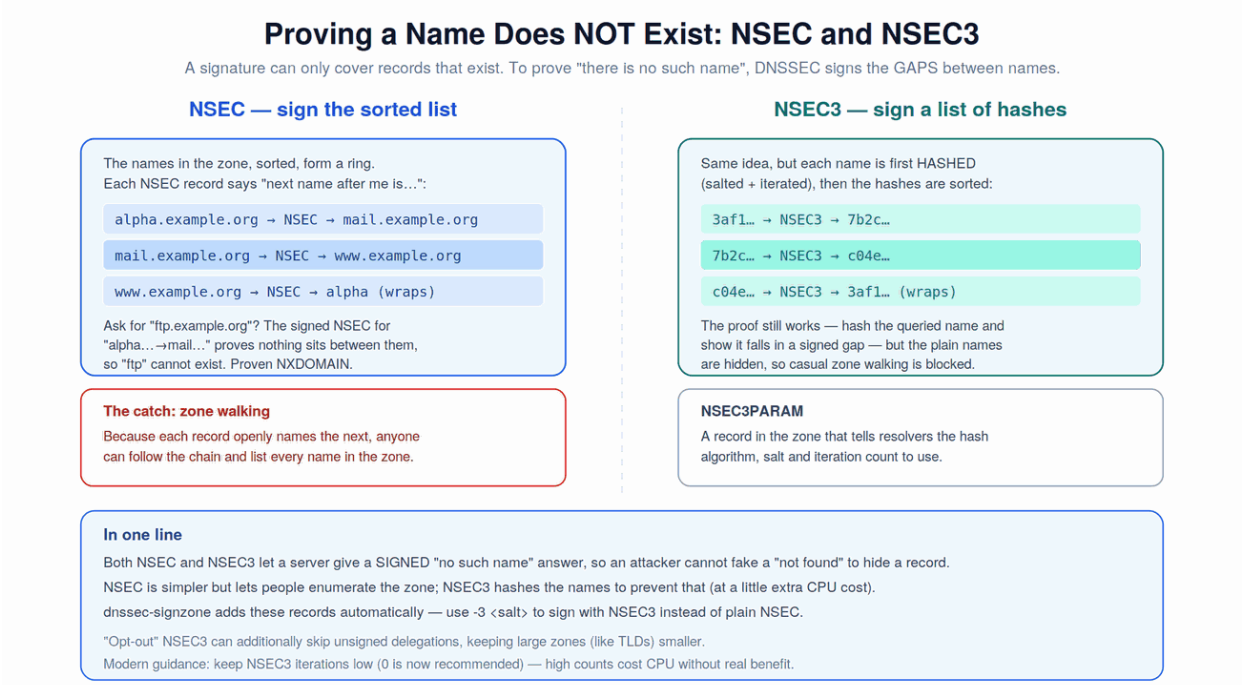


Figure 4 — NSEC signs the gaps between sorted names to prove non-existence; NSEC3 does the same over hashed names to stop zone enumeration.

6. Running BIND as a Signing Authoritative Server

Now the practical half. Signing a zone with BIND has three parts: generate the keys, sign the zone, and give your DS record to the parent. Modern BIND can automate most of this, but you should know the manual tools too.

6.1 Generate the keys with dnssec-keygen

```
# A Zone Signing Key (ZSK) for example.org
$ dnssec-keygen -a ECDSAP256SHA256 -n ZONE example.org

# A Key Signing Key (KSK): add -f KSK (sets the SEP flag)
$ dnssec-keygen -a ECDSAP256SHA256 -f KSK -n ZONE example.org

# Older RSA example (still valid):  -b sets the key size
$ dnssec-keygen -a RSASHA256 -b 2048        -n ZONE example.org   # ZSK
$ dnssec-keygen -a RSASHA256 -b 4096 -f KSK -n ZONE example.org   # KSK
```

Each run creates two files: a `.key` file (the public key, a `DNSKEY` record you $INCLUDE in the zone) and a `.private` file (the secret used for signing — protect it). The file names encode the algorithm number and the key tag, e.g. `Kexample.org.+013+12345.key`.

6.2 Sign the zone with dnssec-signzone

"Smart signing" (`-S`) finds the keys in the key directory for you and writes a signed copy of the zone plus the NSEC/NSEC3 records:

```
# Sign with NSEC (default). -o = zone origin, last arg = zone file
$ dnssec-signzone -S -o example.org db.example.org
  -> writes db.example.org.signed  (and dsset-example.org.)

# Sign with NSEC3 instead: -3 gives the salt (use - for none/empty)
$ dnssec-signzone -S -3 - -o example.org db.example.org

# Classic form, naming the keys explicitly:
$ dnssec-signzone -o example.org -k Kexample.org.+013+11111 \
    db.example.org Kexample.org.+013+22222
```

Then point the zone at the `.signed` file in `named.conf` and reload. But the tidier modern approach is to let BIND sign and maintain the zone itself:

```
// named.conf — let BIND sign and keep the zone signed automatically
zone "example.org" {
    type primary;                // "master" in older BIND
    file "db.example.org";
    key-directory "/etc/bind/keys";
    inline-signing yes;
    auto-dnssec maintain;        // BIND signs & re-signs using the keys it finds
};
```

With `inline-signing` BIND keeps your plain zone file untouched and maintains a separate signed version, re-signing before signatures expire. In BIND 9.16+ a single `dnssec-policy` statement can manage key generation and rollover for you — but the exam still expects the manual tools above.

6.3 Hand your DS to the parent with dnssec-dsfromkey

The chain only connects once the parent publishes your `DS`. Generate it from your KSK and give the result to your registrar (who forwards it to the parent zone):

```
$ dnssec-dsfromkey Kexample.org.+013+11111.key
example.org. IN DS 11111 13 2 3F2B...A9   # submit this to the registrar

# -2 selects SHA-256 explicitly; you can also read it from the dsset- file
$ dnssec-dsfromkey -2 Kexample.org.+013+11111.key
```

Control the running server with `rndc`: `rndc reload` re-reads zones, `rndc reconfig` re-reads `named.conf`, `rndc sign example.org` triggers signing, `rndc loadkeys example.org` picks up new keys, and `rndc dnssec -status example.org` (9.16+) shows where each key is in its lifecycle.

7. Key Rollover and Re-signing

Keys must be replaced from time to time, and the golden rule is: **publish the new key and wait a full TTL before anything depends on it, and keep the old one until its signatures have expired from every cache**. Yank a key too early and resolvers that still have the old data cached will see broken signatures and return `SERVFAIL`.

A **ZSK rollover** is entirely local (the parent never sees the ZSK), so the "pre-publish" method is used: add the new ZSK, wait, switch signing to it, wait, then remove the old one. A **KSK rollover** must involve the parent (the "double-DS" method): publish the new KSK, submit its DS to the parent, wait until the new DS is live everywhere and the old one has aged out of caches, then remove the old KSK.

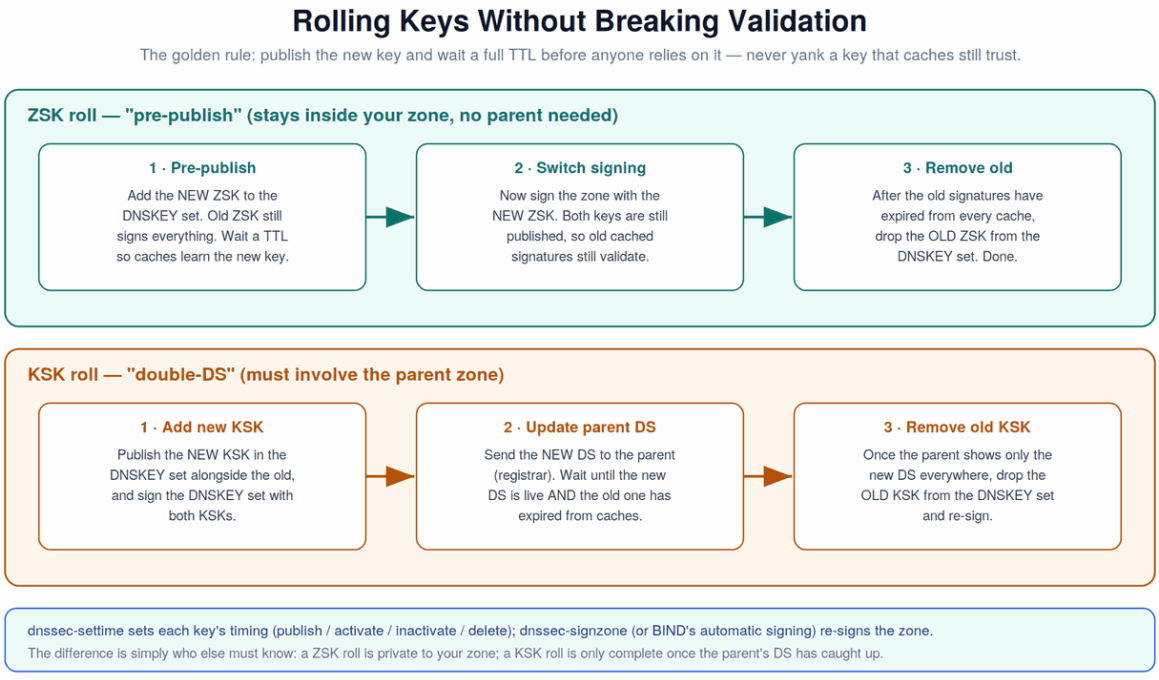


Figure 5 — A ZSK rolls with pre-publish (local only); a KSK rolls with double-DS and is only finished once the parent's DS has caught up.

The timing is driven by `dnssec-settime`, which stamps each key with when it should be published, activated (start signing), inactivated (stop signing) and deleted. BIND then acts on those dates automatically:

```
# Schedule the old ZSK to stop signing now and be deleted in 30 days
$ dnssec-settime -I now -D +30d Kexample.org.+013+22222

# Show a key's current timing metadata
$ dnssec-settime -p all Kexample.org.+013+22222

# Re-sign on demand (or let auto-dnssec / dnssec-policy do it)
$ rndc sign example.org
```

Watch signature expiry, not just keys

Even without a key change, every `RRSIG` has an expiry date. A zone that stops being re-signed will go **bogus** the moment its signatures lapse, even though nothing was "changed". Automatic signing exists precisely to prevent this.

8. Running BIND as a Validating Resolver

The other half of DNSSEC is checking signatures on behalf of clients. In BIND this is essentially one line — validation is on by default in current versions, using the built-in root trust anchor:

```
// named.conf on a recursive resolver
options {
    recursion yes;
    dnssec-validation auto;      // validate using the managed root trust anchor
    // dnssec-validation yes;    // validate using anchors you configure yourself
};
```

When a validating resolver returns a fully validated answer, it sets the **AD** (Authenticated Data) flag in the reply. If validation should succeed but fails — expired or missing signatures, a broken chain — the resolver refuses to release the data and returns `SERVFAIL` instead. Test it from the command line:

```
# Ask a validating resolver and look for the "ad" flag in the header
$ dig @localhost example.org A +dnssec
   ;; flags: qr rd ra ad; ...        <- "ad" means validated
   example.org.  IN A     203.0.113.5
   example.org.  IN RRSIG A ...      <- the signature, shown by +dnssec

# delv does the validation itself and tells you the verdict plainly
$ delv example.org A
   ; fully validated
   example.org.  IN A 203.0.113.5

# A deliberately broken zone should come back bogus / SERVFAIL
$ delv @localhost dnssec-failed.example A
```

dig vs delv

`dig` shows you the raw records and flags (including `ad`) but trusts the resolver's verdict. `delv` performs the DNSSEC validation itself and reports "fully validated", "insecure", or "bogus", which makes it the better tool for diagnosing a chain problem.

9. Securing Server Traffic with TSIG

DNSSEC signs zone **data**. A different problem is authenticating a single **message** between two of your own servers — for example, making sure only an authorised secondary can pull a zone, or that an `rndc` command really came from you. That is what **TSIG (Transaction SIGnature)** does, using a **shared secret** and an HMAC rather than public keys.

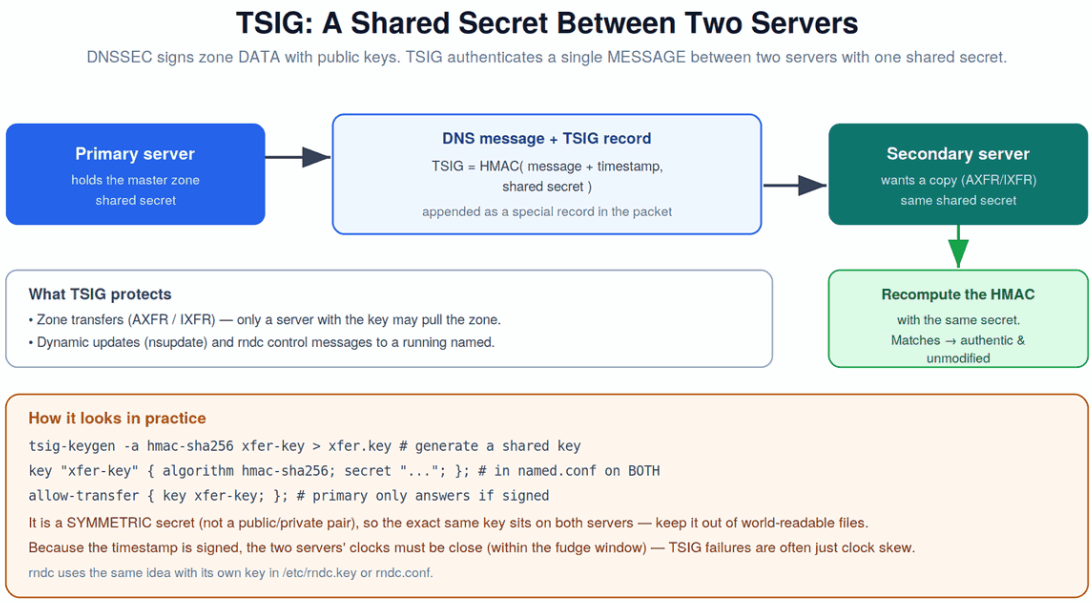


Figure 6 — TSIG appends an HMAC of the message (plus a timestamp) computed with a secret shared by both servers; the receiver recomputes it to confirm authenticity.

Generate a key, put the identical key on both servers, then require it where it matters:

```
# Generate a shared key (both servers get the SAME secret)
$ tsig-keygen -a hmac-sha256 xfer-key
key "xfer-key" {
    algorithm hmac-sha256;
    secret "K8s...==";
};

// On the PRIMARY: only allow transfers signed with the key
zone "example.org" {
    type primary;
    allow-transfer { key xfer-key; };
};

// On the SECONDARY: sign requests to the primary with the key
server 192.0.2.1 { keys { xfer-key; }; };
```

Because the secret is symmetric, guard the file (it is as good as the zone). And because a timestamp is part of the HMAC, the two servers' clocks must stay close — a lot of "TSIG failed" errors are really clock skew. The `rndc` control channel uses the same mechanism with its own key in `/etc/rndc.key`.

10. Publishing Certificate Information: CAA and DANE

Once a zone is DNSSEC-signed, DNS becomes a trustworthy place to publish facts about your certificates. Two record types do this. **CAA** says *which CA is allowed to issue* certificates for your domain, and CAs are required to check it before issuing. **DANE**, via TLSA records, says *which certificate a client should expect*, so trust can come from DNSSEC instead of (or as well as) the public CA system.

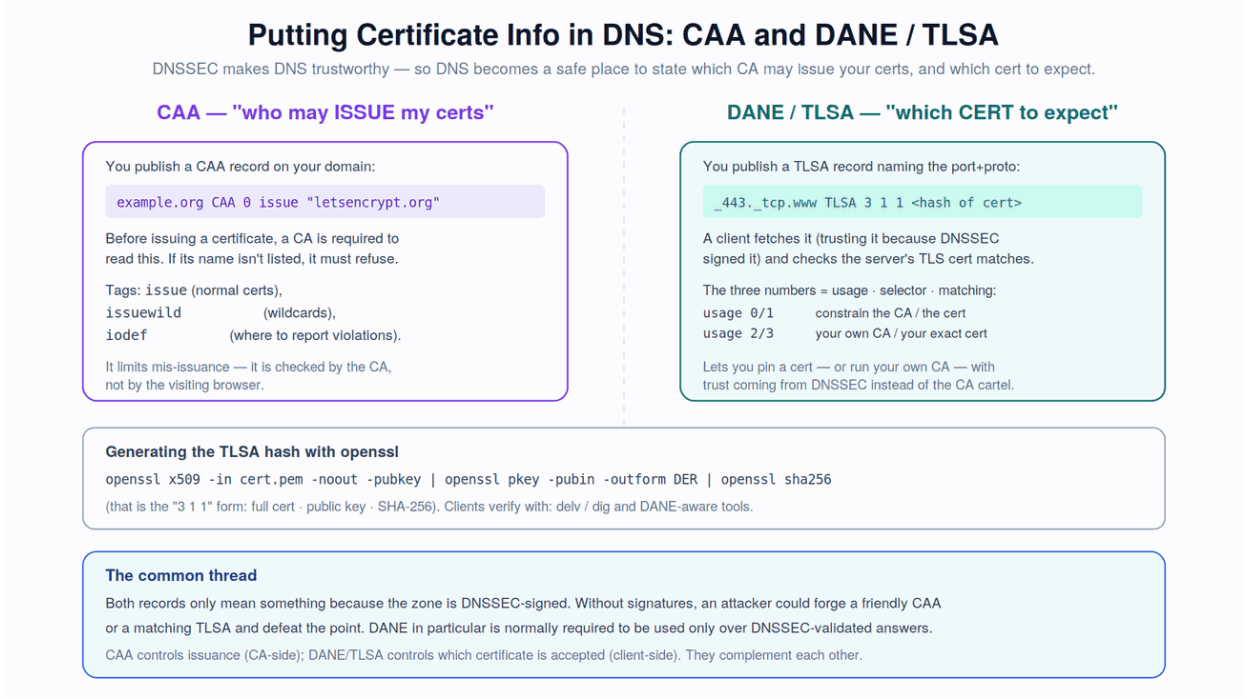


Figure 7 — CAA constrains who may issue your certificates (checked by the CA); DANE/TLSA pins which certificate is accepted (checked by the client). Both rely on DNSSEC.

10.1 CAA records

```
; only Let's Encrypt may issue certs; report violations by email
example.org.  IN CAA 0 issue "letsencrypt.org"
example.org.  IN CAA 0 issuewild ";"        ; forbid wildcard certs
example.org.  IN CAA 0 iodef "mailto:security@example.org"
```

10.2 DANE / TLSA records

A TLSA record lives under a `_port._proto` name and carries three numbers — *usage*, *selector*, *matching type* — followed by the certificate data (often a SHA-256 hash). Usage values 0 and 1 constrain a public-CA certificate; 2 and 3 let you assert your own CA or your own exact certificate.

```
; _port._proto.host : usage(3)=exact cert, selector(1)=pubkey, match(1)=SHA-256
_443._tcp.www.example.org.  IN TLSA 3 1 1 5C1B...F0

# Generate that "3 1 1" hash from the certificate with openssl:
$ openssl x509 -in cert.pem -noout -pubkey \
    | openssl pkey -pubin -outform DER \
    | openssl sha256
```

```
# Verify what is published (over a validated answer):
$ delv _443._tcp.www.example.org TLSA
$ dig  _443._tcp.www.example.org TLSA +dnssec
```

Both records are only meaningful because the zone is signed — without DNSSEC an attacker could forge a friendly CAA or a matching TLSA. CAA governs *issuance* (CA-side); DANE governs which certificate is *accepted* (client-side); together they close both ends.

11. Awareness: Encrypted Transports and Multicast DNS

DNSSEC protects the **integrity** of answers but leaves them readable. Two newer transports add **privacy** by encrypting the query itself between the client and its resolver: **DNS over TLS (DoT)**, which wraps DNS in TLS on port 853, and **DNS over HTTPS (DoH)**, which sends DNS inside ordinary HTTPS on port 443 so it blends in with web traffic. They are complementary to DNSSEC, not a replacement: DoT/DoH hide the conversation, DNSSEC proves the answer. Be **aware** of both and their ports for the exam.

Also be aware of **Multicast DNS (mDNS)**, defined in RFC 6762 and using the `.local` domain on UDP port `5353`. It lets devices on a small local network resolve each other's names with no DNS server at all — each host answers for its own name over multicast. It is the basis of zero-configuration networking (Apple Bonjour, Avahi on Linux) and is meant strictly for the local link, not the global DNS.

12. Command Toolbox

12.1 Everyday dig and delv

Command	What it shows
`dig example.org DNSKEY`	The zone's public keys (KSK + ZSK).
`dig example.org A +dnssec`	The answer plus its RRSIG; look for the ad flag.
`dig example.org DS`	The DS record held in the parent zone.
`dig +multi ...`	Pretty, multi-line record output.
`delv example.org A`	Validates itself: "fully validated", insecure or bogus.
`delv +rtrace / +vtrace`	Trace the resolution / validation step by step.

12.2 The DNSSEC and BIND utilities

Utility	Purpose
`named.conf`	BIND's configuration: zones, options, keys, dnssec-validation.
`dnssec-keygen`	Generate ZSK / KSK key pairs.
`dnssec-signzone`	Sign a zone file, adding RRSIG and NSEC/NSEC3.
`dnssec-settime`	Set a key's publish/activate/inactivate/delete times for rollover.
`dnssec-dsfromkey`	Produce the DS record to give to the parent.
`rndc`	Control a running named: reload, reconfig, sign, loadkeys, dnssec -status.
`tsig-keygen`	Generate a TSIG shared-secret key.
`dig / delv`	Query DNS / query with DNSSEC validation.

Utility	Purpose
`openssl`	Build TLSA hashes and inspect certificates for DANE.

Exam tips — the things people get wrong

- **DNSSEC gives authenticity and integrity, not secrecy** — answers are signed, not encrypted (that is DoT/DoH).
- **The DS lives in the PARENT** and is a hash of the child's KSK; the KSK signs the DNSKEY set, the ZSK signs the data.
- **Roll a ZSK locally (pre-publish); a KSK roll needs the parent (double-DS)** — and never remove a key before its cached signatures expire.
- **NSEC allows zone walking; NSEC3 hashes names to prevent it**; `NSEC3PARAM` carries the salt and iterations.
- A validated answer sets the `ad` flag; a broken chain returns `SERVFAIL`. Use `delv` to see the actual verdict.
- **TSIG is a symmetric shared secret** for transfers and `rndc` — clock-sensitive; CAA limits issuance, DANE/TLSA pins the certificate, and both need DNSSEC.

Topic 332 — Host Security

332.1 Host Hardening

Objective weight: 5

Hardening is the craft of reducing how much an attacker can do to a machine. There is no single switch for it; instead you apply many small controls, each closing off one avenue, so that a failure of any one does not hand over the whole system. This is the idea of defence in depth, and it starts the instant the machine receives power — long before your login prompt appears — and reaches all the way down to the individual system calls a process is allowed to make and how the CPU speculates about the future.

This chapter follows the layers from the outside in. We begin at the firmware and boot loader, move through cutting the attack surface, then work down into per-process confinement (capabilities, systemd sandboxing, chroot), the kernel's memory protections, device control with USBGuard, SSH certificate authorities, and finally the CPU-level Meltdown and Spectre mitigations, closing with an awareness of polkit and of virtualization and containers.

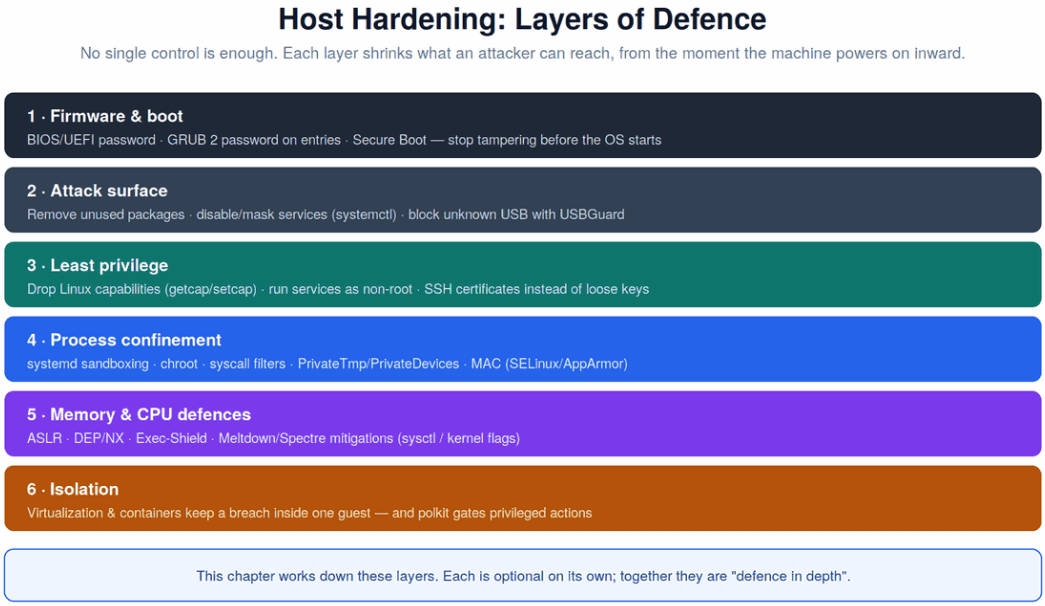


Figure 332.1a — The layers of host hardening, from firmware to CPU-level defences. Each section below tackles one band.

1. Firmware and boot loader (GRUB 2) security

Everything else in this chapter assumes the operating system you configured is the operating system that actually booted. If someone with physical access can change the boot process, none of your later controls matter — they simply boot something else. So the first layer is protecting the boot chain itself.

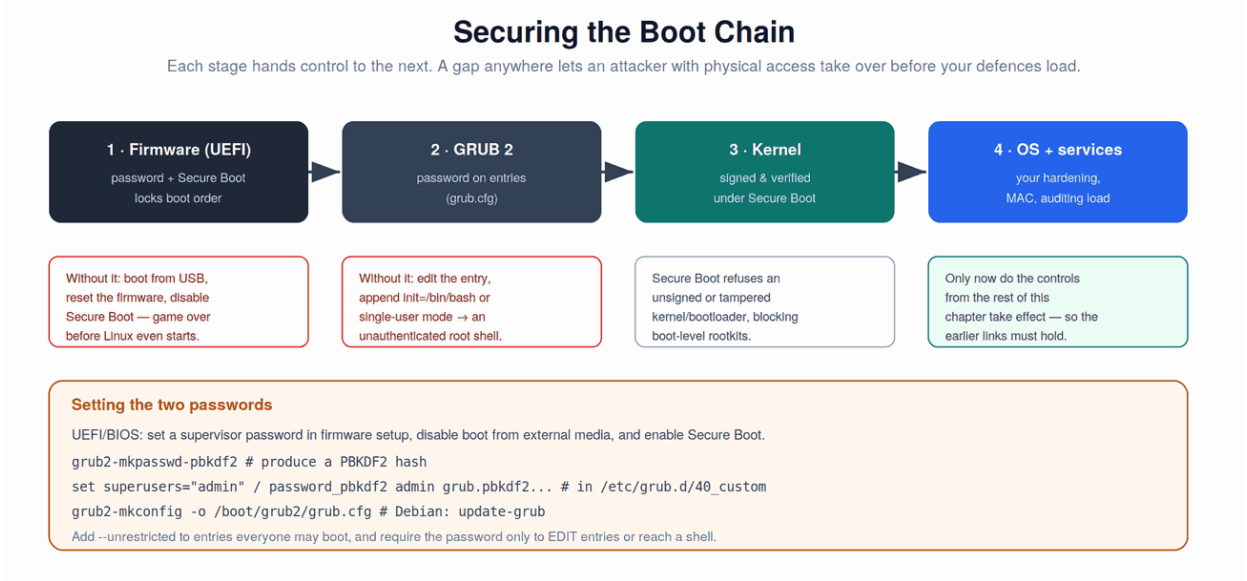


Figure 332.1b — The boot chain: firmware hands to GRUB, GRUB to the kernel, the kernel to your OS. A weak link lets an attacker take over before your defences load.

BIOS/UEFI and Secure Boot

In the firmware setup, set a supervisor (admin) password so an attacker cannot change settings, disable booting from USB and other removable media, and lock the boot order. Where the hardware supports it, enable **Secure Boot**, which makes the firmware refuse to run a boot loader or kernel that is not signed by a trusted key. That stops an entire class of boot-level rootkits (bootkits) that would otherwise load before Linux and hide from every tool you run later.

Why the GRUB password matters

GRUB 2 lets a user at the console edit a boot entry interactively by pressing `e`. That is enormously convenient — and enormously dangerous. By appending `init=/bin/bash` (or booting to single-user/rescue mode) an attacker gets a root shell with **no password prompt at all**, because `init` — normally systemd — is replaced by a shell that runs as root. From there they can reset the real root password. Password-protecting GRUB closes this door.

Setting a GRUB password

Generate a hashed password (never store it in the clear), declare a superuser, and attach the hash. Then regenerate `grub.cfg` so the change takes effect.

```
# 1. Produce a PBKDF2 hash of your chosen password
$ grub2-mkpasswd-pbkdf2
Enter password:
PBKDF2 hash of your password is grub.pbkdf2.sha512.10000.AB12...CD34

# 2. In /etc/grub.d/40_custom (or /etc/grub.d/00_header on some distros):
set superusers="admin"
password_pbkdf2 admin grub.pbkdf2.sha512.10000.AB12...CD34

# 3. Regenerate the config that GRUB actually reads
$ grub2-mkconfig -o /boot/grub2/grub.cfg      # Debian/Ubuntu: update-grub
```

```
# EFI systems may use /boot/efi/EFI/<distro>/grub.cfg
```

Let users boot, but not tamper

By default, setting `superusers` locks every menu entry so even normal booting needs the password. To allow unattended reboots while still protecting entry editing, mark the everyday entries `--unrestricted` (anyone may boot them, but only the superuser may edit them or reach a GRUB command line). This is usually what you want on a server.

2. Disable unused software and services

Every package installed and every service running is code that can contain a vulnerability and a port that can be probed. The cheapest security win available is simply to remove and switch off what you do not use — you cannot be attacked through a service that is not there.

Find what is running and listening

```
$ systemctl list-unit-files --state=enabled     # units that start at boot
$ systemctl list-units --type=service --state=running
$ ss -tulpn                                      # listening TCP/UDP + the process
   LISTEN 0 128 0.0.0.0:22 ... users:(("sshd",pid=812,fd=3))
$ systemctl list-sockets                         # socket-activated services
```

For each listener ask: do I need this? A database bound to `0.0.0.0` that only local apps use should listen on `127.0.0.1` instead; a print or discovery service you never use should be gone entirely.

Disable vs mask

`systemctl disable` stops a unit starting at boot but a dependency (or an admin) can still start it. `systemctl mask` is stronger: it links the unit to `/dev/null` so it **cannot be started at all**, by anything, until unmasked. Use `mask` for services you are certain must never run.

```
$ systemctl disable --now avahi-daemon.service   # stop now + never auto-start
$ systemctl mask     telnet.socket               # hard-off: unstartable
$ systemctl unmask   telnet.socket               # reverse it

# Remove the package entirely where you can
$ dnf remove telnet-server        # or: apt purge telnet
```

3. Linux capabilities: dropping unnecessary privilege

Traditional Unix is all-or-nothing: you are either root (uid 0), able to do everything, or you are not. That is a blunt instrument — a web server that only needs to bind port 80 does not need the power to load kernel modules, yet as root it has it, so any bug in it is a total compromise. Linux **capabilities** break root's authority into around forty independent privileges that can be granted one at a time.

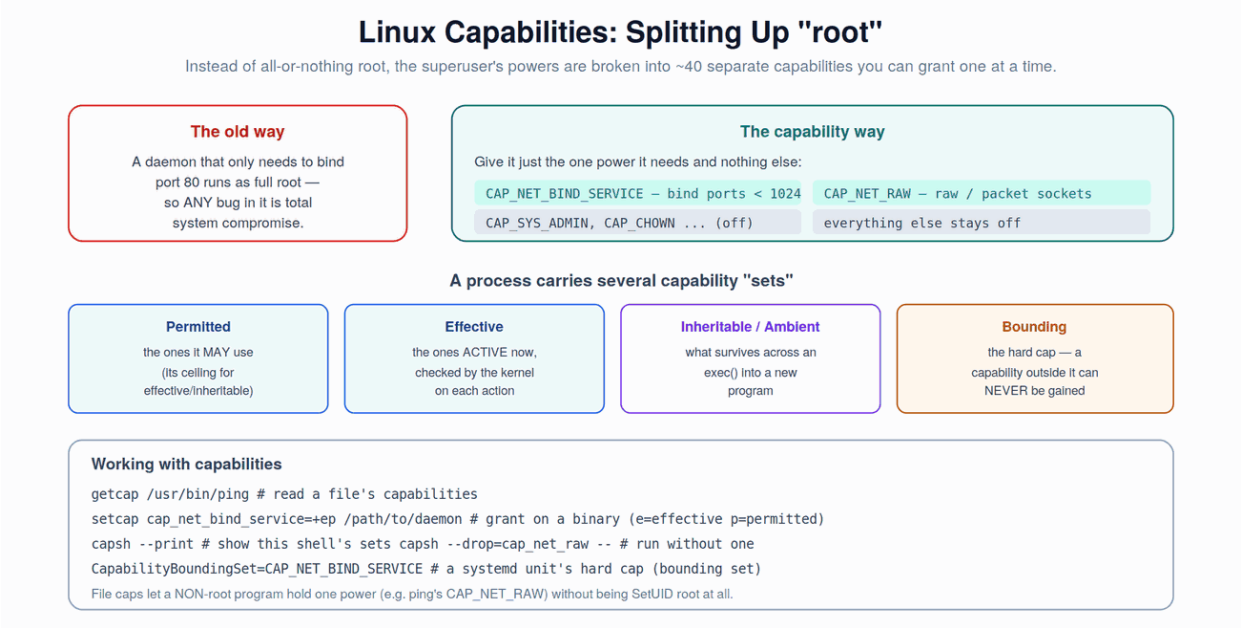


Figure 332.1c — Capabilities split root into individual powers; a process carries several "sets" that decide what it may use now and after exec().

The capability sets

A process carries several sets. The **Permitted** set is the ceiling of what it may hold; the **Effective** set is what the kernel actually checks on each privileged action; the **Inheritable** and **Ambient** sets control what survives an `exec()` into another program; and the **Bounding** set is a hard ceiling — a capability removed from it can never be regained, even via a SetUID-root binary. Files can also carry capabilities, so an ordinary, non-SetUID program can hold exactly one power.

Reading and setting capabilities

```
# Read the capabilities on a file (ping often carries cap_net_raw)
$ getcap /usr/bin/ping
/usr/bin/ping cap_net_raw=ep

# Grant a capability on a binary: e=effective, p=permitted, i=inheritable
$ setcap cap_net_bind_service=+ep /usr/local/bin/mydaemon
$ getcap /usr/local/bin/mydaemon
$ setcap -r /usr/local/bin/mydaemon        # remove all file caps

# Explore capabilities in the current shell with capsh
$ capsh --print                            # show all sets
$ capsh --drop=cap_net_raw -- -c "ping -c1 host"   # run WITHOUT one, to test

# See a running process's capabilities
$ grep Cap /proc/<pid>/status              # then: capsh --decode=<hex>
```

The practical payoff: instead of running your daemon as root, give the binary just `cap_net_bind_service` and run it as an unprivileged user — it can bind port 80 and do nothing else dangerous. For a whole service, systemd's `CapabilityBoundingSet=` (next section) applies the bounding-set ceiling declaratively.

4. Confining processes with systemd unit directives

systemd can wrap any service in a strong sandbox using nothing more than directives in its unit file. There is no code to change and no kernel module to load — internally these settings drive the same primitives (capabilities, seccomp system-call filtering, and mount/user namespaces) that container runtimes use. Because they are declarative and auditable, they are one of the highest-value hardening tools available.

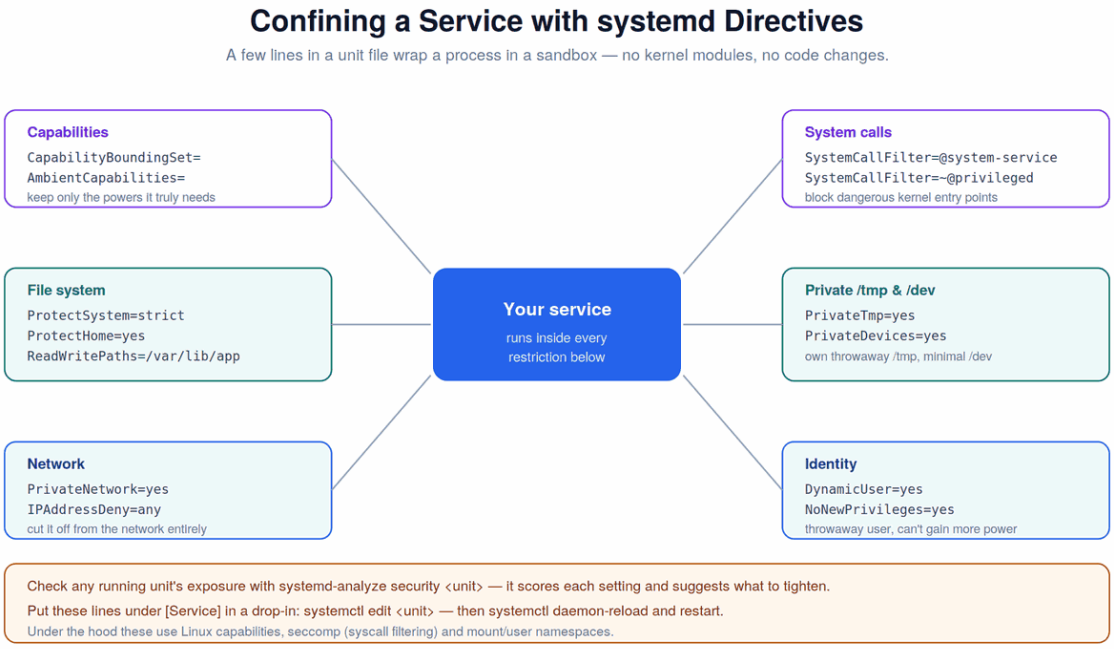


Figure 332.1d — Common systemd sandboxing directives, each closing off one avenue of attack.

The directives that matter

Directive	Effect
`ProtectSystem=strict`	Mounts the entire filesystem read-only except a few areas you name.
`ProtectHome=yes`	Hides /home, /root and /run/user from the service.
`ReadWritePaths=`	The specific paths the service is still allowed to write.
`PrivateTmp=yes`	Gives the service its own /tmp, unlinked from everyone else's.
`PrivateDevices=yes`	Minimal /dev with no access to raw hardware.
`PrivateNetwork=yes`	No network namespace at all — the service cannot reach the network.
`IPAddressDeny=/Allow=`	Fine-grained per-address network filtering.
`NoNewPrivileges=yes`	The process (and its children) can never gain more privilege.
`CapabilityBoundingSet=`	The only capabilities the service may ever hold.
`SystemCallFilter=`	A seccomp allow/deny list of permitted system calls.
`DynamicUser=yes`	Runs under a single-use, allocated-on-the-fly user account.

A worked hardened unit

```
$ systemctl edit myapp.service      # creates a drop-in override
```

```
[Service]
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/var/lib/myapp
PrivateTmp=yes
PrivateDevices=yes
NoNewPrivileges=yes
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
SystemCallFilter=@system-service
SystemCallFilter=~@privileged @obsolete
IPAddressDeny=any
IPAddressAllow=10.0.0.0/8

$ systemctl daemon-reload && systemctl restart myapp.service
```

Measure before and after

`systemd-analyze security myapp.service` gives a unit an exposure score from 0 (locked down) to 10 (fully exposed) and lists exactly which settings would improve it. Run it, apply the suggestions that fit the service, and run it again — it turns hardening into a concrete, checkable task.

5. Working with chroot environments

A **chroot** ("change root") runs a process with a chosen directory acting as its `/`. From inside, the rest of the real filesystem is invisible — the process cannot even name files outside its new root. It is the classic way to jail a service, and is still used by SFTP-only accounts, some DNS and FTP daemons, and rescue environments.

Building a minimal chroot

A program in a chroot can only use what is inside it, so you must copy in the binary and every shared library it needs. Find the libraries with `ldd`.

```
$ mkdir -p /srv/jail/{bin,lib,lib64}
$ cp /bin/bash /srv/jail/bin/
$ ldd /bin/bash                        # list the libraries bash needs
   libtinfo.so.6 => /lib64/libtinfo.so.6
   libc.so.6     => /lib64/libc.so.6
   /lib64/ld-linux-x86-64.so.2
$ cp /lib64/{libtinfo.so.6,libc.so.6,ld-linux-x86-64.so.2} /srv/jail/lib64/

$ chroot /srv/jail /bin/bash          # enter the jail
bash-5.1#  ls /                        # sees only the jail contents
```

chroot is containment, not a security boundary

A `root` process can often break out of a chroot (for example via a second `chroot` call or by creating device nodes), so on its own it is a safety aid, not an airtight jail. For real isolation prefer the systemd directives above, or full containers/VMs. Where you must rely on chroot, ensure the jailed process does not run as root.

6. ASLR, DEP/NX and Exec-Shield

Several kernel and CPU features make memory-corruption exploits — the mechanism behind most buffer overflows — far harder to pull off. They are on by default on modern systems; you must know what they do and how to check and toggle them with sysctl.

Address Space Layout Randomization (ASLR)

ASLR randomises where the stack, heap, libraries and (for PIE binaries) the program itself are placed in memory each time a process starts. Because the attacker cannot predict addresses, exploits that jump to a fixed location fail. It is controlled by `kernel.randomize_va_space`: `0` off, `1` partial, `2` full (the default).

```
$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
$ sysctl -w kernel.randomize_va_space=2      # set live
# persist it: add to /etc/sysctl.conf (or a file in /etc/sysctl.d/)
kernel.randomize_va_space = 2
$ sysctl -p                                   # reload sysctl.conf
```

DEP/NX and Exec-Shield

DEP (Data Execution Prevention) uses the CPU's **NX** (No-eXecute) bit to mark data pages — the stack and heap — as non-executable, so injected shellcode simply will not run. It is a hardware feature, enabled by default; you can confirm it in the kernel log (`dmesg | grep -i "NX (Execute Disable)"`). **Exec-Shield** is an older Red Hat mechanism in the same spirit (approximating NX and adding stack randomisation), toggled via `kernel.exec-shield` on kernels that carry it.

7. Black/white-listing USB devices with USBGuard

A USB port is a powerful, often overlooked, way in. A malicious device can present itself as a keyboard and type commands the instant it is plugged in (a "BadUSB" or Rubber Ducky attack), or as a network adapter that hijacks the machine's routing, or as mass storage that quietly copies data off. **USBGuard** enforces a policy of exactly which USB devices may attach, defaulting to blocking anything not explicitly allowed.

Setting up a policy

```
# Generate an allow-list from the devices currently plugged in
$ usbguard generate-policy > /etc/usbguard/rules.conf
$ systemctl enable --now usbguard

# A rule in rules.conf looks like:
allow id 046d:c52b serial "..." name "USB Receiver" hash "..."

# Manage devices at runtime
$ usbguard list-devices                    # allowed / blocked, with IDs
$ usbguard allow-device 6                  # allow one, this session
$ usbguard block-device 6
$ usbguard allow-device 6 -p               # -p = permanent (writes a rule)
```

The daemon's own behaviour — the default policy for unlisted devices, whether to apply rules to already-connected devices, and so on — is set in `/etc/usbguard/usbguard-daemon.conf`. The device rules themselves live in `/etc/usbguard/rules.conf`. The two files are the exam's key paths.

Do not lock yourself out

Generate the policy while your keyboard and mouse are connected, or a default-deny policy will block the very devices you need to fix it. Test on a machine you can reach another way first.

8. SSH certificates with your own CA

The usual SSH model — copying public keys into every server's `authorized_keys` and trusting host keys on first sight (`known_hosts`) — does not scale and is hard to revoke. A better model is to run a small **certificate authority**: the CA signs short-lived certificates for both host keys and user keys, and then servers and clients only have to trust the one CA.

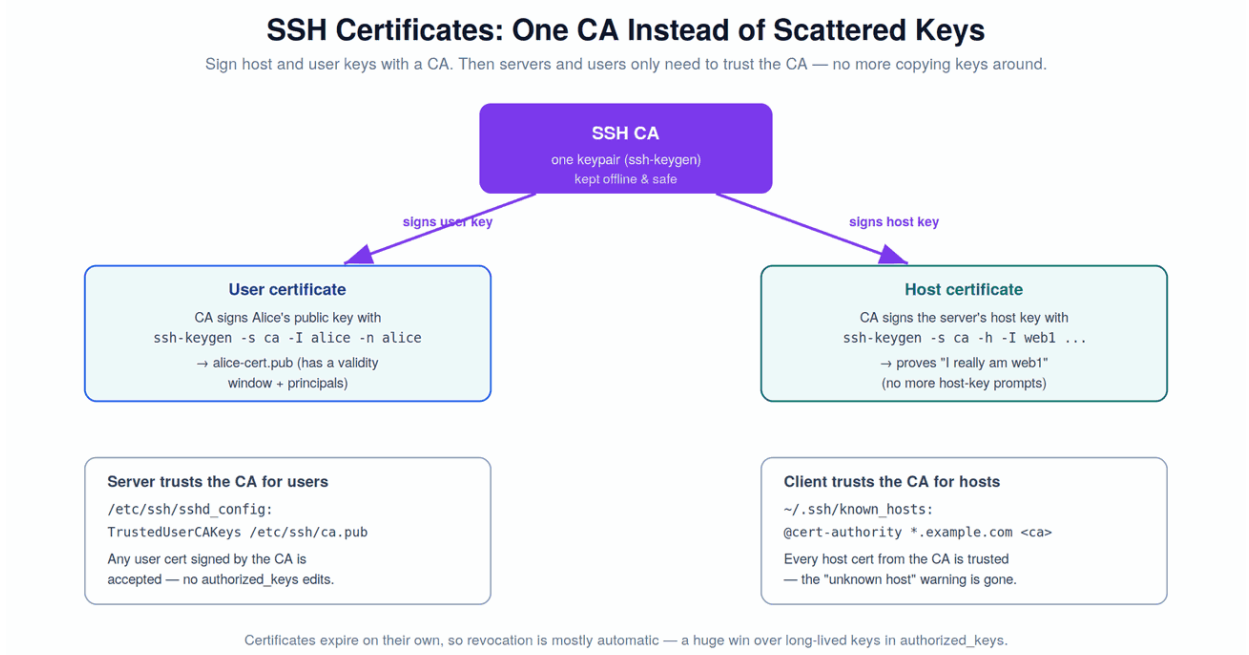


Figure 332.1e — One CA signs host and user certificates; servers trust it for users, clients trust it for hosts.

Create the CA and sign certificates

```
# 1. The CA keypair — keep the private key offline and very safe
$ ssh-keygen -t ed25519 -f ssh_ca -C "SSH CA"

# 2. Sign a USER certificate
#    -I identity (for logs)  -n principals (allowed login names)  -V validity
$ ssh-keygen -s ssh_ca -I alice@corp -n alice,admin -V +52w alice.pub
Signed user key alice-cert.pub: id "alice@corp" ... valid ...

# 3. Sign a HOST certificate  (-h marks it as a host key)
$ ssh-keygen -s ssh_ca -h -I web1 -n web1.example.com,web1 -V +52w \
    /etc/ssh/ssh_host_ed25519_key.pub
```

Tell servers and clients to trust the CA

```
# On each SERVER — trust the CA for USER certs, present its host cert
#   /etc/ssh/sshd_config :
```

```
TrustedUserCAKeys /etc/ssh/ssh_ca.pub
HostCertificate   /etc/ssh/ssh_host_ed25519_key-cert.pub
$ systemctl reload sshd

# On each CLIENT — trust the CA for HOST certs
#   ~/.ssh/known_hosts  (one line):
@cert-authority *.example.com ssh-ed25519 AAAA...  SSH CA
```

Now any user certificate the CA signed is accepted by every server — no `authorized_keys` edits — and every host certificate the CA signed is trusted by every client, so the "authenticity of host cannot be established" warning disappears. The relevant files live under `/etc/ssh/` on the server side and `~/.ssh/` per user.

> **Certificates make revocation easy**
>
> Because you set a short validity with `-V`, a compromised key mostly expires by itself. For immediate revocation, maintain a Key Revocation List and point `RevokedKeys` at it in `sshd_config`. You can also constrain certificates at signing time — for example `-O force-command=...` or `-O source-address=...` — to lock a certificate to one command or network.

9. Meltdown, Spectre and their mitigations

Meltdown and Spectre are a family of hardware vulnerabilities discovered in 2018. They exploit a performance feature of modern CPUs: speculative and out-of-order execution, where the processor runs instructions ahead of time, guessing which way a branch will go, and discards the work if it guessed wrong. The trouble is that the discarded work leaves a trace in the CPU cache, and by carefully timing memory access an attacker can read secrets — even across the boundary between a program and the kernel, or between two virtual machines on the same host.

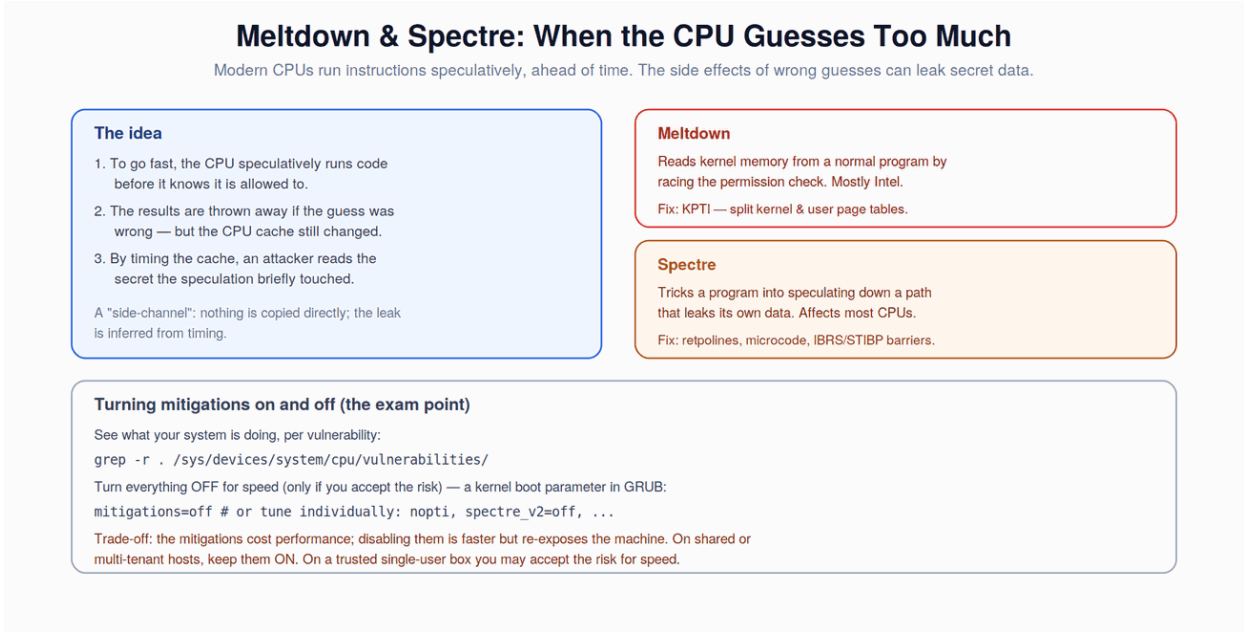


Figure 332.1f — Speculative execution leaks secrets through a cache timing side-channel; the two families and their mitigations.

Meltdown (mostly Intel) lets an ordinary program read kernel memory by racing the permission check; its main fix is `KPTI` (Kernel Page-Table Isolation), which fully separates kernel and user page tables. **Spectre** (nearly all

CPUs) tricks a program into speculatively executing down a path that leaks its own data; its fixes are subtler — retpolines, CPU microcode updates, and barriers such as IBRS/STIBP.

Checking and controlling the mitigations

The kernel exposes the status of every such vulnerability under `/sys/devices/system/cpu/vulnerabilities/`. Each file says whether the machine is vulnerable, mitigated, and how.

```
$ grep -r . /sys/devices/system/cpu/vulnerabilities/
/sys/.../meltdown:Mitigation: PTI
/sys/.../spectre_v2:Mitigation: Retpolines, IBPB: conditional, ...
/sys/.../spec_store_bypass:Mitigation: Speculative Store Bypass disabled ...

# Disable ALL mitigations (a kernel boot parameter — edit the GRUB entry
# or /etc/default/grub GRUB_CMDLINE_LINUX, then rebuild grub.cfg):
mitigations=off
# or tune individually: nopti (Meltdown off), spectre_v2=off, ...
```

The trade-off is the entire purpose of this objective

These mitigations cost real performance (sometimes significant, especially for syscall-heavy workloads). Disabling them makes the machine faster but re-exposes it. The rule: on shared, multi-tenant, or internet-facing hosts keep them **on**; on a trusted, single-purpose, physically isolated box you may accept the risk for speed. Know how to read `/sys/devices/system/cpu/vulnerabilities/` and how to set `mitigations=off` in GRUB.

10. polkit, virtualization and containers (awareness)

polkit (formerly PolicyKit) is the framework that lets an unprivileged process ask to perform a specific privileged action, subject to a policy — it is why a desktop user can mount a removable disk, install an update, or restart a service after an authentication prompt, without ever holding full root. Actions are defined by applications and authorised by rules; you should be aware that it provides fine-grained, per-action authorisation as an alternative to blanket sudo access.

You should also appreciate the security value of **virtualization and containerization**. A full virtual machine runs its own kernel under a hypervisor, so a compromise is contained within that guest — a strong boundary. A container shares the host kernel but isolates a process tree using namespaces and cgroups; it is lighter and faster but a thinner boundary, because a kernel vulnerability can potentially be reached from inside. Both are hardening tools in the sense that they shrink the scope of impact of a breach: an attacker who compromises one workload is confined to it rather than owning the whole host.

Reference — key files and tools (332.1)

Item	Role
`grub.cfg`	The generated GRUB config where the password hash and entries live.
`systemctl`	Enable/disable/mask units; the tool for cutting attack surface.
`getcap / setcap / capsh`	Read / set file capabilities; inspect and drop them in a shell.
`sysctl / /etc/sysctl.conf`	Read/set kernel parameters like ASLR; persist them.
`usbguard`	Enforce a USB device policy.

Item	Role
`/etc/usbguard/rules.conf`	The USB device allow/block rules.
`/etc/usbguard/usbguard-daemon.conf`	USBGuard daemon behaviour (default policy, etc.).
`ssh-keygen`	Create the CA and sign host/user SSH certificates.
`/etc/ssh/, ~/.ssh/, sshd_config`	Server and client SSH configuration and trust.
`chroot`	Run a process with a different filesystem root.

Exam tips — 332.1

- **Password-protect GRUB**, or a console attacker appends `init=/bin/bash` for a passwordless root shell; the hash lives in `grub.cfg`. Use `--unrestricted` to allow booting but not editing.
- **mask is stronger than disable** — a masked unit cannot be started at all. Find listeners with `ss -tulpn`.
- **getcap/setcap on files, capsh in a shell**; `CapabilityBoundingSet=` is the per-unit hard ceiling; file caps let a non-root program hold one power.
- **systemd sandboxing: ProtectSystem, PrivateTmp, PrivateDevices, PrivateNetwork, NoNewPrivileges, SystemCallFilter**; score with `systemd-analyze security`.
- **chroot needs the binary AND its libraries (ldd)**; it is containment, not an airtight jail — do not run root inside.
- **ASLR = kernel.randomize_va_space (2 = full) via sysctl**; DEP/NX is the CPU no-execute bit; Exec-Shield is the older equivalent.
- **USBGuard: rules.conf + usbguard-daemon.conf, generate-policy then default-deny**.
- **SSH CA: -h signs host certs; TrustedUserCAKeys trusts user certs; @cert-authority in known_hosts trusts host certs**.
- **Meltdown/Spectre: read /sys/devices/system/cpu/vulnerabilities/, toggle with mitigations=off in GRUB**; keep on for shared/exposed hosts.

332.2 Host Intrusion Detection

Objective weight: 5

Prevention is never perfect, so the second discipline of host security is detection: knowing, after the fact or in near-real-time, that something on the machine has changed or that a suspicious action has occurred. This objective covers four complementary approaches — recording events with the Linux Audit system, hunting for known-bad software with rootkit and malware scanners, verifying installed files against the package manager, and maintaining your own file-integrity baseline with AIDE — plus the discipline of automating all of it with cron.

A theme runs through all of them: a detection tool that is never looked at, or whose baseline the attacker can also edit, is worthless. Detection is a process, not a one-off command.

1. The Linux Audit system

The Linux Audit system is a kernel subsystem that records security-relevant events — system calls, file accesses, logins, changes to user accounts — according to rules you configure. Nothing is blocked; the value is an authoritative trail of **what happened, when, and by whom**. A user-space daemon, `auditd`, writes the events to disk, and `ausearch` and `aureport` read them back.

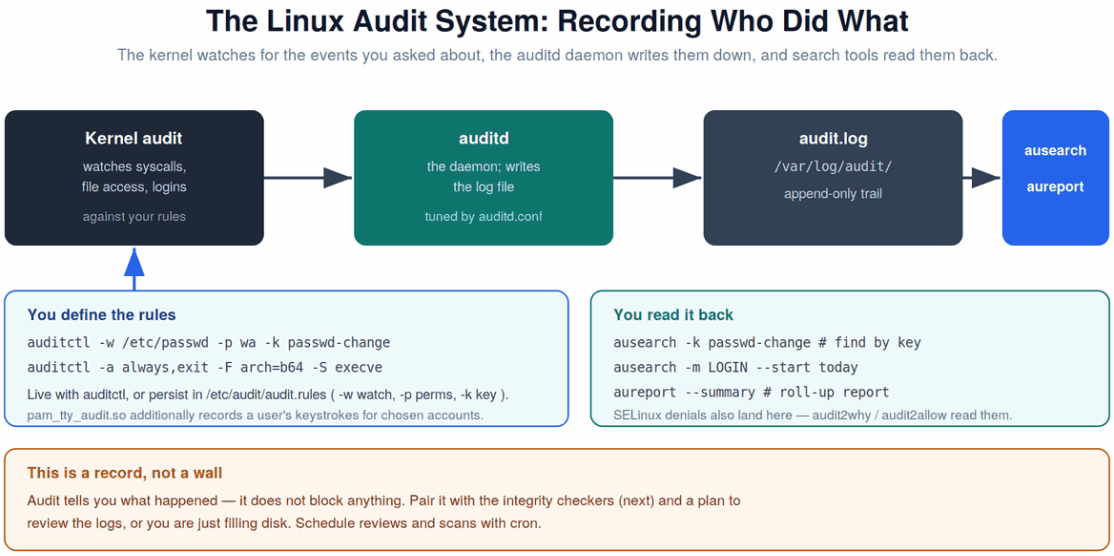


Figure 332.2a — You define rules; the kernel matches events; auditd logs them; ausearch and aureport read them back.

Writing audit rules

Rules are set live with `auditctl` and made persistent in `/etc/audit/rules.d/*.rules` (compiled into `/etc/audit/audit.rules`). There are two main kinds. A **file watch** (`-w`) fires when a path is accessed; a **syscall rule** (`-a`) fires on a system call, optionally filtered by fields. A **key** (`-k`) is a label you attach so you can find the events again later.

```
# Watch a file for writes (w) and attribute changes (a), tagged with a key
$ auditctl -w /etc/passwd -p wa -k passwd-change
$ auditctl -w /etc/shadow -p wa -k shadow-change

# Log every program execution on a 64-bit system
```

```
$ auditctl -a always,exit -F arch=b64 -S execve -k exec

# Log use of a syscall only by a specific user id
$ auditctl -a always,exit -F arch=b64 -S openat -F auid=1000 -k user-open

$ auditctl -l        # list active rules
$ auditctl -s        # status (enabled, backlog, lost events)
```

Reading the log back

```
$ ausearch -k passwd-change                # everything tagged passwd-change
$ ausearch -m LOGIN --start today          # by message type + time
$ ausearch -ua alice -i                    # by user, -i = interpret ids/names

$ aureport --summary                       # high-level roll-up
$ aureport -au                             # authentication report
$ aureport -f                              # file-access report
```

Daemon tuning and keystroke auditing

The daemon's behaviour is set in `/etc/audit/auditd.conf`: how large the logs may grow, how many to keep, and — importantly for high-security systems — what to do when the disk fills (`space_left_action`, `disk_full_action`, which can be set to halt the machine rather than lose audit records). Separately, the PAM module `pam_tty_audit.so` records the actual keystrokes typed by chosen accounts — useful for auditing administrator sessions.

```
# In /etc/pam.d/... enable TTY auditing for root logins:
session required pam_tty_audit.so enable=root
```

Audit records, it does not block

The audit system produces evidence; it prevents nothing. That makes two things essential: actually reviewing the logs (or shipping them to a central log server / SIEM out of the attacker's reach), and rotating them so they do not fill the disk. Note that SELinux denials (Topic 333) are recorded here too, which is why `ausearch -m avc` is the way to find them.

2. chkrootkit

`chkrootkit` is a simple, fast script that checks for the signatures of known rootkits — trojaned system binaries, tell-tale files, hidden processes, and network interfaces stuck in promiscuous mode. It needs no configuration; you just run it, ideally from cron and ideally comparing today's output with yesterday's.

```
$ chkrootkit
Checking `ls`... not infected
Checking `ifconfig`... not infected
Searching for suspicious files...  nothing found

$ chkrootkit -q          # quiet: only print problems (good for cron)
```

3. rkhunter

`rkhunter` (Rootkit Hunter) is a more thorough cousin. As well as scanning for rootkits and backdoors it checks system binaries against a database of known-good properties, looks for suspicious kernel modules, and flags configuration weaknesses. Its behaviour is tuned in `/etc/rkhunter.conf`.

The update / baseline / check cycle

rkhunter must be kept current or it misses new threats, and its file-property baseline must be updated whenever you legitimately patch the system — otherwise every update looks like an attack. The order is: refresh its data, re-baseline the file properties, then run the check.

```
$ rkhunter --update             # refresh the signature data files
$ rkhunter --propupd            # baseline current file properties (after patching)
$ rkhunter --check --sk         # run the check (--sk = do not pause between
sections)
$ rkhunter --check --rwo        # report warnings only (cron-friendly)

# results and details go to /var/log/rkhunter.log
```

Only run --propupd when you trust the system

`rkhunter --propupd` tells rkhunter "the current binaries are correct — remember them". Run it right after a clean install or a legitimate update, never after you suspect a compromise, or you will bless the attacker's changes as the new baseline.

4. Linux Malware Detect (maldet)

Linux Malware Detect (`maldet`) is a signature-based malware scanner aimed particularly at shared and web-facing servers — it is very good at finding malicious PHP, web shells, and other content dropped into upload directories. Its configuration is in `conf.maldet`, where you can enable email alerts, quarantine, and real-time monitoring of a directory tree via inotify.

```
$ maldet -u                     # update signatures (do this regularly)
$ maldet -a /var/www            # scan a path now
$ maldet -a /home?/username/public_html   # scan matching paths
$ maldet --monitor /var/www     # watch a tree in real time (inotify)
$ maldet --report <SCANID>      # view a past scan report
$ maldet -q <SCANID>            # quarantine hits from a scan
# behaviour (alerts, quarantine, monitoring) configured in conf.maldet
```

5. Verifying installed files with RPM and DPKG

Your package manager recorded a checksum, size, permissions, owner and timestamp for every file it installed, so it can tell you which of those files have since changed. This is a fast, zero-setup integrity check for anything that came from a package. Use `rpm -V` on RHEL-family systems and `dpkg -V` (or `debsums`) on Debian.

```
$ rpm -Va                       # verify ALL installed packages
$ rpm -V openssh-server         # verify one package
S.5....T.  c /etc/ssh/sshd_config

$ dpkg -V                       # Debian: verify all
$ dpkg -V openssh-server
```

The flag columns tell you exactly what differs, one character per attribute: `S` size, `5` MD5 checksum, `T` mtime, `M` mode/permissions, `U`/`G` owner/group, `L` symlink, `D` device. A `c` marks the file as a config file (changes there are expected). A changed checksum on a **binary** — like `/bin/ls` showing `5` — is a classic sign of a trojaned system tool.

> **The verification data can be tampered too**
>
> On a compromised host the RPM/DPKG database itself could be edited to hide changes, so package verification is a useful quick check, not absolute proof. For stronger assurance, verify from known-good media or combine it with AIDE, whose database you keep off the machine.

6. AIDE: your own file-integrity database

AIDE (Advanced Intrusion Detection Environment) is the most flexible integrity checker. Unlike package verification it is not limited to packaged files, and unlike the rootkit scanners it looks for *any* unexpected change rather than known signatures. The workflow is: fingerprint the system when it is known-clean into a baseline database, store that database safely, then re-scan later and diff.

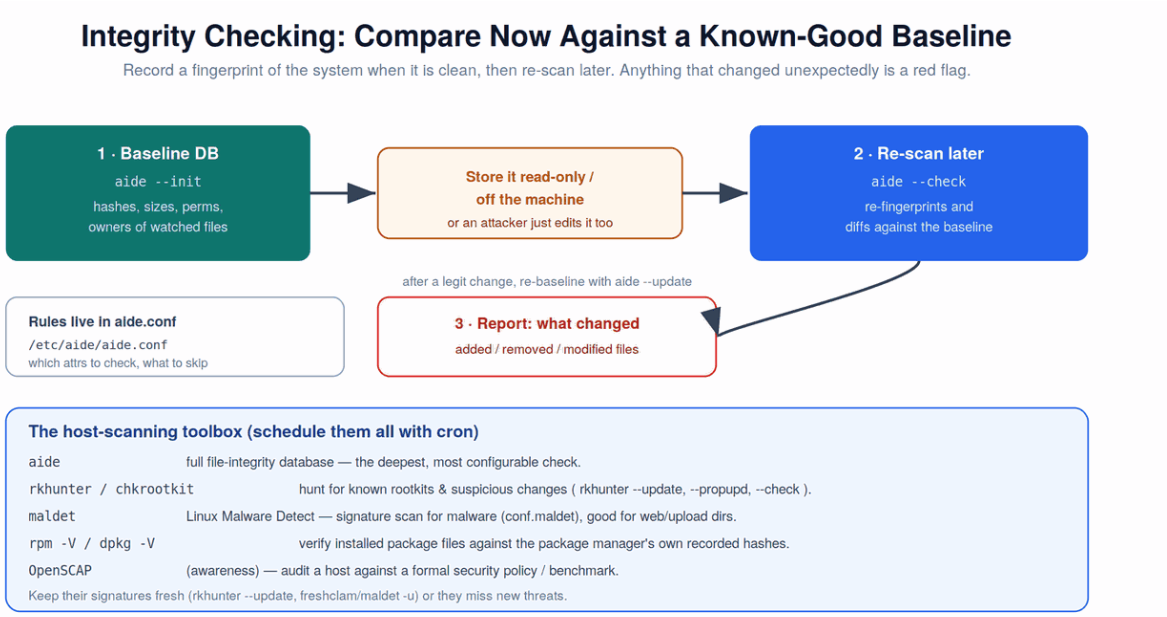


Figure 332.2b — Baseline once, store it safely, re-scan later, review the differences; and the wider host-scanning toolbox.

The workflow

```
$ aide --init                                   # build the baseline database
AIDE, version ...  ... database at /var/lib/aide/aide.db.new.gz
$ mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
#   ...now copy aide.db.gz somewhere read-only / off the machine...

$ aide --check                                  # compare current state vs baseline
Added entries / Removed entries / Changed entries ...

$ aide --update                                 # re-scan AND write a fresh
baseline
                                                # (use after a legitimate change)
```

The configuration and rule language

AIDE's rules live in `/etc/aide/aide.conf` (some distributions split them under `/etc/aide/`). You define named **groups** of attributes to check, then apply them to directories, and you can exclude paths that change constantly. Attribute letters include `p` permissions, `i` inode, `n` link count, `u`/`g` owner/group, `s` size, `m` mtime, `c` ctime, and hashes `md5`/`sha256`/`sha512`.

```
# /etc/aide/aide.conf — define rule groups, then apply them
Binlib   = p+i+n+u+g+s+b+m+c+sha256      # strict: for binaries/libraries
ConfFiles = p+i+u+g+sha256                # config files
Logs     = p+i+n+u+g                      # logs grow, so skip size/hash

/bin      Binlib
/sbin     Binlib
/etc      ConfFiles
/var/log  Logs
!/var/log/journal    # exclude a noisy path entirely (leading !)
```

Protect the baseline, and automate the checks

If the baseline database sits writable on the same machine, an attacker who changes a file can simply update the database to match — defeating the whole scheme. Keep a copy read-only or, better, off the host. And schedule the scan (`aide --check`) from **cron** with the results emailed to you; an integrity check run once at install time tells you nothing tomorrow.

7. OpenSCAP (awareness)

Be aware of **OpenSCAP**, a toolkit that evaluates a system against a formal, machine-readable security policy expressed in the SCAP standard (for example a CIS benchmark or a government STIG). Where AIDE answers "has this file changed?", OpenSCAP answers "does this system comply with the standard?", producing a scored report and often remediation guidance. The command-line tool is `oscap`. You need awareness of its purpose, not its detailed configuration.

8. Automating host scans with cron

Every tool in this chapter is only useful if it runs regularly, and none of them do so by themselves. cron is the glue: schedule the updates and the scans, and have the output mailed to an administrator or shipped off-host, so a change is noticed within a day rather than at the next audit.

```
# /etc/cron.d/security-scans
17 3 * * *  root  rkhunter --update && rkhunter --check --rwo | mail -s "rkhunter $
(hostname)" admin@example.com
30 3 * * *  root  aide --check | mail -s "AIDE $(hostname)" admin@example.com
45 3 * * 0  root  maldet -u ; maldet -a /var/www
00 4 * * *  root  chkrootkit -q | mail -s "chkrootkit $(hostname)"
admin@example.com
```

Reference — key files and tools (332.2)

Item	Role
`auditd / auditctl`	The audit daemon; the tool to set/list audit rules.
`ausearch / aureport`	Search the audit log; generate summary reports.

Item	Role
`auditd.conf / audit.rules`	Daemon behaviour; persistent audit rules.
`pam_tty_audit.so`	Record keystrokes for chosen accounts.
`chkrootkit / rkhunter`	Rootkit scanners; rkhunter config in `/etc/rkhunter.conf`
`maldet / conf.maldet`	Linux Malware Detect and its configuration.
`rpm -V / dpkg -V`	Verify installed files against the package database.
`aide / /etc/aide/aide.conf`	File-integrity database and its rules.

Exam tips — 332.2

- `auditctl -w PATH -p wa -k KEY` sets a watch; `-a always,exit -F arch=b64 -S execve` logs a syscall; read back with `ausearch -k` and `aureport`.
- **Tune the daemon in auditd.conf**; `pam_tty_audit.so` logs keystrokes; SELinux denials are in the audit log (`ausearch -m avc`).
- **rkhunter order: --update, --propupd (only when clean), then --check**; config `/etc/rkhunter.conf`.
- **maldet uses conf.maldet**; `-u` update, `-a` scan, `--monitor` watch.
- **rpm -V / dpkg -V flags: S size, 5 checksum, T mtime, M mode**; a changed checksum on a binary is suspicious; `c` = config file.
- **AIDE: --init to baseline, --check to compare, --update to re-baseline**; rules in `/etc/aide/aide.conf`; keep the DB off-host.
- **Automate everything with cron; OpenSCAP checks policy compliance (awareness)**.

332.3 Resource Control

Objective weight: 3

A machine can be denied service not only by an outside flood but by a single local process that consumes all the CPU, exhausts memory until the out-of-memory killer starts shooting, opens every file handle, or forks until the process table is full. Resource control is about putting boundaries on what programs may consume, so one misbehaving or hostile workload cannot starve the rest. Linux gives you two mechanisms: the older, per-process ulimits, and the far more capable control groups (cgroups), which systemd organises for you.

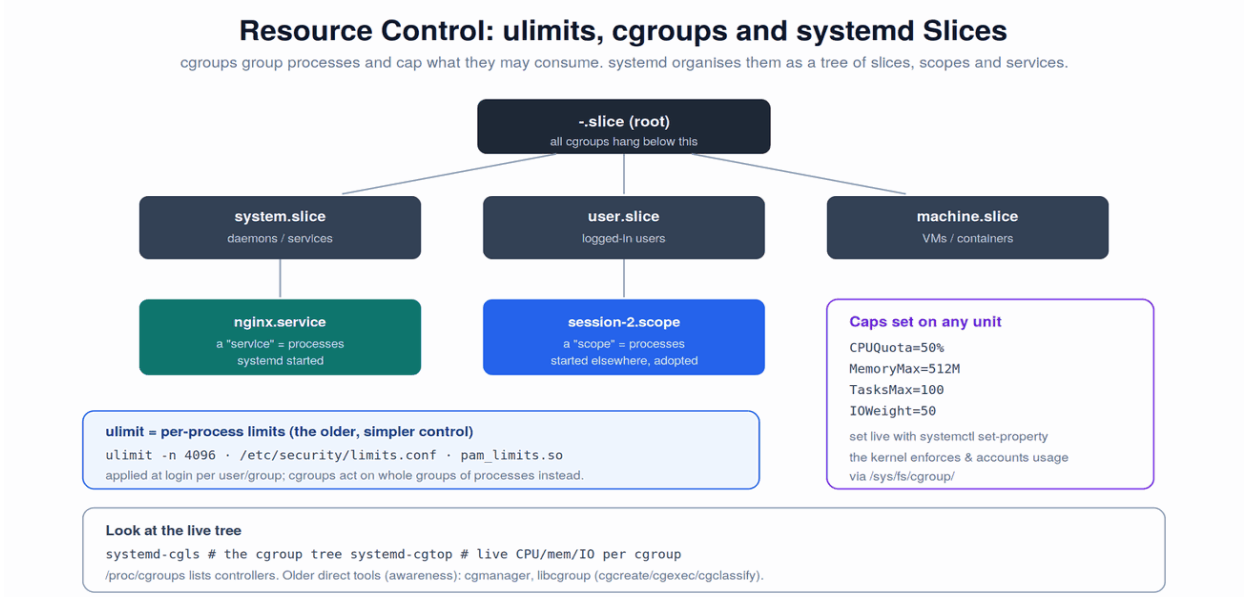


Figure 332.3 — ulimits cap a single process; cgroups cap and account for whole groups of processes, arranged by systemd into slices, scopes and services.

1. ulimits

A **ulimit** is a limit the kernel enforces on a process and its children — maximum open files, maximum address space, largest core dump, number of processes, locked memory, and more. Each limit has a **soft** value (the currently enforced ceiling, which a user may raise up to the hard value) and a **hard** value (the absolute maximum, which only root may raise).

Setting limits interactively

```
$ ulimit -a               # show all current (soft) limits
$ ulimit -n               # current soft limit on open files
$ ulimit -Hn              # the HARD limit on open files
$ ulimit -n 4096          # raise the soft limit for THIS shell + children
$ ulimit -u 200           # max user processes
$ ulimit -c unlimited     # allow core dumps
```

Persistent limits with limits.conf and pam_limits

Interactive changes vanish at logout. To set limits per user or group persistently, edit `/etc/security/limits.conf` (or a file in `/etc/security/limits.d/`). These are applied at login by the PAM module `pam_limits.so`. Each line is `domain type item value`: the domain is a user, a `@group`, or `*`; the type is `soft`, `hard` or `-` (both).

```
# /etc/security/limits.conf
#  <domain>     <type>  <item>    <value>
alice           hard    nofile    8192      # alice: max 8192 open files
@developers     soft    nproc     200       # group: soft cap on processes
*               hard    core      0         # everyone: no core dumps

# pam_limits must be in the PAM stack (usually already is):
#   session required pam_limits.so
```

2. Control groups (cgroups)

Where a ulimit governs a single process, a **control group** governs a whole set of processes together — you can cap the total CPU time, total memory, block-I/O bandwidth and number of tasks for the group as a unit, and the kernel also **accounts** for what the group actually uses. cgroups are the foundation beneath containers and systemd's resource management.

The live hierarchy is exposed as a virtual filesystem under `/sys/fs/cgroup/`, and the available controllers (cpu, memory, io, pids, …) are listed in `/proc/cgroups`. Modern systems use the unified **cgroup v2** hierarchy (a single tree), which replaced the older **v1** design of one tree per controller.

```
$ cat /proc/cgroups                    # which controllers exist
$ ls /sys/fs/cgroup/                    # the live cgroup tree
$ cat /sys/fs/cgroup/system.slice/nginx.service/memory.current   # live usage
```

3. Managing cgroups and process association

The cleanest way to place a process in a resource-controlled group is to let systemd do it. `systemd-run` launches a command in a fresh transient scope with limits attached; `systemctl set-property` changes limits on an existing unit. You can also move a running process by writing its PID into a cgroup's `cgroup.procs` file.

```
# Launch a command in a transient scope with limits
$ systemd-run --scope -p MemoryMax=200M -p CPUQuota=20% stress --cpu 2

# Move an already-running PID into a cgroup (v2)
$ echo 12345 > /sys/fs/cgroup/my.slice/cgroup.procs

# Older libcgroup way (awareness): create, then run a process in it
$ cgcreate -g cpu,memory:/limited
$ cgexec   -g cpu,memory:/limited  ./myprogram
$ cgclassify -g cpu,memory:/limited 12345
```

4. systemd slices, scopes and services

systemd models the cgroup tree with three unit types. A **service** (`.service`) is a set of processes that systemd itself started — a daemon. A **scope** (`.scope`) is a set of processes started by something else but registered with systemd — a user login session, a VM, a container. A **slice** (`.slice`) is a branch of the tree that groups services and scopes and can carry limits inherited by everything beneath it. The standard slices are `system.slice` (daemons), `user.slice` (user sessions) and `machine.slice` (VMs and containers), all under the root `-.slice`.

This hierarchy is why you can, for example, cap *all* user sessions together by putting a memory limit on `user.slice`, or guarantee daemons a share of CPU independent of what users are doing.

5. Limiting resources with systemd unit directives

The practical tool for resource control on a systemd host is a handful of directives, settable in a unit file, in a drop-in, or live with systemctl. The kernel enforces them via cgroups.

Directive	Limits
`CPUQuota=50%`	CPU time — 50% means half of one core's worth.
`CPUWeight=100`	Relative CPU share when there is contention.
`MemoryMax=512M`	Hard memory ceiling (OOM-killed if exceeded).
`MemoryHigh=400M`	Soft ceiling — throttled, reclaimed under pressure.
`TasksMax=100`	Maximum number of tasks (processes/threads).
`IOWeight=50 / IOReadBandwidthMax=`	Block-I/O share / absolute bandwidth cap.

```
# In a unit or drop-in ([Service] section):
CPUQuota=50%
MemoryMax=512M
TasksMax=100

# Or set live, without editing files (persists as a drop-in):
$ systemctl set-property nginx.service MemoryMax=512M CPUQuota=50%

# Put a ceiling on ALL user sessions at once:
$ systemctl set-property user.slice MemoryMax=4G
```

6. Inspecting the live cgroup tree

Two tools show what is happening. `systemd-cgls` prints the cgroup hierarchy as a tree of slices, scopes and services with their processes; `systemd-cgtop` is a `top`-like live view of CPU, memory, and I/O **per cgroup** — invaluable for spotting which service is eating the machine.

```
$ systemd-cgls                # the whole cgroup tree
$ systemd-cgls /system.slice  # just the daemons
$ systemd-cgtop               # live resource use per cgroup
$ systemctl status nginx.service   # shows the unit's cgroup + current usage
```

7. cgmanager and libcgroup (awareness)

Before systemd managed cgroups universally, other tools did. Be **aware** of `cgmanager` (a daemon that brokered cgroup management, common in older LXC setups) and the `libcgroup` utilities — `cgcreate`, `cgexec`, `cgclassify`, `cgset` — which manipulate cgroups directly and can persist them via `/etc/cgconfig.conf`. On a modern systemd host you rarely need them, but they appear in the objective.

Reference — key files and tools (332.3)

Item	Role
`ulimit`	Per-process (shell) resource limits.
`/etc/security/limits.conf`	Persistent per-user/group limits.
`pam_limits.so`	PAM module that applies limits.conf at login.
`/sys/fs/cgroup/`	The live cgroup hierarchy (virtual filesystem).
`/proc/cgroups`	The available cgroup controllers.
`systemd-cgls / systemd-cgtop`	Show the cgroup tree / live per-cgroup usage.
`systemctl set-property`	Apply resource limits to a unit or slice.

Exam tips — 332.3

- **ulimit is per-process with soft and hard values**; persist in `/etc/security/limits.conf`, applied by `pam_limits.so` (`domain type item value`).
- **cgroups limit AND account for groups of processes**; tree in `/sys/fs/cgroup/`, controllers in `/proc/cgroups`.
- **service = systemd-started, scope = adopted, slice = a limitable branch** (system/user/machine).
- **Limit units with CPUQuota, MemoryMax, TasksMax, IOWeight**; set live with `systemctl set-property`.
- **systemd-cgls shows the tree, systemd-cgtop shows live usage**; `cgmanager`/`libcgroup` are awareness.

Topic 333 — Access Control

333.1 Discretionary Access Control (DAC)

Objective weight: 3

Discretionary Access Control is the access model you meet on your first day with Linux: the owner of a file decides who may read, write or run it. It is called "discretionary" because that power is at the owner's discretion — they can hand out access freely, and the superuser can override anything. This objective covers the three layers of Linux DAC: the classic owner/group/other permission bits (with the special SetUID, SetGID and sticky bits), Access Control Lists for finer-grained grants, and extended attributes, the metadata store that underpins both ACLs and SELinux labels.

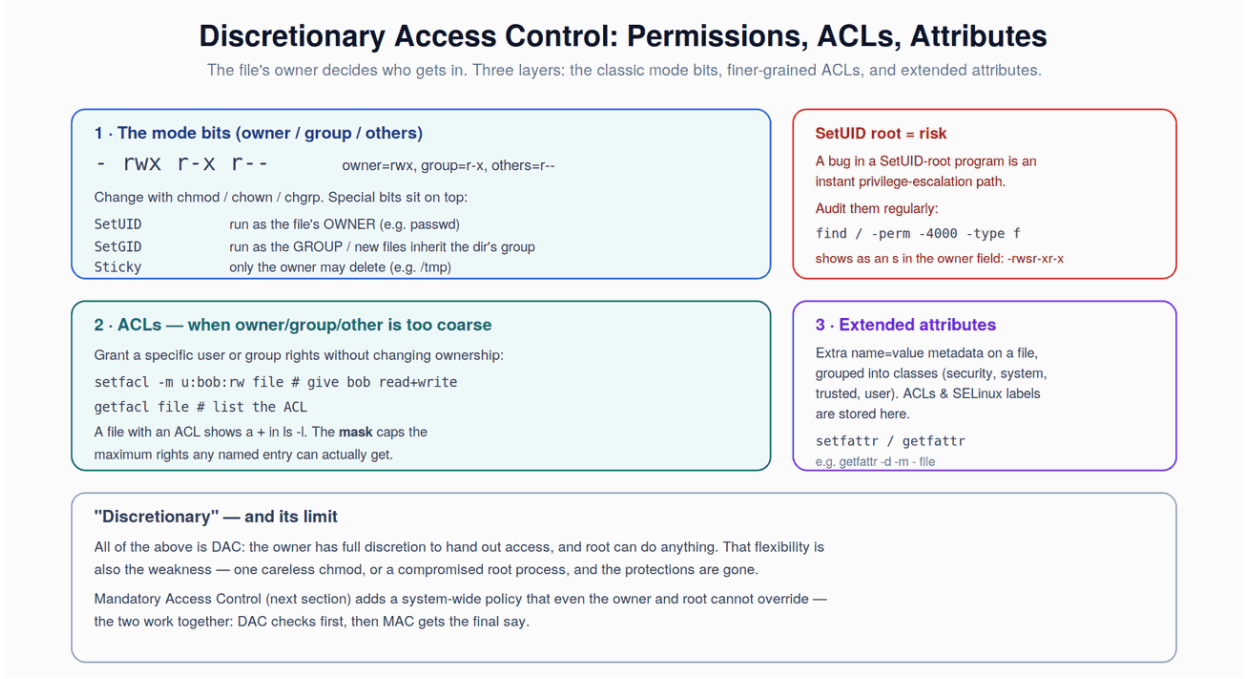


Figure 333.1 — The three layers of Linux DAC, and why "discretionary" has a limit that motivates Mandatory Access Control.

1. Ownership and the permission model

Every file has an **owner** (a user) and a **group**, and three sets of permissions — for the owner, for members of the group, and for everyone else ("other"). Each set has three bits: **read** (r), **write** (w) and **execute** (x). On a directory the meanings shift slightly: `r` lists the names, `w` creates/deletes entries, and `x` lets you enter it and access files by name.

Octal and symbolic notation

Permissions are written two ways. **Symbolic** notation names who and what (`u+x`, `go-w`). **Octal** notation encodes each triple as a digit — r=4, w=2, x=1, summed — so `rwxr-xr--` is `754`.

```
$ ls -l report.txt
-rw-r--r-- 1 alice staff 2048 Jun 10 09:00 report.txt
#  │└─ owner rw, group r, other r

$ chmod 640 report.txt        # octal: owner rw, group r, other none
$ chmod g+w,o-r report.txt    # symbolic equivalent of granting group write
$ chown alice:staff report.txt   # set owner and group
```

```
$ chgrp developers report.txt    # set group only
```

The umask

New files do not get all permissions; the **umask** subtracts a default set. A umask of `022` removes write from group and other, so a new file (base 666) becomes 644 and a new directory (base 777) becomes 755. Tighten it to `077` for private-by-default files.

```
$ umask           # show current mask
0022
$ umask 077       # new files: owner-only by default
```

2. The special permission bits: SetUID, SetGID, sticky

Above the nine ordinary bits sit three special bits that change how execution and ownership behave.

SetUID

A file with **SetUID** runs with the privileges of its *owner*, not the user who launched it. This is how `/usr/bin/passwd` — owned by root — lets an ordinary user update `/etc/shadow`. It shows as an `s` in the owner's execute position (`-rwsr-xr-x`).

SetGID

On a file, **SetGID** runs it with the *group's* privileges. On a **directory** it does something very useful: new files created inside inherit the directory's group rather than the creator's primary group — ideal for shared project folders. It shows as `s` in the group execute position.

The sticky bit

On a world-writable directory the **sticky bit** restricts deletion so that only a file's *owner* (or root) may remove it — which is exactly why anyone can create files in `/tmp` but not delete each other's. It shows as `t` in the other-execute position (`drwxrwxrwt`).

```
$ chmod 4755 prog     # SetUID (leading 4)   -> -rwsr-xr-x
$ chmod 2775 shared   # SetGID dir (leading 2) -> group inherited
$ chmod 1777 /tmp     # sticky (leading 1)     -> drwxrwxrwt
$ chmod u+s / g+s / +t   # symbolic forms
```

SetUID-root binaries are a top privilege-escalation target

A single bug in a SetUID-root program can hand an attacker root. Audit them regularly and remove the bit where it is not needed:

`find / -perm -4000 -type f 2>/dev/null` (SetUID) and `find / -perm -2000 -type f` (SetGID).

3. Access Control Lists (ACLs)

The owner/group/other model is coarse: it can express "the owner, one group, and everyone else", but not "Bob and Carol may write, the audit group may read, nobody else anything". **ACLs** add exactly that — any number of named-user and named-group entries on a single file, without changing its ownership. A file that carries an ACL shows a `+` after its permission string in `ls -l`.

Setting and reading ACLs

```
$ setfacl -m u:bob:rw report.txt      # grant user bob read+write
$ setfacl -m g:audit:r report.txt     # grant group audit read
$ getfacl report.txt
# file: report.txt
# owner: alice
user::rw-
user:bob:rw-
group::r--
mask::rw-
other::r--

$ setfacl -x u:bob report.txt         # remove bob's entry
$ setfacl -b report.txt               # remove ALL acls
```

The mask, and default ACLs

The **mask** entry is the maximum effective permission any named user or group entry can have — if the mask is `r--`, an entry granting `rw-` is effectively cut to `r--`. It is recalculated as you add entries, and is how the group permission in `ls -l` relates to the ACL. A **default ACL** on a directory (`setfacl -d`, or the `d:` prefix) is inherited by every new file created inside it — the ACL equivalent of the SetGID trick.

```
$ setfacl -d -m u:bob:rwx /project    # default ACL: inherited by new files
$ setfacl -m m:r report.txt           # set the mask explicitly
$ getfacl /project | grep default
```

4. Extended attributes and attribute classes

Beneath ACLs and SELinux labels lies a general mechanism: **extended attributes** (xattrs) — arbitrary `name=value` pairs attached to a file by the filesystem. They are grouped into **classes** by a namespace prefix: `user` (anything you like, controlled by normal file permissions), `trusted` (visible only to privileged processes), `security` (used by SELinux and capabilities), and `system` (used by the kernel, e.g. to store ACLs).

```
$ setfattr -n user.comment -v "reviewed 2026-06" report.txt
$ getfattr -n user.comment report.txt
# file: report.txt
user.comment="reviewed 2026-06"

$ getfattr -d report.txt              # dump the user namespace
$ getfattr -d -m - report.txt         # dump ALL namespaces
$ setfattr -x user.comment report.txt # remove an attribute
```

Related but different: file attributes with chattr

Do not confuse xattrs with ext/btrfs/xfs **file attributes**, set with `chattr` and shown with `lsattr`. The most security-relevant is `+i` (immutable): even root cannot modify or delete an immutable file until the attribute is removed — useful for locking down config or log files. They are a separate mechanism from the `getfattr`/`setfattr` extended attributes the objective names, but worth knowing alongside them.

5. The limit of DAC — and the bridge to MAC

Everything in this objective is discretionary, and that is both its strength and its weakness. Because the owner (and root) decides, a single careless `chmod 777`, an over-broad ACL, or — worst — a compromised root process can hand access to an attacker, and nothing in the DAC model stops it. Root is all-powerful. **Mandatory Access Control** (the next objective) adds a system-wide policy that even the owner and root cannot loosen. The two work together: the kernel applies DAC first, and only if DAC allows the action does MAC get its say.

Reference — key tools (333.1)

Tool	Role
`chmod / chown / chgrp`	Set permissions and ownership.
`getfacl / setfacl`	Read / modify Access Control Lists.
`getfattr / setfattr`	Read / modify extended attributes.
`chattr / lsattr`	Set/show ext filesystem attributes (e.g. immutable).
`umask`	Default-permission mask for new files.

Exam tips — 333.1

- **SetUID runs as owner, SetGID as group (and sets group on new files in a dir), sticky = only owner deletes**.
- **Octal: r=4 w=2 x=1; leading digit 4=SUID, 2=SGID, 1=sticky**. Find SUID with `find / -perm -4000`.
- **setfacl -m / getfacl for ACLs; a + in ls -l marks one; the mask caps effective rights; -d sets a default ACL**.
- **setfattr / getfattr for extended attributes; classes are user, trusted, security, system**.

333.2 Mandatory Access Control (SELinux)

Objective weight: 5

Mandatory Access Control puts a system-wide security policy in charge — one that constrains every process, that users cannot loosen, and that even root is bound by. Linux's most capable MAC system is SELinux, originally from the NSA and now standard on the Red Hat family. This is a weight-5 objective and the most detailed in the topic, so we build it carefully: first the four access-control concepts, then how to read a label, the operating modes, managing file contexts and booleans, roles and users, policy, and finally the all-important workflow for diagnosing a denial. We close with awareness of AppArmor and Smack.

1. DAC, MAC, type enforcement and RBAC

Four terms must be crisply distinguished. **DAC** (discretionary) — the owner decides, as in 333.1, and root can override. **MAC** (mandatory) — a central policy decides, and no user (not even root) can relax it. **Type Enforcement (TE)** — SELinux's core mechanism: every process and object carries a label containing a *type*, and the policy is a list of which type may perform which operation on which. **RBAC** (role-based) — users are assigned *roles*, and roles are authorised to enter particular domains, layering on top of TE to control what a logged-in person may become.

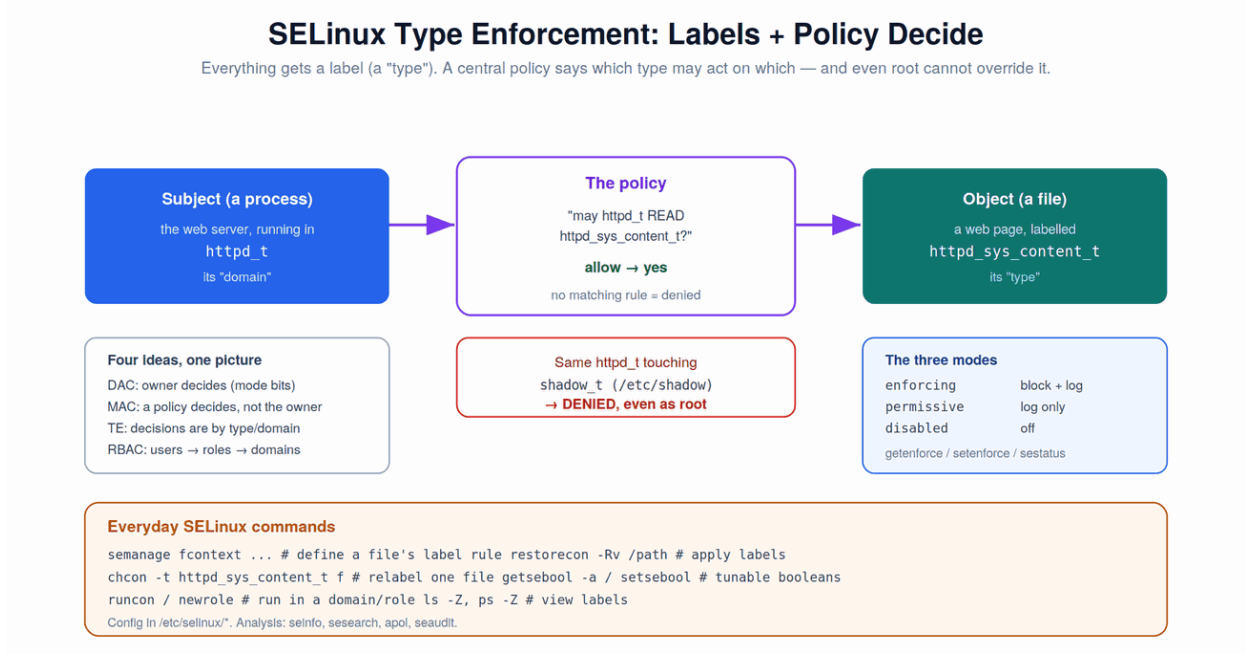


Figure 333.2a — Type enforcement: labels on subject and object, a policy that allows or denies, and the three modes.

2. Reading a context (label)

Everything SELinux protects carries a **context** (label) of four colon-separated parts: `user:role:type:level`. The **type** is the one that matters for day-to-day work — it is what type enforcement checks (a process's type is called its *domain*). The user and role drive RBAC; the level (sensitivity + categories) is only used by MLS/MCS policies. You see contexts everywhere with the `-Z` flag.

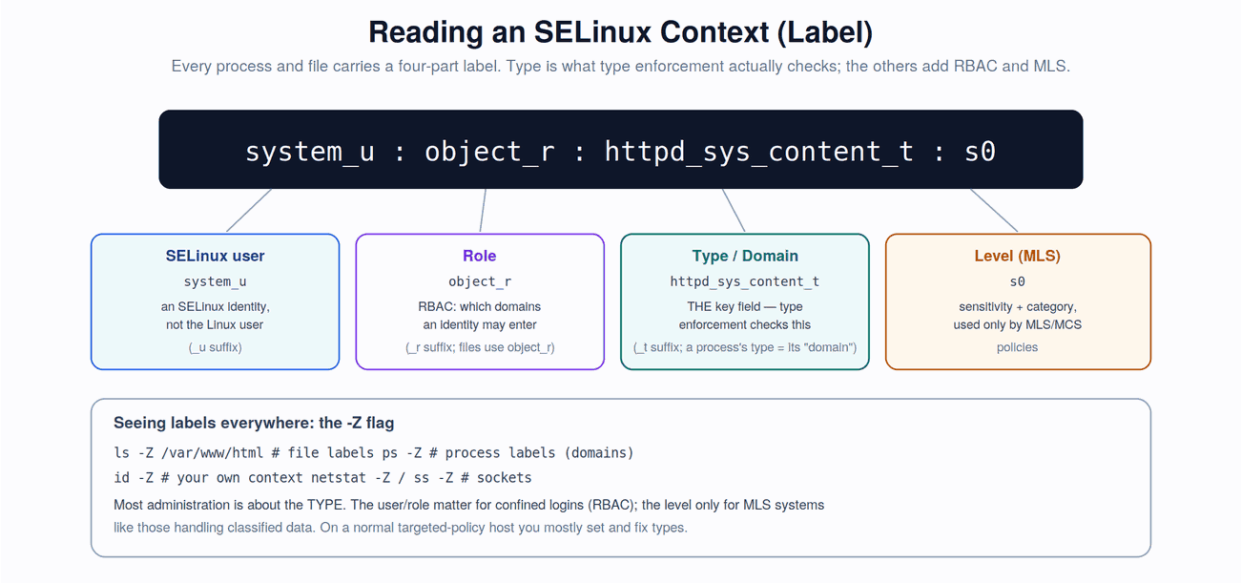


Figure 333.2b — The four parts of an SELinux context, and the -Z flag that reveals them on files, processes and sockets.

```
$ ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 index.html
$ ps -Z | grep httpd
system_u:system_r:httpd_t:s0 ... /usr/sbin/httpd
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0
```

3. Modes and status

SELinux runs in one of three modes. `enforcing` blocks and logs policy violations; `permissive` blocks nothing but logs what it *would* have blocked (invaluable for testing); `disabled` turns it off entirely. The runtime mode can be flipped between enforcing and permissive instantly, but switching to or from `disabled` requires editing `/etc/selinux/config` and rebooting (and often a full relabel).

```
$ getenforce                # Enforcing / Permissive / Disabled
$ sestatus                  # full status: mode, policy, mount info
$ setenforce 0              # -> permissive (1 -> enforcing), runtime only
$ selinuxenabled && echo on  # scriptable yes/no (exit status)

# Persistent mode + policy in /etc/selinux/config :
SELINUX=enforcing           # enforcing | permissive | disabled
SELINUXTYPE=targeted        # the policy to load
```

4. Labels and file contexts

Most SELinux problems are really labelling problems, so managing file contexts is the core skill. There are two levels. A **permanent rule** says "files matching this path pattern should have this type"; you add it with `semanage fcontext`, and then apply it to files on disk with `restorecon`. A **one-off relabel** with `chcon` changes a file's label immediately but is *not* remembered — a future `restorecon` or filesystem relabel will revert it. Always prefer the `semanage + restorecon` pair for anything permanent.

```
# Permanent: define the rule, then apply it
$ semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
$ restorecon -Rv /web

# One-off (not persistent): relabel a single file now
$ chcon -t httpd_sys_content_t /web/index.html

# Restore everything to policy defaults
$ restorecon -Rv /var/www
$ setfiles /etc/selinux/targeted/contexts/files/file_contexts /path   # low-level
$ fixfiles relabel            # relabel the whole filesystem (or:
touch /.autorelabel; reboot)
```

restorecon vs setfiles vs fixfiles

They are the same idea at different scopes: `restorecon` relabels given files from the policy's rules, `setfiles` is the lower-level tool it is built on (used during installation), and `fixfiles` is a convenience wrapper that relabels whole filesystems or packages. Use `restorecon -Rv` for everyday fixes.

5. Booleans: tunable policy switches

Rather than force you to write policy for common variations, SELinux ships **booleans** — named on/off switches that toggle whole groups of rules, such as "may the web server connect to the network?" or "may it read users' home directories?". Toggling a boolean is usually the correct fix when the *type* is right but a specific behaviour is being blocked. Use `-P` to make the change persist across reboots.

```
$ getsebool -a | grep httpd            # list booleans for a subsystem
httpd_can_network_connect --> off
$ setsebool -P httpd_can_network_connect on   # -P = persistent
$ togglesebool httpd_enable_cgi        # flip a boolean (runtime)
$ semanage boolean -l | grep httpd     # show booleans with descriptions
```

6. Users, roles and login mapping (RBAC)

SELinux has its own **users** (identities ending `_u`) that are distinct from Linux accounts, and **roles** (ending `_r`) that gate which domains a user may enter. Linux logins are mapped to SELinux users so that, for example, ordinary users land in the confined `user_u`/`staff_u` while admins may assume `sysadm_r`. This is how SELinux confines *people*, not just services.

```
$ semanage user -l                     # SELinux users and their roles
$ semanage login -l                    # Linux user -> SELinux user mapping
$ semanage login -a -s staff_u alice   # map the Linux user alice to staff_u

$ newrole -r sysadm_r                  # switch to another role (prompts for auth)
$ runcon -t mytype_t -- cmd            # run a command in a chosen type/context
$ setcon <context>                     # set the current process context
```

7. Policy types and modules

A running system loads one **policy**. The usual one is `targeted`, which confines a defined set of high-risk services and leaves everything else in the loose `unconfined_t` domain — a practical balance. At the other

extreme is `MLS` (Multi-Level Security), which enforces sensitivity levels for classified environments. Policy is modular: `semodule` lists, installs and removes `.pp` policy modules, and all of it lives under `/etc/selinux/`.

```
$ sestatus | grep "Policy"       # which policy is loaded
$ semodule -l                    # list installed policy modules
$ semodule -i mymodule.pp        # install a module
$ semodule -r mymodule           # remove one
```

8. Troubleshooting: from a denial to the right fix

When SELinux blocks an action it records an **AVC** (Access Vector Cache) denial in the audit log. The disciplined workflow is: find the denial, understand *why* it happened, and then choose the **narrowest** correct fix — usually a relabel or a boolean, and only as a last resort a custom policy module.

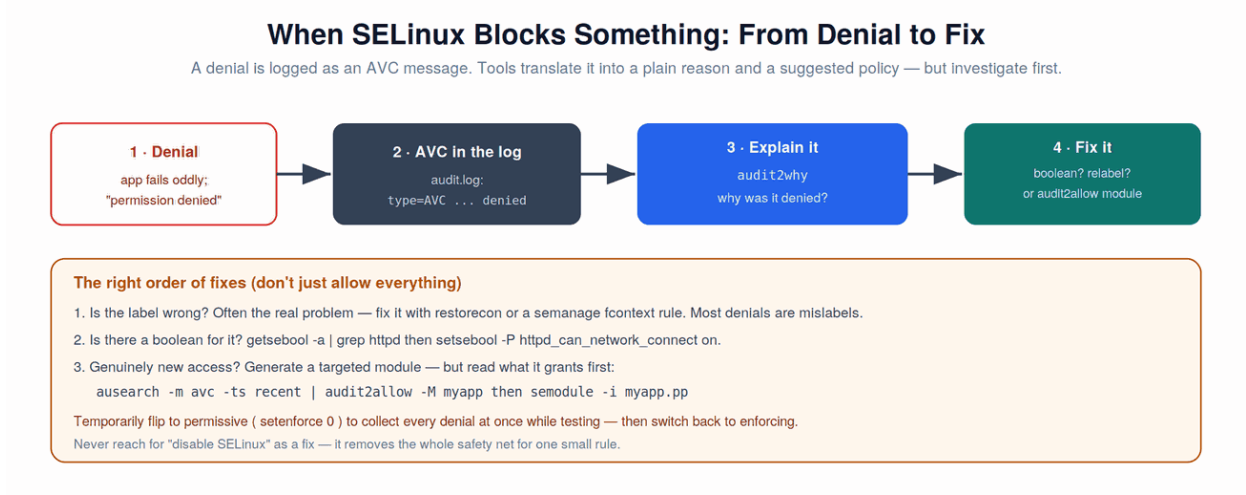


Figure 333.2c — From a denial, to the AVC log entry, to an explanation, to the right fix.

```
# 1. Find recent denials
$ ausearch -m avc -ts recent
$ journalctl -t setroubleshoot          # human-readable, if setroubleshoot is
installed

# 2. Explain them
$ ausearch -m avc -ts recent | audit2why
$ sealert -a /var/log/audit/audit.log   # detailed analysis + suggestions

# 3. If (and only if) the access is legitimate and no boolean/label fits,
#    generate a targeted module — READ what it grants first:
$ ausearch -m avc -ts recent | audit2allow -M myapp
$ cat myapp.te                          # inspect the proposed rules
$ semodule -i myapp.pp                   # install it
```

Fix in the right order — never just disable SELinux

1. Wrong label? Most denials are. Fix with `restorecon` or a `semanage fcontext` rule. 2. Is there a boolean? `setsebool -P`. 3. Only then, a custom module. To gather every denial in one pass while testing, flip to `permissive` (`setenforce 0`), reproduce the problem, fix, then return to `enforcing`. Disabling SELinux to "fix" one rule throws away the entire safety net.

9. Analysis tools

For understanding a policy rather than just reacting to it, SELinux offers query tools: `seinfo` summarises a policy (how many types, roles, booleans), `sesearch` searches the allow-rules ("what may httpd_t do?"), `apol` is a graphical policy analyser, and `seaudit` is a graphical audit-log analyser.

```
$ seinfo                          # policy summary (types, roles, booleans...)
$ seinfo -t | grep httpd          # list types matching httpd
$ sesearch -A -s httpd_t -t httpd_sys_content_t   # allow-rules for that pair
```

10. AppArmor and Smack (awareness)

SELinux is not the only MAC system, and all of them plug into the kernel's **LSM** (Linux Security Modules) framework. **AppArmor** (Ubuntu, SUSE) confines programs by *file path* rather than by label, using one profile per application — much easier to read and write than SELinux policy, at the cost of some granularity (its profiles are in `/etc/apparmor.d/`, managed with `aa-enforce`/`aa-complain`). **Smack** (Simplified Mandatory Access Control Kernel) uses simple text labels and is popular in embedded systems. For the exam you need awareness of their existence and broad approach — path-based vs label-based — not their configuration.

Reference — key commands and files (333.2)

Item	Role
`getenforce / setenforce / sestatus`	Show / change mode; full status.
`selinuxenabled`	Scriptable check whether SELinux is on.
`getsebool / setsebool / togglesebool`	Read / set / flip policy booleans.
`semanage`	Manage fcontext, login, user, boolean, port rules.
`restorecon / setfiles / fixfiles`	Apply policy labels to files.
`chcon`	Change one file's label (not persistent).
`runcon / newrole / setcon`	Run/switch in a context or role.
`audit2why / audit2allow`	Explain a denial; generate a policy module.
`seinfo / sesearch / apol / seaudit`	Analyse policy and audit logs.
`/etc/selinux/*`	SELinux configuration and policy.

Exam tips — 333.2

- **MAC policy binds even root; SELinux's core is type enforcement on labels**; a context is `user:role:type:level` and the type is what matters.
- **getenforce/setenforce/sestatus/selinuxenabled for mode; persistent in /etc/selinux/config** (needs reboot to reach/leave disabled).
- **semanage fcontext + restorecon is permanent; chcon is temporary**; `restorecon/setfiles/fixfiles` apply labels.
- **Booleans: getsebool -a, setsebool -P, togglesebool** — the fix when the type is right but a behaviour is blocked.
- **Denials are AVCs; ausearch -m avc finds them, audit2why explains, audit2allow builds a module** — fix labels/booleans first, never disable SELinux.

- **AppArmor confines by path, Smack uses simple labels; both are LSMs** — awareness only.

Topic 334 — Network Security

334.1 Network Hardening

Objective weight: 4

Network hardening secures the wire itself: controlling who may join the network, encrypting wireless links, watching what actually flows across them, and detecting the rogue devices that try to insert themselves into the path. Because so much of the work is about seeing traffic, the packet-capture tools live in this objective too.

1. Wireless security mechanisms

Wi-Fi is a shared radio medium — anyone in range can listen — so encryption is essential. The standards evolved through several generations, and knowing which is safe matters.

Standard	Verdict
`WEP`	Fundamentally broken (crackable in minutes). Never use.
`WPA (TKIP)`	A stop-gap over WEP hardware; weak, deprecated.
`WPA2 (AES-CCMP)`	The long-standing baseline; solid when using AES/CCMP.
`WPA3 (SAE)`	Current; stronger key exchange, resists offline cracking.

Each generation comes in two modes. **Personal (PSK)** uses one shared passphrase for the whole network — simple, but you cannot revoke one user without changing everyone's, and the passphrase can be captured and cracked offline. **Enterprise** authenticates every user individually against a RADIUS server using 802.1X/EAP, so each person has their own credentials that can be revoked. Note that hiding the SSID and filtering MAC addresses are *not* real security — both are trivially bypassed — so they supplement, never replace, proper encryption.

2. Authenticating nodes with FreeRADIUS (802.1X)

802.1X puts a gatekeeper on every switch port and Wi-Fi association. The device wanting on (the **supplicant**) must authenticate before it gets any network access; the switch or access point (the **authenticator**) does not decide itself — it relays the conversation to a central **RADIUS server**, which checks the credentials and returns Accept or Reject. FreeRADIUS is the standard Linux RADIUS server.

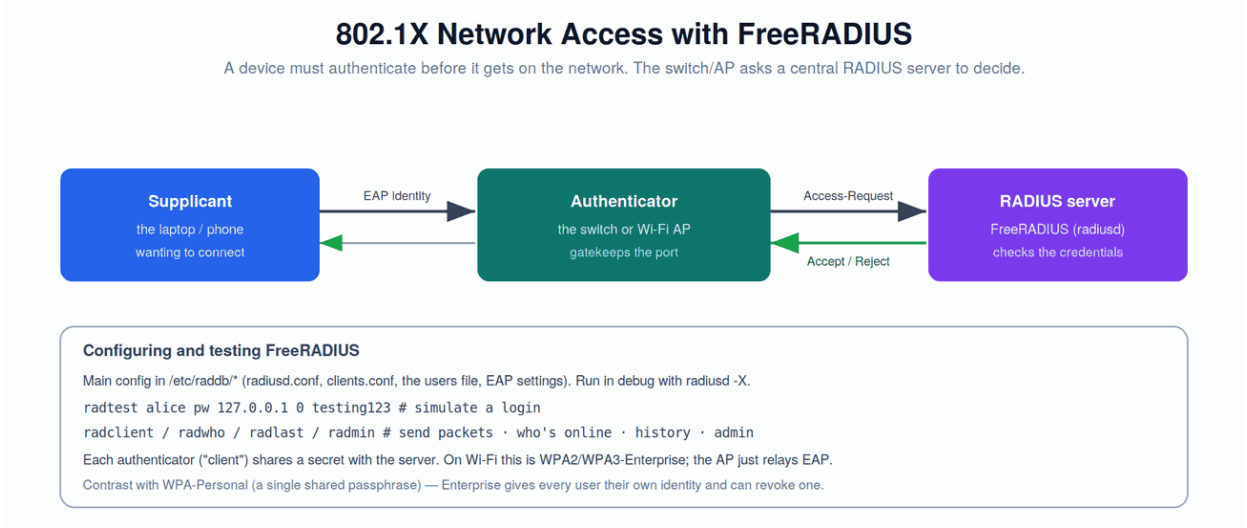


Figure 334.1 — 802.1X: the supplicant authenticates via EAP, the authenticator relays, and the RADIUS server decides.

EAP methods

The authentication itself uses EAP (Extensible Authentication Protocol). Common methods are `PEAP` and `EAP-TTLS` (the user sends a username/password inside a server-authenticated TLS tunnel) and `EAP-TLS` (both sides present certificates — the strongest, but needs a client-cert PKI).

Configuration and testing

FreeRADIUS configuration lives under `/etc/raddb/`: `radiusd.conf` (the main file), `clients.conf` (each authenticator/NAS and its shared secret), the `users` file or an SQL/LDAP backend (where credentials live), and the EAP module settings. The single most useful troubleshooting technique is to run the daemon in the foreground with `-X`, which prints every step of each request.

```
# clients.conf — define an authenticator (a switch/AP)
client ap-lobby {
    ipaddr = 192.0.2.20
    secret = SharedSecretHere
}

$ radiusd -X                          # run in full debug mode
$ radtest alice password 127.0.0.1 0 testing123   # simulate a login
Received Access-Accept ...
```

Tool	Use
`radiusd`	The FreeRADIUS server daemon (run -X to debug).
`radtest`	Send a test authentication request.
`radclient`	Craft and send arbitrary RADIUS packets.
`radwho / radlast`	Who is currently logged in / login history.
`radmin`	Administrative control socket.

3. Analysing traffic: tcpdump, Wireshark and tshark

To troubleshoot or investigate a network you capture packets and read them. `tcpdump` is the lightweight command-line capturer, `Wireshark` the powerful graphical analyser, and `tshark` Wireshark's command-line form. A crucial distinction: **capture filters** (BPF syntax, like `tcp port 80`) decide what is recorded and are used by all three; **display filters** (Wireshark syntax, like `http.request`) decide what is shown from an existing capture and are richer.

```
$ tcpdump -i eth0 -nn port 53           # DNS, no name/port resolution
$ tcpdump -i eth0 -w cap.pcap host 10.0.0.5   # write a capture file
$ tcpdump -r cap.pcap -A "tcp port 80"  # read back, -A = show ASCII payload

$ tshark -r cap.pcap -Y "http.request"  # DISPLAY filter on a saved file
$ tshark -qz io,stat,1                   # statistics: 1-second buckets
$ tshark -qz conv,tcp                    # TCP conversation summary
# Wireshark GUI: "Follow > TCP Stream", Statistics menu, colour rules
```

Capture filter vs display filter

They use different syntaxes and happen at different times. `port 80` (BPF) tells the kernel what to capture; `http` (Wireshark) tells the analyser what to display from what was already captured. Mixing them up is a

classic mistake.

4. Wireless analysis with Kismet

Kismet is a passive wireless detector and sniffer. With an adapter in **monitor mode** it discovers access points and clients (including those with hidden SSIDs), maps the RF environment, and captures 802.11 frames without ever associating — ideal for finding rogue or unauthorised access points on your premises. Be **aware** of the offensive counterparts: `aircrack-ng` (a suite for capturing handshakes and cracking WEP/WPA keys) and `bettercap` (a modular MITM and network-attack framework).

5. Rogue router advertisements and DHCP

On IPv6, hosts learn their gateway from **Router Advertisements** (RAs) sent via the Neighbor Discovery Protocol. A malicious host that sends its own RAs — or, on IPv4, a rogue DHCP server — can make victims route their traffic through it, enabling a man-in-the-middle. Defences: `ndpmon` monitors NDP and alerts on spoofed RAs, new routers and address anomalies; on managed switches, **RA Guard** and **DHCP snooping** block advertisements from ports that should not send them.

```
$ ndpmon              # daemon watching NDP; alerts on rogue RAs / NDP forgery
# switch-side: enable IPv6 RA Guard and DHCP snooping on access ports
# quick manual look:
$ tcpdump -i eth0 -nn icmp6 and "ip6[40] == 134"   # 134 = Router Advertisement
```

Reference and exam tips — 334.1

- **WPA2/WPA3-Enterprise = 802.1X + RADIUS (per-user); Personal = one PSK**; WEP is broken; hidden SSID/MAC filtering are not security.
- **FreeRADIUS: radiusd (-X to debug), config in /etc/raddb/*, clients.conf for NAS secrets, radtest to test**; `radclient/radwho/radlast/radmin`.
- **tcpdump captures (BPF filters), Wireshark/tshark analyse (display filters + statistics)**.
- **Kismet = passive wireless discovery/capture (monitor mode)**; `ndpmon` detects rogue RAs/NDP forgery; aircrack-ng/bettercap are awareness.

334.2 Network Intrusion Detection

Objective weight: 4

Where host intrusion detection watches a machine from the inside, network intrusion detection watches the wire — inspecting traffic for the signatures of attacks, and actively scanning hosts for the weaknesses an attacker would exploit. This objective covers bandwidth monitoring, the Snort IDS/IPS, and the OpenVAS vulnerability scanner. A recurring theme: a detector is only as good as its most recent update, so keeping signatures current — via cron — is part of the job.

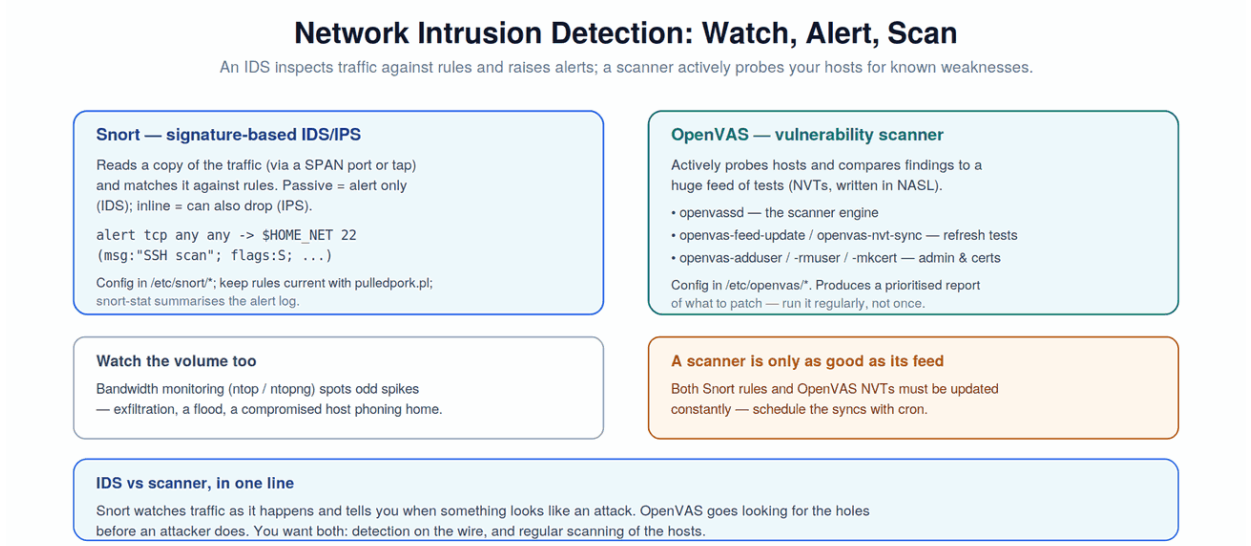


Figure 334.2 — Snort watches traffic for attack signatures; OpenVAS actively scans hosts for known weaknesses.

1. Bandwidth monitoring

Volume tells a story that signatures miss. A sudden outbound surge can be data exfiltration; a spike of connections can be a host recruited into a botnet or a flood in progress. `ntop` and its modern successor `ntopng` give a live, web-based view of who is talking to whom, over which protocols, and how much — a baseline of "normal" so the abnormal stands out.

2. Snort: signature-based IDS/IPS

Snort inspects network traffic against a set of rules and raises alerts on matches. It has three roles: a simple `sniffer`, a `packet logger`, and — the important one — a `network IDS`. Deployed **passively** (reading a mirror/SPAN port or a tap) it is an IDS that only alerts; deployed **inline** (traffic passes through it) it becomes an IPS that can *drop* malicious packets. Configuration and rules live under `/etc/snort/`.

The anatomy of a Snort rule

A rule has a **header** (action, protocol, source, direction, destination) and **options** in parentheses. Key options: `msg` (the alert text), `sid` (a unique rule id), `rev` (revision), `content` (a byte/string to match), `flags` (TCP flags), and `flow` (connection direction/state).

```
# header ..................... ( options )
alert tcp any any -> $HOME_NET 22 ( \
    msg:"SSH connection attempt"; flags:S; \
    flow:to_server; sid:1000001; rev:1; )
```

```
$ snort -T -c /etc/snort/snort.conf          # test the configuration
$ snort -A console -q -c /etc/snort/snort.conf -i eth0   # run as NIDS
$ snort-stat < /var/log/snort/alert          # summarise the alert log
```

Keeping rules current

Threats change daily, so Snort's rule sets must be refreshed. `pulledpork.pl` (or the newer PulledPork 3) downloads and manages rule updates from the Snort/Talos and community feeds, merging them into your local rule files. Schedule it from cron.

3. OpenVAS: vulnerability scanning

OpenVAS (the open scanner at the heart of Greenbone) takes the opposite approach to Snort: instead of watching traffic, it **actively probes** your hosts and compares what it finds against a large feed of tests called **NVTs** (Network Vulnerability Tests), which are written in the **NASL** language. The result is a prioritised report of the vulnerabilities on each host — what to patch, and how urgently.

```
$ openvassd                       # the scanner engine
$ openvas-feed-update             # refresh the NVT feed (older: openvas-nvt-sync)
$ openvas-mkcert                  # create the server certificate
$ openvas-adduser / openvas-rmuser        # manage scan users
# configuration under /etc/openvas/*
```

A scanner is only as good as its feed

Both Snort's rules and OpenVAS's NVTs must be updated constantly, or they miss the newest attacks — automate the syncs (and the scans themselves) with **cron**. And remember the division of labour: Snort tells you about attacks *as they cross the wire*; OpenVAS finds the holes *before* an attacker does. A mature setup runs both.

Reference and exam tips — 334.2

- **ntop/ntopng monitor bandwidth** — spikes signal exfiltration or floods.
- **Snort: config /etc/snort/*, rules updated by pulledpork.pl, summarised by snort-stat**; passive = IDS, inline = IPS; a rule = header + options (msg, sid, content, flags, flow).
- **OpenVAS tests are NVTs written in NASL**; `openvassd`, `openvas-feed-update / openvas-nvt-sync`, `openvas-adduser/-rmuser`, `openvas-mkcert`; config /etc/openvas/*.

334.3 Packet Filtering

Objective weight: 5

A firewall decides which packets may pass and which are dropped. On Linux this is netfilter, the in-kernel packet-filtering framework, driven from user space by iptables (and ip6tables for IPv6). This is the heaviest objective in the topic, so we build it thoroughly: the firewall architectures you are protecting, the path a packet takes through netfilter, how to write rules with the standard matches and targets, connection tracking and NAT, IP sets for scale, persistence, and awareness of the newer nftables.

1. Firewall architectures and the DMZ

Before rules come design. The guiding principle is **default-deny**: block everything, then allow only what is needed. For networks that host public services, the key pattern is the **DMZ** (demilitarised zone) — a semi-trusted segment between the Internet and the internal LAN, holding the servers that must be reachable from outside (web, mail, DNS). If one of those servers is breached, the attacker is contained in the DMZ; a second boundary still protects the trusted network. The classic rule: the Internet may reach the DMZ, but the DMZ may not initiate connections into the LAN.

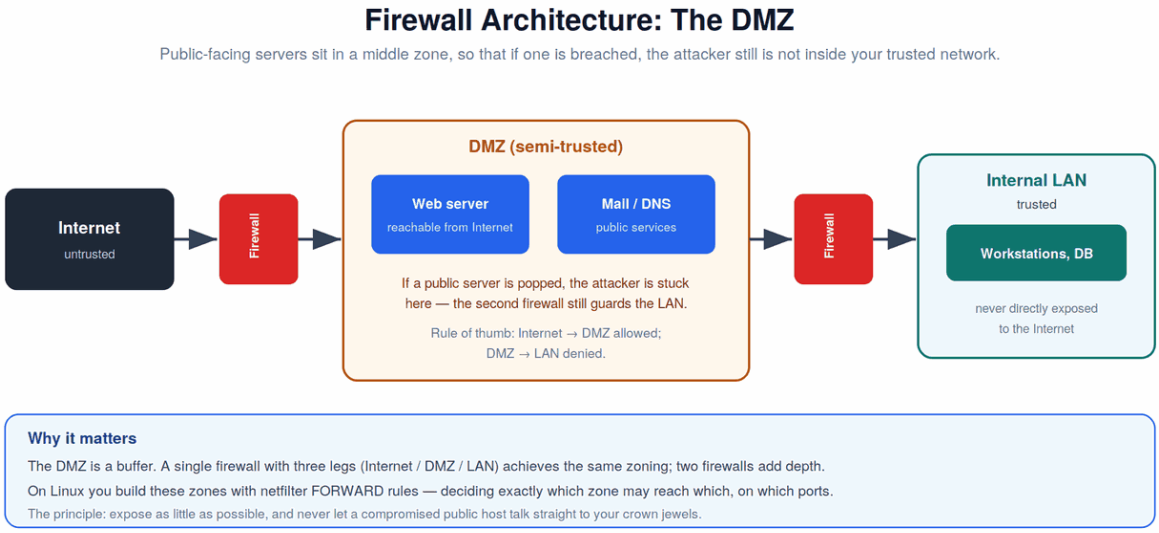


Figure 334.3a — A DMZ isolates public servers so a breach there cannot reach the internal LAN.

This can be built with two firewalls (Internet → firewall → DMZ → firewall → LAN) or a single three-legged firewall with separate interfaces for Internet, DMZ and LAN. On Linux the zoning is enforced with netfilter `FORWARD` rules deciding exactly which zone may reach which, on which ports.

2. How a packet traverses netfilter

netfilter hooks into the kernel's network path at fixed points called **chains**, grouped into **tables** by purpose. A routing decision early on determines a packet's path: one destined for *this host* goes `PREROUTING → INPUT`; one *this host originates* goes `OUTPUT → POSTROUTING`; one merely *routed through* goes `PREROUTING → FORWARD → POSTROUTING`.

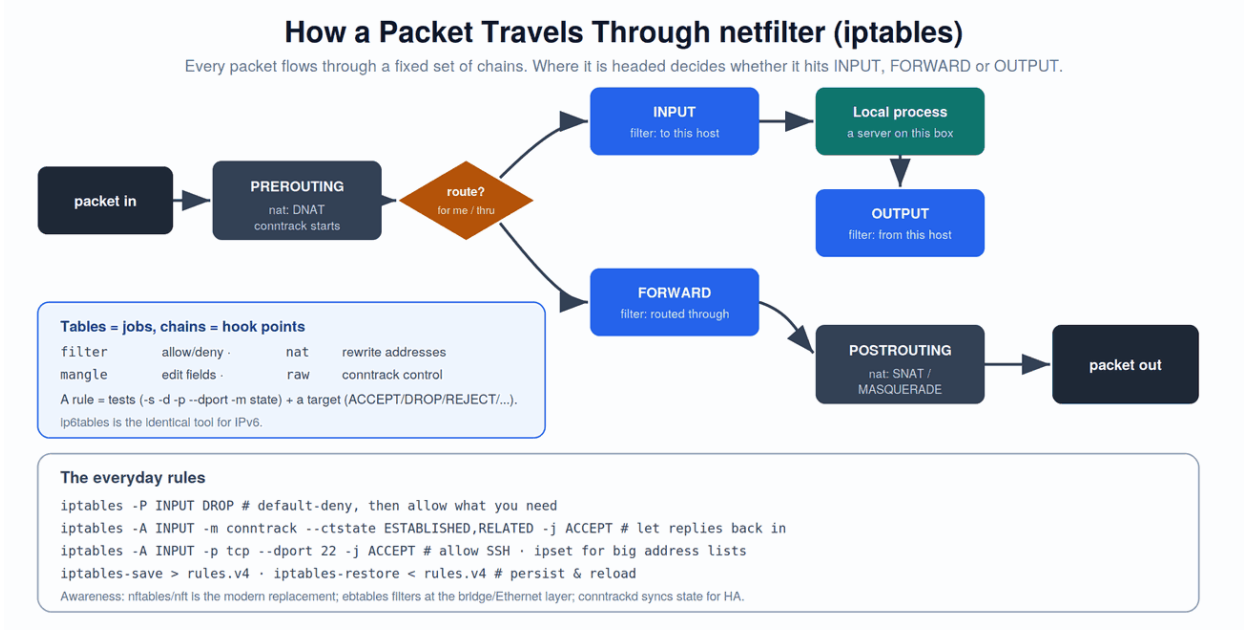


Figure 334.3b — The chains a packet crosses, and where filtering and NAT happen.

Table	Job
`filter`	Allow or deny (INPUT, OUTPUT, FORWARD). The default table.
`nat`	Rewrite addresses (PREROUTING, POSTROUTING, OUTPUT).
`mangle`	Alter packet fields (TOS, TTL, marks).
`raw`	Connection-tracking exemptions, processed first.

3. Writing rules: matches and targets

A rule is a set of **tests** plus a **target** (the action if all tests match). Basic tests come from the core: `-s`/`-d` addresses, `-p` protocol, `-i`/`-o` interfaces, `--sport`/`--dport` ports. Extended matches load with `-m`: `conntrack` (state), `multiport` (many ports), `limit` (rate-limit, e.g. for logging), `mac`, `string`, `iprange`. Common targets are `ACCEPT`, `DROP` (silently discard), `REJECT` (discard with an error), `LOG`, and the NAT targets `SNAT`/`DNAT`/`MASQUERADE`/`REDIRECT`.

A minimal default-deny ruleset

```
# Policies: default-deny inbound and forwarded, allow our own output
$ iptables -P INPUT DROP
$ iptables -P FORWARD DROP
$ iptables -P OUTPUT ACCEPT

# Always allow loopback and established/related replies
$ iptables -A INPUT -i lo -j ACCEPT
$ iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
$ iptables -A INPUT -m conntrack --ctstate INVALID -j DROP

# Allow the services this host offers
$ iptables -A INPUT -p tcp --dport 22 -j ACCEPT             # SSH
$ iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT  # web
$ iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT
```

```
# Log then drop the rest (rate-limited so logs do not flood)
$ iptables -A INPUT -m limit --limit 5/min -j LOG --log-prefix "DROP: "
```

IPv6 is separate

`ip6tables` is the identical tool for IPv6 and must be configured too — an IPv4-only firewall on a dual-stack host leaves the IPv6 side fully exposed. Remember IPv6 needs certain ICMPv6 (Neighbor Discovery) traffic to function, so do not blanket-drop ICMPv6.

4. Connection tracking and NAT

netfilter is **stateful**: the connection tracker (conntrack) records every flow, classifying packets as `NEW`, `ESTABLISHED`, `RELATED` or `INVALID`. This is what lets a single "accept established/related" rule admit all the return traffic for connections you allowed, rather than opening holes for every reply. **NAT** rewrites addresses: `SNAT`/`MASQUERADE` change the source on the way out (so many hosts share one public IP), `DNAT` changes the destination on the way in (port-forwarding). Routing between interfaces additionally needs `net.ipv4.ip_forward=1`.

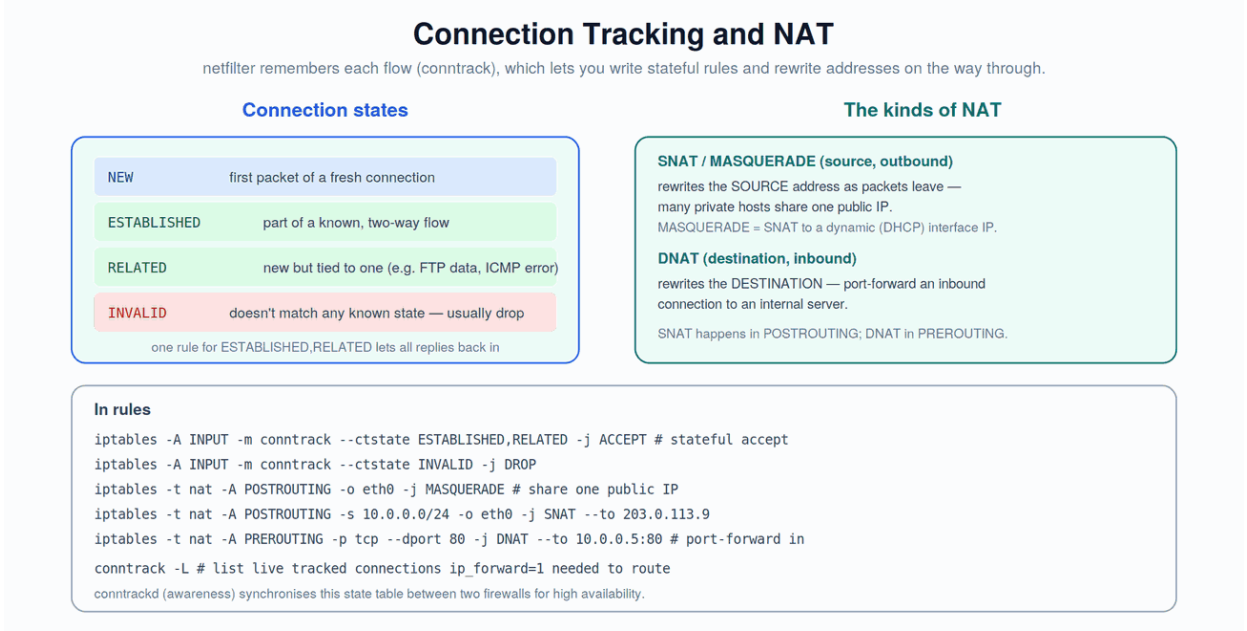


Figure 334.3c — Connection-tracking states and the kinds of NAT, with the rules that use them.

5. IP sets

When a rule must match thousands of addresses — a blocklist, an allow-list of partner networks — testing them one rule at a time is slow. An **ipset** stores a whole collection (of IPs, networks, ports, or MAC addresses) in an efficient structure, and a single iptables rule matches against the set. Sets can be updated live without touching the rules.

```
$ ipset create blocklist hash:ip           # a set of IP addresses
$ ipset add blocklist 198.51.100.7
$ ipset add blocklist 203.0.113.0/24        # networks too
$ iptables -A INPUT -m set --match-set blocklist src -j DROP
$ ipset list blocklist
```

```
$ ipset save > /etc/ipset.conf              # persist
```

6. Making rules persist

iptables rules live in kernel memory and vanish on reboot. Save them to a file and restore them at boot with the save/restore pair — and do the same for IPv6.

```
$ iptables-save  > /etc/iptables/rules.v4
$ iptables-restore < /etc/iptables/rules.v4
$ ip6tables-save > /etc/iptables/rules.v6
$ ip6tables-restore < /etc/iptables/rules.v6
# distributions wire these into a service (netfilter-persistent / iptables.service)
```

7. Awareness: nftables, ebtables, conntrackd

Be aware of the modern stack. **nftables** (command `nft`) is the designated successor to iptables: one framework and one syntax covering IPv4, IPv6, ARP and bridging, with named sets and maps built in and better performance. **ebtables** filters at the Ethernet/bridge layer (Layer 2), useful on bridging firewalls. **conntrackd** replicates the connection-tracking table between two firewalls so a failover does not drop every established connection (high availability).

Reference and exam tips — 334.3

- **To this host: PREROUTING→INPUT; from it: OUTPUT→POSTROUTING; through it: PREROUTING→FORWARD→POSTROUTING**.
- **Tables: filter, nat, mangle, raw**. A rule = matches (-s -d -p --dport -m …) + a target (ACCEPT/DROP/REJECT/LOG/…). Configure `ip6tables` too.
- **Stateful: accept ESTABLISHED,RELATED, drop INVALID**; SNAT/MASQUERADE out (POSTROUTING), DNAT in (PREROUTING); routing needs `ip_forward=1`.
- **ipset matches large address lists in one rule**; persist with `iptables-save/-restore` (and ip6tables).
- **Awareness: nftables/nft (successor), ebtables (Layer 2), conntrackd (HA state sync)**.

334.4 Virtual Private Networks

Objective weight: 4

A VPN carries private traffic across an untrusted network — usually the Internet — inside an encrypted tunnel, so two endpoints can communicate as if directly connected and no one in between can read or alter the data. This objective covers the two fundamental tunnel shapes and three tools: OpenVPN, IPsec (via strongSwan), and WireGuard.

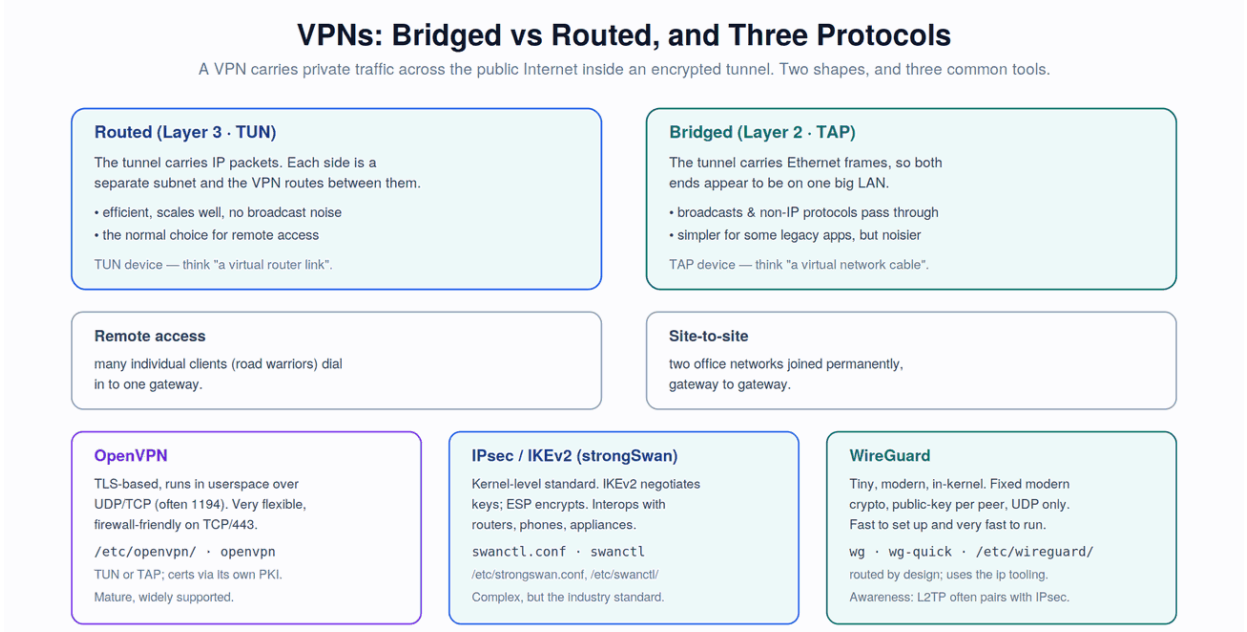


Figure 334.4 — Routed vs bridged tunnels, remote-access vs site-to-site, and the three protocols compared.

1. Bridged vs routed, remote-access vs site-to-site

A **routed** VPN operates at Layer 3 using a TUN device: it carries IP packets and routes between two subnets. It is efficient and the usual choice. A **bridged** VPN operates at Layer 2 using a TAP device: it carries Ethernet frames so both ends appear on a single LAN, which lets broadcasts and non-IP protocols pass but adds overhead and noise. Independently, a deployment is either **remote-access** (many individual clients dial in to a gateway) or **site-to-site** (two whole networks joined permanently, gateway to gateway).

2. The three protocols compared

	OpenVPN	IPsec / IKEv2	WireGuard
Runs in	userspace	kernel	kernel
Based on	TLS	IKE + ESP	modern fixed crypto
Transport	UDP or TCP	UDP (500/4500)	UDP only
Config	/etc/openvpn/	/etc/swanctl/	/etc/wireguard/
Strength	flexible, firewall-friendly	standard, interoperable	tiny, very fast

3. OpenVPN

OpenVPN is a mature, TLS-based VPN. It runs in userspace over UDP or TCP (commonly 1194, or 443/TCP to pass restrictive firewalls), uses its own certificate PKI, and can run in TUN (routed) or TAP (bridged) mode. Its files live under `/etc/openvpn/`.

```
# Server (server.conf, abbreviated):
port 1194
proto udp
dev tun
ca ca.crt   cert server.crt   key server.key   dh dh.pem
server 10.8.0.0 255.255.255.0
push "redirect-gateway def1"

# Client (client.ovpn): remote <server> 1194 ; ca/cert/key ; dev tun
$ openvpn --config client.ovpn
$ systemctl start openvpn-server@server     # or openvpn-client@client
```

4. IPsec with strongSwan

IPsec is the kernel-level industry standard, and **IKEv2** is the modern protocol that negotiates the keys before ESP encrypts the traffic. It interoperates broadly with routers, firewalls and mobile devices. On Linux the common implementation is **strongSwan**, configured through `/etc/swanctl/swanctl.conf` (with `/etc/strongswan.conf` and `/etc/strongswan.d/` for the daemon) and driven by the `swanctl` command.

```
# /etc/swanctl/swanctl.conf (abbreviated):
connections {
   site-a {
      remote_addrs = 198.51.100.2
      local  { auth = psk }
      remote { auth = psk }
      children { net { local_ts = 10.1.0.0/24; remote_ts = 10.2.0.0/24 } }
   }
}

$ swanctl --load-all                 # load config
$ swanctl --initiate --child net     # bring up the tunnel
$ swanctl --list-sas                 # show security associations
```

5. WireGuard

WireGuard is a small, modern, in-kernel VPN. It uses one fixed set of strong cryptographic primitives, one public/private key **per peer**, UDP only, and is routed by design. It is remarkably short to configure and very fast. Files live under `/etc/wireguard/`, managed with `wg` and the `wg-quick` helper, using the ordinary `ip` tooling for the interface.

```
$ wg genkey | tee privatekey | wg pubkey > publickey    # a keypair per peer

# /etc/wireguard/wg0.conf
[Interface]
Address = 10.9.0.1/24
```

```
ListenPort = 51820
PrivateKey = <server-private>

[Peer]
PublicKey = <client-public>
AllowedIPs = 10.9.0.2/32       # which src IPs this peer may use / routes to it

$ wg-quick up wg0        # bring it up (down to stop)
$ wg show                 # status: handshakes, transfer, endpoints
```

AllowedIPs does double duty

In WireGuard, a peer's `AllowedIPs` is both the routing table (which destinations go down this tunnel) and the access-control list (which source addresses that peer is permitted to use). Getting it right is most of WireGuard configuration.

Be **aware** of **L2TP**, an older tunnelling protocol with no encryption of its own; it is almost always paired with IPsec as "L2TP/IPsec".

Reference and exam tips — 334.4

- **Routed = TUN = Layer 3 (IP); bridged = TAP = Layer 2 (Ethernet)**; remote-access vs site-to-site.
- **OpenVPN = TLS/userspace (/etc/openvpn/), TUN or TAP**; WireGuard = in-kernel, key-per-peer (`wg`, `wg-quick`, /etc/wireguard/), AllowedIPs = routes + ACL.
- **strongSwan = IPsec/IKEv2**; `swanctl` + `/etc/swanctl/swanctl.conf`, plus `/etc/strongswan.conf` and `/etc/strongswan.d/`.
- **L2TP has no encryption of its own — paired with IPsec**.

Topic 335 — Threats & Vulnerability Assessment

335.1 Common Security Vulnerabilities and Threats

Objective weight: 2

This objective is conceptual: you need to recognise the major categories of security threat and understand, in principle, how each works and what defends against it. The long list in the syllabus becomes far more manageable once you sort it by what the attacker is targeting — an individual host, the network, an application, or people and their credentials. Everything ties back to the defences built in the earlier topics.

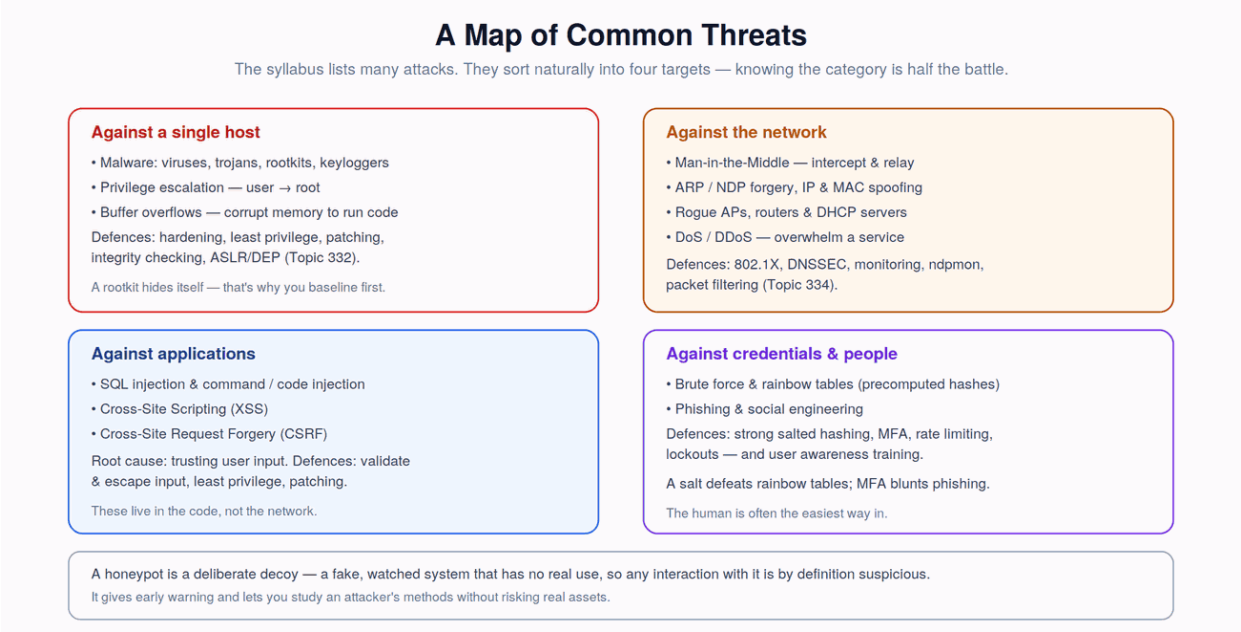


Figure 335.1 — The common threats grouped by target: host, network, application, and credentials/people.

1. Threats against an individual host

Malware comes in several forms with distinct behaviours. A **virus** attaches itself to a legitimate file and spreads when that file runs. A **trojan** masquerades as something useful to trick the user into running it. A **rootkit** embeds deep in the system and actively hides its own presence (which is why you baseline a clean system first, per Topic 332). A **keylogger** records keystrokes to steal passwords and other secrets.

Two host attacks are about turning limited access into full control. **Privilege escalation** exploits a bug or misconfiguration (a vulnerable SetUID binary, a weak sudo rule, a kernel flaw) to go from an ordinary user to root. It is often reached through a **buffer overflow**: the attacker sends more data than a program's buffer can hold, overwriting adjacent memory — including the return address — to redirect execution into their own injected code. The memory protections from Topic 332 (ASLR, DEP/NX) exist precisely to blunt this, and hardening, least privilege and patching are the broader defence.

2. Threats against the network

A **Man-in-the-Middle (MITM)** attacker sits between two parties, relaying and potentially altering their traffic while both believe they are talking directly. It is usually set up by lying about addresses: **ARP forgery** (IPv4) or **NDP forgery** (IPv6) poison the mapping between IP and hardware addresses, and **IP/MAC spoofing** impersonates a trusted host. An attacker may also plant a **rogue access point, router or DHCP server** to become the victim's default gateway. Separately, a **DoS** (Denial of Service) — or the distributed **DDoS** from many

sources — floods a service until it cannot serve legitimate users. The defences are Topic 334's: 802.1X, DNSSEC, RA Guard/ndpmon, monitoring and filtering.

3. Threats against applications

Application attacks nearly all share one root cause: **trusting user input**. **SQL injection** smuggles database commands through an input field that is concatenated into a query; **command/code injection** does the same for shell commands or interpreted code. **Cross-Site Scripting (XSS)** injects script that runs in other users' browsers (stored XSS lives in the site's data; reflected XSS bounces off a crafted link). **Cross-Site Request Forgery (CSRF)** tricks a logged-in victim's browser into silently submitting an attacker-chosen request, abusing the trust the site places in the browser's session. The defence is to validate and escape all input, use parameterised queries and anti-CSRF tokens, and apply least privilege so a breach is contained.

4. Threats against credentials and confidentiality

A **brute-force** attack simply tries many passwords until one works; **rainbow tables** speed this up enormously by pre-computing the hashes of huge password lists so a stolen hash can be reversed by lookup. The countermeasure is a **salt** — a unique random value mixed into each password before hashing — which makes pre-computation useless, together with a slow hash (bcrypt, Argon2), rate limiting and account lockouts. **Phishing** and **social engineering** bypass technology altogether, manipulating a person into revealing a secret or taking a harmful action; multi-factor authentication and user-awareness training are the main defences.

5. Honeypots

A **honeypot** is a decoy: a system (or service, or file) that has no legitimate purpose and is closely watched. Because no genuine user should ever interact with it, *any* activity against it is inherently suspicious — so it gives high-signal early warning of an intruder, and lets defenders study an attacker's tools and techniques without exposing real assets. A broader "honeynet" extends the idea to a whole decoy network.

Exam tips — 335.1

- **Malware families: virus (attaches), trojan (disguised), rootkit (self-hiding), keylogger**.
- **MITM enabled by ARP/NDP forgery, IP/MAC spoofing, and rogue AP/router/DHCP**; DoS/DDoS overwhelms a service.
- **App attacks come from trusting input: SQL/command/code injection, XSS, CSRF**.
- **Salt defeats rainbow tables; MFA + awareness blunt phishing/social engineering**.
- **A honeypot is a decoy where any activity is by definition suspicious**.

335.2 Penetration Testing

Objective weight: 3

Penetration testing is authorised, methodical attack simulation: deliberately trying to break into a system, with permission, in order to find and fix its weaknesses before a real attacker does. This objective covers the ethics and law, the phases of a test, the nmap scanner in depth, the architecture of Metasploit, and awareness of the wider tooling.

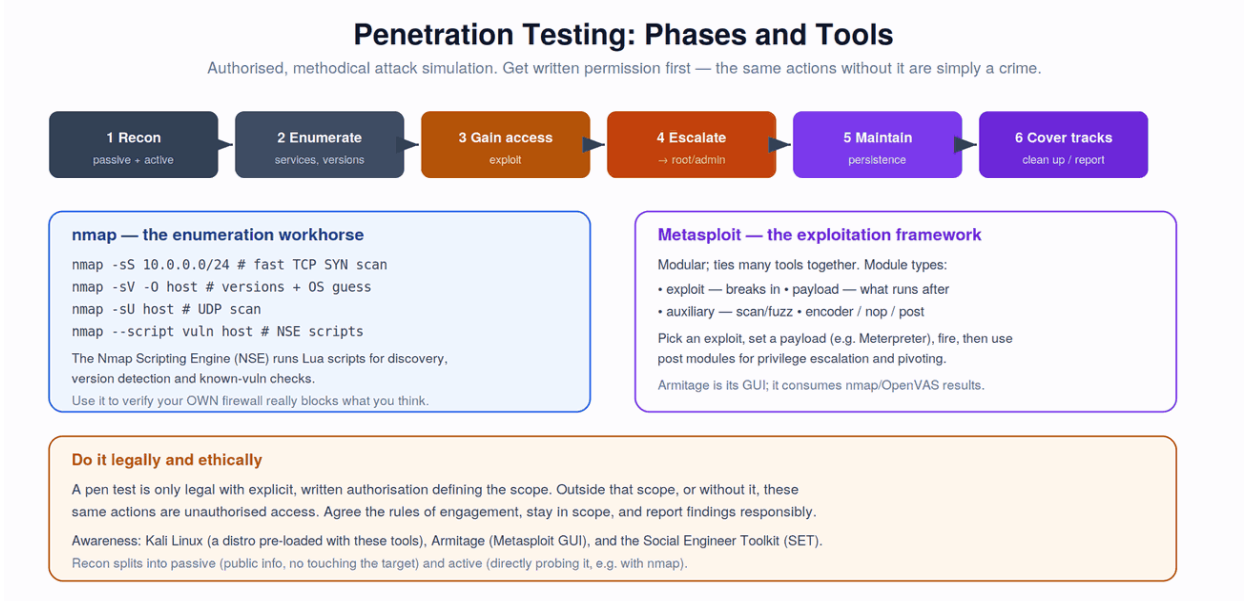


Figure 335.2a — The phases of a penetration test, and the two core tools.

1. Ethics and the law

The single most important point of this objective: a penetration test is only lawful with **explicit, written authorisation** that defines the **scope** — which systems, which techniques, over what window. Perform the very same actions outside that scope, or without authorisation, and they are **unauthorised access** — a crime in essentially every jurisdiction. Professional testing agrees "rules of engagement" up front (scope, timing, contacts, what to do on a real incident), stays strictly within them, avoids unnecessary damage, protects any data seen, and reports findings responsibly to the owner.

> **Authorisation is not a formality**
>
> "I was only testing" is not a defence. Ethical hacking and criminal hacking use identical tools and techniques; the *only* difference is prior, written permission and staying within the agreed scope.

2. The phases of a test

A test proceeds through recognised phases. **Information gathering (reconnaissance)** splits into *passive* — collecting public information (DNS records, search engines, social media) without ever touching the target — and *active* — directly probing it, for example with nmap. **Enumeration** catalogues services, versions, users and shares. **Gaining access** exploits a weakness to get a foothold. **Privilege escalation** turns that foothold into administrative control. **Maintaining access** establishes persistence (a backdoor or scheduled task). **Covering tracks** removes traces — though a legitimate test instead documents everything and restores the system, ending in an honest report.

3. nmap: scanning and enumeration

nmap is the principal tool of the reconnaissance and enumeration phases — and the tool the exam expects you to be able to drive. It discovers live hosts, finds open ports, identifies the service and version behind each, and can even guess the operating system. Its default scan is the **SYN ("half-open")** scan: nmap sends a SYN, and if the port answers SYN-ACK (open) it replies with RST instead of completing the handshake — faster and quieter than a full connection.

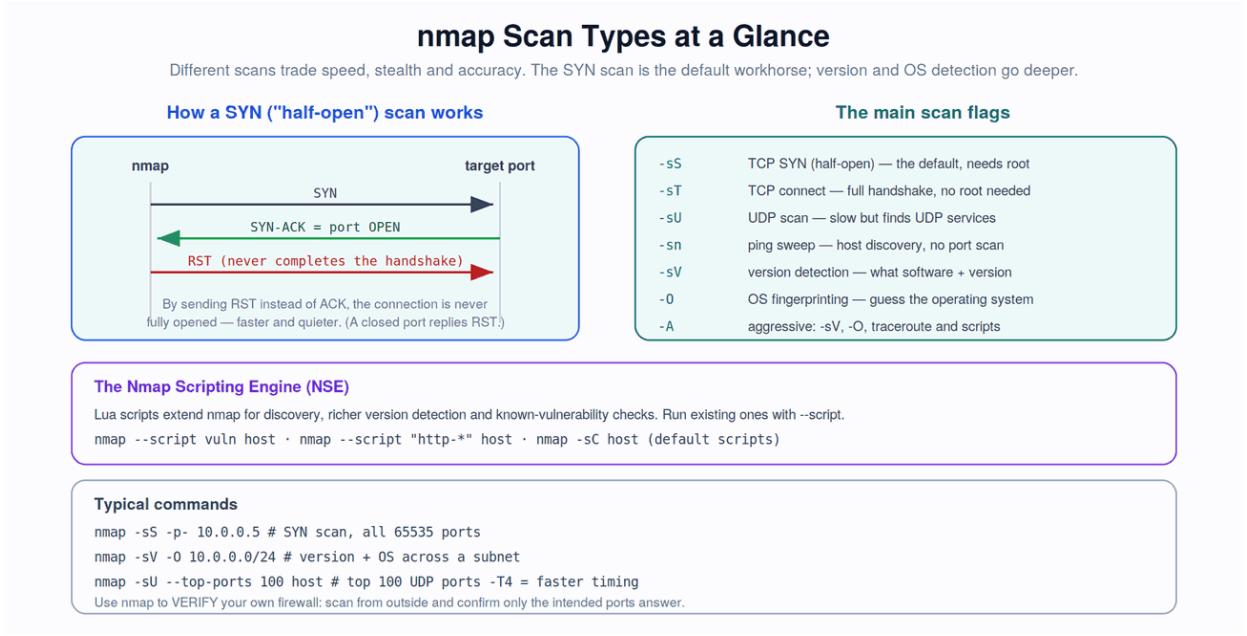


Figure 335.2b — How a SYN scan works, the main scan flags, and the Nmap Scripting Engine.

The scan types

Flag	Scan
`-sS`	TCP SYN / half-open — the default (needs root).
`-sT`	TCP connect — completes the handshake (no root needed).
`-sU`	UDP scan — slower, finds UDP services.
`-sn`	Ping sweep — host discovery only, no port scan.
`-sV`	Version detection — identify software and version.
`-O`	OS fingerprinting — guess the operating system.
`-A`	Aggressive — combines -sV, -O, traceroute and scripts.

```
$ nmap -sn 10.0.0.0/24              # which hosts are up
$ nmap -sS -p- 10.0.0.5             # SYN scan, all 65535 ports
$ nmap -sV -O 10.0.0.5             # versions + OS guess
$ nmap -sU --top-ports 100 10.0.0.5   # common UDP ports
$ nmap -T4 -A 10.0.0.5             # fast + aggressive
```

The Nmap Scripting Engine (NSE)

The **NSE** runs Lua scripts that extend nmap far beyond port scanning — service discovery, richer version probing, and checks for known vulnerabilities. You are expected to be able to *run existing scripts*, not write them.

```
$ nmap -sC 10.0.0.5                 # run the default script set
$ nmap --script vuln 10.0.0.5      # known-vulnerability scripts
$ nmap --script "http-*" 10.0.0.5  # all HTTP scripts
```

Use nmap to verify your OWN defences

The most legitimate everyday use of nmap is checking your own work: scan your host or firewall from the outside and confirm that *only* the ports you intended are open. If a scan reveals a service you did not mean to expose, your hardening (Topic 332) or filtering (Topic 334) has a gap.

4. Metasploit's architecture

Metasploit is a modular exploitation framework that ties many capabilities together behind one console (`msfconsole`). Understanding it means understanding its **module types**.

Module type	Purpose
`exploit`	Code that breaks into a target via a specific vulnerability.
`payload`	What runs after the exploit succeeds (e.g. a Meterpreter shell).
`auxiliary`	Scanners, fuzzers and other non-exploit tools.
`post`	Post-exploitation: privilege escalation, pivoting, data collection.
`encoder`	Reshape a payload to evade simple signature detection.
`nop`	No-operation generators used to pad payloads.

A typical flow: choose an `exploit`, set a `payload` (commonly **Meterpreter**, a powerful in-memory agent), set options (target address, port), and run it; then use `post` modules to escalate and pivot deeper. Metasploit **integrates other tools** — it can import nmap and OpenVAS results to pick relevant exploits, tying the reconnaissance and exploitation phases together.

```
$ msfconsole
msf6 > search type:exploit ssh
msf6 > use exploit/...
msf6 > set RHOSTS 10.0.0.5
msf6 > set PAYLOAD linux/x64/meterpreter/reverse_tcp
msf6 > exploit
```

5. Awareness: Kali, Armitage and SET

Be aware of the surrounding ecosystem. **Kali Linux** is a distribution pre-loaded with hundreds of penetration-testing tools (nmap, Metasploit, Wireshark, aircrack-ng and more) — the usual working environment for a tester. **Armitage** is a graphical front-end for Metasploit that visualises targets and sessions. The **Social Engineer Toolkit (SET)** automates human-focused attacks such as cloned phishing pages and malicious payloads, underscoring that people are often the easiest target.

Exam tips — 335.2

- **A pen test needs explicit written authorisation and a defined scope** — the same actions without it are illegal.
- **Phases: recon (passive/active) → enumeration → gaining access → privilege escalation → maintaining access → covering tracks**.
- **nmap: -sS SYN (default), -sT connect, -sU UDP, -sn ping sweep, -sV versions, -O OS, -A aggressive**; NSE via `--script`.
- **Metasploit module types: exploit, payload, auxiliary, post, encoder, nop**; Meterpreter is a payload; it imports nmap/OpenVAS results.
- **Kali = tool distro, Armitage = Metasploit GUI, SET = social-engineering toolkit** (awareness).